Riassunti e Appunti di Deep Learning

Appunti di corso



Università di Catania

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Triennale in Informatica (L-31)

Autore
Emanuele Galiano

Anno accademico
Anno Accademico 2026/2027

Versione: 11 novembre 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Deep Learning* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania dal Prof. **[Nome docente]** nell'anno accademico 2026/2027. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, teoremi, immagini ed esempi in un unico documento, e integrando gli appunti presi a lezione con le slide fornite dal docente.

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure, se ancora presente online, ad aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/Deep-Learning`

In particolare, all'interno del repository GitHub è presente il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo.

Licenza

Questo documento, intitolato *Riassunti e Appunti di Deep Learning – Appunti di corso*, è distribuito sotto licenza **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autore: Emanuele Galiano
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: 2026/2027

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.

- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution-ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti nell'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution-ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano

Indice

Licenza **v**

Diritti concessi . . . v
Condizioni . . . v
Assenza di garanzia . . . vi
Testo completo della licenza . . . vi

I Core Models and Methods **1**

1 Ripasso: dal Machine Learning alle reti neurali **3**

1.1 Gli ingredienti di un algoritmo di Machine Learning . . . 3
1.2 Regressione logistica . . . 4
1.3 Softmax . . . 5
1.3.1 Il modello . . . 5
1.3.2 Funzione di costo e learning . . . 6
1.3.3 Overparametrizzazione . . . 7
1.3.4 Il softmax come rete neurale . . . 7
1.4 Loss function e Multi-Class SVM . . . 8
1.5 Il ruolo della rappresentazione . . . 10
1.6 Reti neurali . . . 10
1.6.1 Multi-Layer Perceptron . . . 10
1.6.2 Le due parti di una rete neurale . . . 12
1.6.3 Funzioni di attivazione . . . 12
1.6.4 Il teorema di approssimazione universale . . . 13
1.6.5 Classificazione multiclasse e regressione . . . 14
1.7 Training e inferenza . . . 15
1.8 Come si apprendono i parametri? . . . 16

2 Backpropagation **17**

2.1 Il problema dell'apprendimento . . . 17
2.2 Grafi computazionali . . . 19
2.2.1 B-diagram . . . 20
2.2.2 Valutazione in due passi . . . 20
2.2.3 L'algoritmo sul grafo . . . 22
2.2.4 Il comportamento delle porte . . . 24
2.3 Backpropagation su una rete multistrato . . . 24

2.3.1 La funzione di costo . . . 24
2.3.2 La rete estesa . . . 25
2.3.3 Il calcolo nel singolo nodo . . . 25
2.3.4 L'ultimo layer: softmax e cross-entropy . . . 27
2.3.5 Forward e backward pass . . . 28
2.3.6 L'algoritmo di backpropagation . . . 30

3 Convolutional Neural Network 33
3.1 Dalle reti neurali alle immagini . . . 33
3.1.1 Representation learning . . . 33
3.1.2 MLP e immagini . . . 35
3.2 Convolutional Neural Network . . . 36
3.2.1 Architettura e componenti . . . 36
3.2.2 La convoluzione 2D . . . 37
3.2.3 Convoluzione 3D, padding, dilation e stride . . . 40
3.2.4 Local receptive field e shared weights . . . 43
3.2.5 Dimensione dell'output . . . 44
3.2.6 Un esempio reale: AlexNet . . . 45
3.3 Sparse connections e parameter sharing . . . 46
3.3.1 Sparse connections . . . 46
3.3.2 Parameter sharing . . . 46
3.3.3 Convoluzione efficiente . . . 47
3.4 I layer di una CNN . . . 48
3.4.1 Funzioni di attivazione . . . 48
3.4.2 Feature map . . . 49
3.4.3 Normalizzazione . . . 50
3.4.4 Pooling . . . 50
3.4.5 Fully connected layer . . . 52
3.5 Regole pratiche . . . 52

4 Addestrare una CNN 55
4.1 Dropout . . . 55
4.2 Batch normalization . . . 56
4.3 Data augmentation . . . 56
4.4 Transfer learning . . . 57
4.5 Quando conviene il fine-tuning . . . 57

5 Metric Learning 61
5.1 Algoritmi basati sulle distanze . . . 61
5.1.1 K-Nearest Neighbor . . . 61
5.1.2 K-means . . . 63
5.1.3 Misure di distanza . . . 63
5.2 Metric learning . . . 64
5.2.1 Supervised e weakly supervised . . . 65
5.2.2 Mahalanobis distance learning . . . 65

5.3 Deep metric learning . . . 66
5.3.1 L'idea di partenza . . . 66
5.3.2 Approcci contrastive . . . 66
5.3.3 Contrastive loss . . . 67
5.3.4 Il caso weakly supervised . . . 68
5.3.5 Siamese network . . . 69
5.3.6 Triplet loss . . . 70
5.3.7 Quadruplet loss . . . 71
5.3.8 Limiti degli approcci contrastive . . . 72
5.4 Center loss . . . 72

6 Modelli Generativi **75**
6.1 Supervised vs Unsupervised Learning . . . 75
6.1.1 Regola di Bayes applicata ai modelli generativi . . . 77
6.1.2 Tassonomia dei modelli generativi . . . 77
6.2 Modelli autoregressivi . . . 78
6.3 Autoencoder . . . 80
6.3.1 Un esempio: l'autoencoder lineare . . . 82
6.3.2 Deep autoencoder . . . 83
6.3.3 Training dell'autoencoder . . . 85
6.3.4 Undercomplete e overcomplete . . . 85
6.4 Autoencoder regolarizzati . . . 86
6.4.1 Sparse autoencoder . . . 86
6.4.2 Denoising autoencoder . . . 87
6.4.3 Contractive autoencoder . . . 87

Bibliografia **89**

Parte I

Core Models and Methods

Capitolo 1

Ripasso: dal Machine Learning alle reti neurali

Il Deep Learning non nasce dal nulla: le reti neurali profonde riutilizzano quasi tutti gli strumenti già visti nel Machine Learning classico, ovvero un modello parametrico, una funzione di loss da minimizzare, la discesa del gradiente e la regolarizzazione. In questo capitolo riprendiamo questi strumenti partendo dalla regressione logistica e dal modello softmax, mostriamo che il softmax può già essere visto come una rete neurale molto semplice e arriviamo, attraverso il problema della rappresentazione dei dati, al Multi-Layer Perceptron. Il capitolo si chiude con la domanda che motiva tutto il capitolo successivo: come si calcolano i gradienti di una rete con uno o più hidden layer?

1.1 Gli ingredienti di un algoritmo di Machine Learning

Qualunque algoritmo di Machine Learning supervisionato che vedremo è costruito a partire dagli stessi elementi:

- **Un modello**: una funzione h_ϑ che, dato un input x, restituisce una predizione.
- **Dei parametri**: il vettore (o la matrice) ϑ che determina il comportamento del modello e che va appreso dai dati.
- **Un set di dati**: il training set $\mathcal{D} = \{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, composto da m esempi etichettati.
- **Un algoritmo di learning**: la procedura che, minimizzando una funzione di costo $J(\vartheta)$, trova i parametri migliori per il modello.

Ogni input $x^{(i)}$ è un vettore di n feature. Per semplificare la notazione si aggiunge in testa una feature fittizia $x_0^{(i)} = 1$, detta **bias**, in modo che il termine noto del modello possa essere trattato come un parametro qualunque:

$$x^{(i)} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ \vdots \\ x_n^{(i)} \end{bmatrix} \in \mathbb{R}^{n+1}, \qquad x_0^{(i)} = 1$$

dove $x_1^{(i)}, \dots, x_n^{(i)}$ sono le feature vere e proprie dell'i-esimo esempio.

Discesa del gradiente. Come algoritmo di learning useremo quasi sempre la **discesa del gradiente**, ovvero un metodo iterativo che aggiorna i parametri nella direzione opposta al gradiente della funzione di costo:

$$\vartheta_j \leftarrow \vartheta_j - \alpha \frac{\partial J(\vartheta)}{\partial \vartheta_j} \qquad \forall j \text{ simultaneamente}$$

dove $\alpha > 0$ è il **learning rate**, un iperparametro che controlla l'ampiezza del passo. Valori tipici si scelgono su una scala approssimativamente logaritmica, ad esempio $0.1,\ 0.03,\ 0.01,\ 0.003,\ 0.001$: un α troppo grande rischia di far divergere l'algoritmo, mentre uno troppo piccolo rende la convergenza molto lenta.

Regolarizzazione. L'overfitting è uno dei problemi che si possono incontrare durante il training di un modello. Per contrastarlo si aggiunge alla funzione di costo un **termine di regolarizzazione**, che penalizza parametri di valore elevato:

$$J(\vartheta) = \underbrace{-\frac{1}{m}\Big[\cdots\Big]}_{\text{loss sui dati}} + \underbrace{\frac{\lambda}{2m}\sum_{j=1}^{n}\vartheta_j^2}_{\text{regolarizzazione}}$$

dove λ controlla l'intensità della penalizzazione e, tipicamente, il parametro del bias ϑ_0 non viene regolarizzato. Un λ grande porta a modelli più semplici, ma se è troppo grande il modello rischia di andare in underfitting.

1.2 Regressione logistica

La **regressione logistica** è il framework di learning per distinguere tra due classi, ovvero per la classificazione binaria con $y^{(i)} \in \{0, 1\}$. Il modello applica la funzione **sigmoide** a una combinazione lineare delle feature:

$$h_\vartheta(x) = \sigma(\vartheta^T x) = \frac{1}{1 + e^{-\vartheta^T x}}, \qquad \vartheta = [\vartheta_0, \vartheta_1, \dots, \vartheta_n]^T \in \mathbb{R}^{n+1}$$

e restituisce quindi un valore in $(0, 1)$, interpretabile come la probabilità $P(y = 1 \mid x)$.

La funzione di costo è la **cross-entropy loss** (o binary cross-entropy):

$$J(\vartheta) = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)} \log h_\vartheta(x^{(i)}) + (1 - y^{(i)}) \log\big(1 - h_\vartheta(x^{(i)})\big)\Big]$$

che penalizza fortemente le predizioni sicure ma sbagliate: se $y^{(i)} = 1$ e $h_\vartheta(x^{(i)}) \to 0$ (o viceversa) la loss diverge. I parametri si trovano con la discesa del gradiente e, se necessario, con la regolarizzazione vista sopra.

Per eseguire una classificazione multiclasse con la regressione logistica si può ricorrere all'approccio **one vs all**, addestrando un classificatore binario per ogni classe. Questa strategia presenta però diverse criticità: il numero di classificatori cresce con il numero di classi, possono

comparire regioni di incertezza e le probabilità restituite dai singoli classificatori non sono facilmente confrontabili. Per questo motivo si introduce un modello pensato direttamente per il caso multiclasse.

1.3 Softmax

Il **softmax** è il framework di learning per eseguire la classificazione multiclasse, ed è un'estensione della regressione logistica. Le etichette appartengono ora a K classi, $y^{(i)} \in \{0, 1, \dots, K-1\}$, e vengono rappresentate con il **one-hot encoding**, ovvero come un vettore di K componenti tutte nulle tranne quella corrispondente alla classe corretta, che vale 1.

1.3.1 Il modello

Dato un input x, vogliamo stimare la probabilità che appartenga a ciascuna delle classi:

$$P_\vartheta(y = k \mid x) \qquad \forall k = 0, 1, \dots, K-1.$$

La nostra ipotesi deve quindi stimare un vettore di K probabilità, e per farlo ogni classe k ha il proprio vettore di parametri $\vartheta^{(k)} \in \mathbb{R}^{n+1}$.

Definizione 1.1 (Modello softmax). Il modello softmax restituisce la distribuzione di probabilità sulle K classi:

$$h_\vartheta(x) = \begin{bmatrix} P_\vartheta(y = 0 \mid x) \\ P_\vartheta(y = 1 \mid x) \\ \vdots \\ P_\vartheta(y = K-1 \mid x) \end{bmatrix} = \frac{1}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T} x}} \begin{bmatrix} e^{\vartheta^{(0)T} x} \\ e^{\vartheta^{(1)T} x} \\ \vdots \\ e^{\vartheta^{(K-1)T} x} \end{bmatrix}$$

ovvero, per la singola classe c:

$$h_\vartheta^{(c)}(x) = P_\vartheta(y = c \mid x) = \frac{e^{\vartheta^{(c)T} x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T} x}}.$$

Il denominatore normalizza la distribuzione, per cui, una volta fissati (appresi) i parametri, la somma delle probabilità restituite è sempre pari a 1:

$$\sum_{k=0}^{K-1} P_\vartheta(y = k \mid x) = 1.$$

I parametri del modello non sono più un vettore ma una matrice, in cui la colonna k-esima

contiene i parametri $\vartheta^{(k)}$ della classe k:

$$\vartheta = \begin{bmatrix} \vartheta_0^{(0)} & \vartheta_0^{(1)} & \cdots & \vartheta_0^{(K-1)} \\ \vartheta_1^{(0)} & \vartheta_1^{(1)} & \cdots & \vartheta_1^{(K-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \vartheta_n^{(0)} & \vartheta_n^{(1)} & \cdots & \vartheta_n^{(K-1)} \end{bmatrix} \in \mathbb{R}^{(n+1)\times K}.$$

1.3.2 Funzione di costo e learning

Anche il softmax usa la cross-entropy loss, che confronta il vettore one-hot dell'etichetta con le probabilità uscite dal modello. Per scriverla definiamo la **funzione indicatrice**:

$$\mathbf{1}\{\text{proposizione}\} = \begin{cases} 1 & \text{se la proposizione è vera} \\ 0 & \text{se la proposizione è falsa} \end{cases}$$

La funzione di costo del softmax è quindi:

$$J(\vartheta) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{c=0}^{K-1} \mathbf{1}\{y^{(i)} = c\} \log h_\vartheta^{(c)}(x^{(i)})\right] + \underbrace{\frac{\lambda}{2m}\sum_{c=0}^{K-1}\sum_{j=1}^{n}\left(\vartheta_j^{(c)}\right)^2}_{\text{regolarizzazione}}$$

dove, per ogni esempio, sopravvive solo il termine della classe corretta: la loss è quindi $-\log$ della probabilità assegnata alla classe giusta.

L'obiettivo del learning è trovare i parametri che minimizzano $J(\vartheta)$, cosa che si fa con la discesa del gradiente (o con la sua versione stocastica). La derivata parziale rispetto al generico parametro vale:

$$\frac{\partial J(\vartheta)}{\partial \vartheta_j^{(c)}} = -\frac{1}{m}\sum_{i=1}^{m}\left[\left(\mathbf{1}\{y^{(i)} = c\} - h_\vartheta^{(c)}(x^{(i)})\right) x_j^{(i)}\right] + \frac{\lambda}{m}\vartheta_j^{(c)}$$

e l'aggiornamento viene ripetuto fino a convergenza:

$$\vartheta_j^{(c)} \leftarrow \vartheta_j^{(c)} - \alpha\frac{\partial J(\vartheta)}{\partial \vartheta_j^{(c)}} \qquad \forall j = 0, \ldots, n,\ \forall c = 0, \ldots, K-1 \text{ simultaneamente.}$$

Osservazione (Gradiente rispetto ai logit). Chiamiamo **logit** le quantità $z_k = \vartheta^{(k)T}x$, ovvero i valori prima della normalizzazione. Per un singolo esempio, il gradiente della cross-entropy rispetto a ciascun logit ha una forma molto semplice:

$$\delta_k = \frac{\partial L}{\partial z_k} = p_k - y_k$$

dove p_k è la probabilità predetta e y_k la componente k-esima del vettore one-hot. Il gradiente rispetto al peso che collega la feature x_j al nodo k è poi semplicemente $\delta_k x_j$, mentre quello rispetto al bias è δ_k. È la stessa espressione della formula precedente, letta nodo per nodo: questa idea di *errore locale* propagato all'indietro tornerà nel capitolo 2.

1.3.3 Overparametrizzazione

Il modello softmax è **overparametrizzato**, ovvero, per ogni ipotesi che descrive i dati, esistono infinite configurazioni dei parametri che producono esattamente la stessa mappa dagli input x alle predizioni. Infatti, sottraendo a ogni vettore $\vartheta^{(k)}$ lo stesso vettore arbitrario $\psi \in \mathbb{R}^{n+1}$:

$$\frac{e^{(\vartheta^{(c)}-\psi)^T x}}{\sum_{k=0}^{K-1} e^{(\vartheta^{(k)}-\psi)^T x}} = \frac{e^{\vartheta^{(c)T}x}\, e^{-\psi^T x}}{e^{-\psi^T x}\sum_{k=0}^{K-1} e^{\vartheta^{(k)T}x}} = \frac{e^{\vartheta^{(c)T}x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T}x}}$$

il fattore comune $e^{-\psi^T x}$ si semplifica e l'ipotesi h_ϑ non cambia.

Relazione con la regressione logistica. Questa proprietà permette di mostrare facilmente che, nel caso particolare $K = 2$, il softmax si riduce alla regressione logistica. Scegliendo $\psi = \vartheta^{(1)}$:

$$h_\vartheta(x) = \begin{bmatrix} \dfrac{e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x}}{e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x} + e^{\mathbf{0}^T x}} \\ \dfrac{e^{\mathbf{0}^T x}}{e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x} + e^{\mathbf{0}^T x}} \end{bmatrix} = \begin{bmatrix} 1 - \dfrac{1}{1 + e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x}} \\ \dfrac{1}{1 + e^{(\vartheta^{(0)}-\vartheta^{(1)})^T x}} \end{bmatrix}$$

dove si è usato $e^{\mathbf{0}^T x} = 1$. La differenza $\vartheta^{(1)} - \vartheta^{(0)}$ può essere sostituita da un unico vettore di parametri ϑ, ottenendo proprio $h^{(1)}_\vartheta(x) = \sigma(\vartheta^T x)$ e $h^{(0)}_\vartheta(x) = 1 - h^{(1)}_\vartheta(x)$. Allo stesso modo, per $K = 2$ e senza regolarizzazione, la funzione di costo del softmax diventa:

$$J(\vartheta) = -\frac{1}{m}\sum_{i=1}^{m}\left[(1 - y^{(i)}) \log\left(1 - h^{(1)}_\vartheta(x^{(i)})\right) + y^{(i)} \log h^{(1)}_\vartheta(x^{(i)})\right]$$

ovvero la cross-entropy della regressione logistica: la regressione logistica è quindi un caso particolare del softmax.

1.3.4 Il softmax come rete neurale

Il softmax segue un modello lineare, tuttavia può essere rappresentato come una rete neurale molto semplice, con un solo layer di pesi tra input e output. I suoi *ingredienti* sono tre:

1. **Modello lineare**: per ogni classe k si calcola $z_k = \vartheta^{(k)T} x$.
2. **Logit**: si applica l'esponenziale, e^{z_k}.
3. **Normalizzazione**: si divide per $\sum_j e^{z_j}$, ottenendo probabilità che sommano a 1.

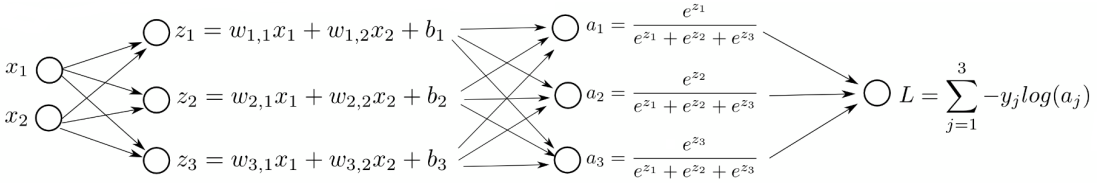


Figura 1.1. Il softmax come rete neurale con due feature e tre classi: dai logit $z_i = w_{i,1}x_1 + w_{i,2}x_2 + b_i$ alle probabilità $a_i = e^{z_i} / \sum_{k=1}^{3} e^{z_k}$, fino alla cross-entropy loss $L = \sum_{j=1}^{3} -y_j \log(a_j)$. Il bias b_i corrisponde al parametro $\vartheta_0^{(i)}$ associato alla feature $x_0 = 1$.

Esempio. Consideriamo tre classi e un input con due feature, $x = [1,\ 0.2,\ 0.1]^T$ (la prima componente è il bias). Siano i parametri, scritti per righe (una riga per classe):

$$\vartheta^T = \begin{bmatrix} -1 & 1 & 0 \\ 1 & 0.5 & 1 \\ 2 & 1 & 1.5 \end{bmatrix}$$

e supponiamo che la classe corretta sia la prima, $y = [1, 0, 0]^T$.

Soluzione. Calcoliamo in ordine i tre ingredienti del softmax:

Classe	Modello lineare z_k	Logit e^{z_k}	Normalizzazione p_k
0	$-1 + 0.2 + 0 = -0.8$	0.45	0.03
1	$1 + 0.1 + 0.1 = 1.2$	3.32	0.23
2	$2 + 0.2 + 0.15 = 2.35$	10.49	0.74
		$\sum = 14.26$	$\sum = 1$

Il modello assegna alla classe corretta una probabilità di appena 0.03, per cui la loss su questo campione è alta: $L = -1 \cdot \log(0.0315) \approx 3.46$. La loss complessiva si ottiene facendo la media di questi valori su tutti gli m campioni del training set.

1.4 Loss function e Multi-Class SVM

Ci si potrebbe chiedere se anche una SVM sia una rete neurale. Una SVM lineare, come il softmax, è una **macchina lineare**: calcola un vettore di punteggi $f = Wx + b$, uno per classe. Ciò che cambia tra i due modelli è soltanto la funzione di loss: cambiando la loss di questo framework lineare si ottiene la **Multi-Class SVM**, che usa la **hinge loss**:

$$L = \underbrace{\frac{1}{N} \sum_i \sum_{j \neq y_i} \max(0,\ f_j - f_{y_i} + 1)}_{\text{data loss}} + \underbrace{\lambda \sum_k \sum_l W_{k,l}^2}_{\text{regularization loss}}$$

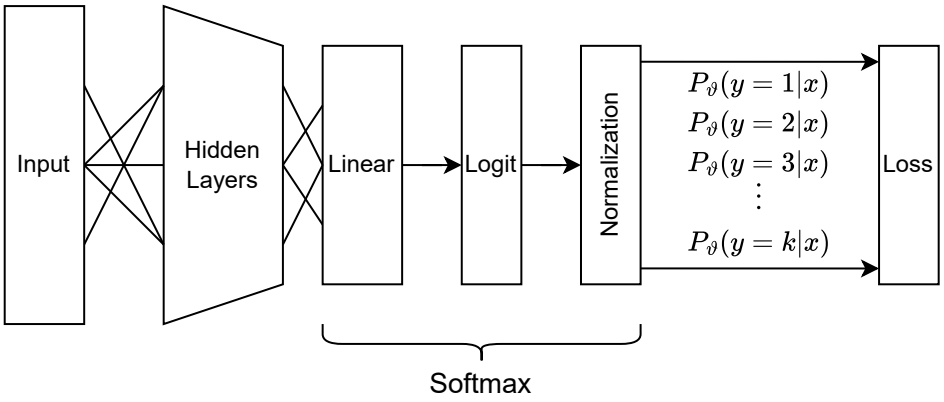


Figura 1.2. Struttura "standard" di una rete neurale per la classificazione: dopo l'input e gli hidden layer, gli ultimi blocchi (modello lineare, logit e normalizzazione) sono esattamente un softmax, che produce le probabilità $P_\vartheta(y = k \mid x)$ su cui viene calcolata la loss.

dove f_{y_i} è il punteggio della classe corretta. In questo senso una SVM lineare può essere vista come la stessa rete a un solo layer del softmax, con una loss diversa; tipicamente, però, il termine "rete neurale" si riserva a modelli che apprendono anche una rappresentazione degli input, come vedremo tra poco.

Esempio. Siano $f = [-2.85,\ 0.86,\ 0.28]^T$ i punteggi prodotti da $Wx + b$ per un esempio la cui classe corretta è $y_i = 2$, ovvero quella con punteggio 0.28.

Soluzione. Con la hinge loss si sommano i contributi delle classi errate:

$$L_{\text{hinge}} = \max(0,\ -2.85 - 0.28 + 1) + \max(0,\ 0.86 - 0.28 + 1) = 0 + 1.58 = 1.58.$$

Con la cross-entropy, invece, si applicano esponenziale e normalizzazione,

$$e^f = [0.058,\ 2.36,\ 1.32]^T \quad \Longrightarrow \quad p = [0.015,\ 0.631,\ 0.353]^T,$$

e si prende il logaritmo della probabilità della classe corretta: $L_{\text{CE}} = -\log(0.353) \approx 1.04$.

Loss per la classificazione binaria. In generale, la loss complessiva è la somma delle loss sui singoli esempi, $L = \sum_{i=1}^{m} \text{loss}\big(f(x^{(i)}), y^{(i)}\big)$. Nel caso binario, con $y^{(i)} \in \{-1, +1\}$, alcuni esempi sono:

- **Zero-one loss**: $\mathbf{1}\{\text{sign}(f(x^{(i)})) \neq y^{(i)}\}$, che conta gli errori ma non è né continua né derivabile.
- **Hinge loss**: $\max\big(0,\ 1 - f(x^{(i)})\, y^{(i)}\big)$, usata dalle SVM.
- **Logistic loss**: $\log\big(1 + \exp(-f(x^{(i)})\, y^{(i)})\big)$, usata dalla regressione logistica.

Allo stesso modo, la regressione lineare usa la **least squares loss** $\frac{1}{2}(y - z)^2$, mentre le reti neurali per la classificazione usano la cross-entropy. In tutti i casi la funzione di costo da minimizzare con la discesa del gradiente è $J(\vartheta) = \sum_{i=1}^{m} L\big(h_\vartheta(x^{(i)}), y^{(i)}\big)$, e il modello restituisce un valore continuo nella regressione o una classe nella classificazione.

1.5 Il ruolo della rappresentazione

Un modello lineare come il softmax produce sempre decision boundary lineari: funziona bene quando i dati sono **linearmente separabili**, ma fallisce quando non lo sono, come nel caso di due classi disposte a cerchi concentrici. Una soluzione nota è quella di trasformare le feature con una funzione $\phi(x)$, ad esempio un kernel, che mappa i dati in uno spazio a dimensione più alta in cui diventano separabili da un iperpiano (figura 1.3).

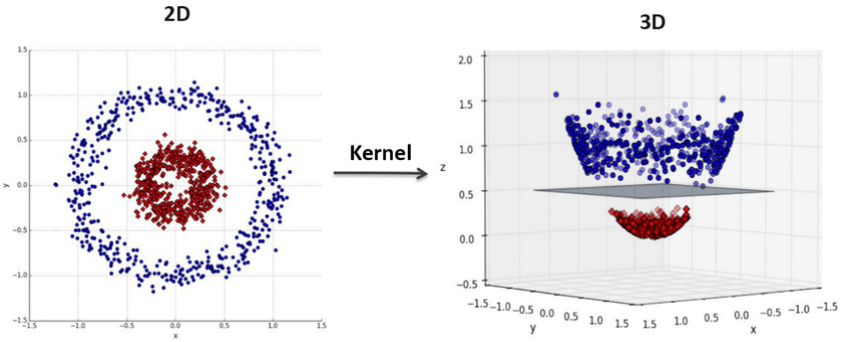


Figura 1.3. Dati non linearmente separabili nello spazio di input bidimensionale (a sinistra): due classi disposte a cerchi concentrici. Applicando una trasformazione ϕ che aggiunge una dimensione (a destra), le due classi diventano separabili da un piano nello spazio delle feature.

Il problema è che scegliere la $\phi(x)$ giusta non è semplice: dipende dai dati e dal task, e una rappresentazione costruita a mano (*hand crafted*) potrebbe non essere quella migliore. La domanda diventa allora: possiamo **apprendere** la funzione $\phi(x)$ direttamente dai dati? La risposta sono le reti neurali e il Deep Learning, che fanno **representation learning**, ovvero imparano in modo supervisionato, a seconda del task, come trasformare l'input in una rappresentazione utile.

1.6 Reti neurali

La prima rete neurale studiata fu il **percettrone** di Rosenblatt, ovvero un singolo neurone che calcola una somma pesata degli input (con la soglia incorporata come peso ϑ_0 del bias $x_0 = 1$) e restituisce un'uscita binaria. Il percettrone è dotato di una procedura di learning automatico dei pesi, ma, come tutti i modelli lineari, risolve solo problemi linearmente separabili: non è in grado, ad esempio, di calcolare lo XOR.

1.6.1 Multi-Layer Perceptron

Il **Multi-Layer Perceptron** (MLP) nasce da due motivazioni:

1. Dobbiamo adattare il modello a dati che vivono in uno spazio non lineare, come nel problema dello XOR.

2. Dato un insieme di dati (ad esempio immagini o documenti di testo), dobbiamo imparare una buona rappresentazione di essi, e non usarne una costruita a mano.

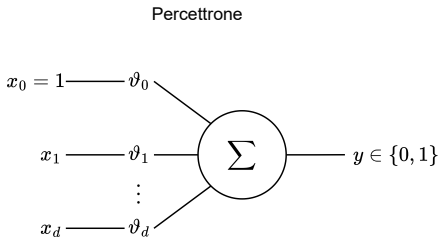


Figura 1.4. Il percettrone di Rosenblatt: le feature $x_1, \dots, x_d$ e il bias $x_0 = 1$ vengono pesati dai parametri $\vartheta_0, \dots, \vartheta_d$ e sommati; l'uscita è binaria, $y \in \{0, 1\}$.

Un MLP è organizzato a strati: un **input layer**, uno o più **hidden layer** e un **output layer**, in cui ogni neurone di un layer è connesso a tutti i neuroni del layer successivo (figura 1.5). Ogni layer, ad eccezione dell'output, contiene anche un'unità di bias che vale sempre 1.

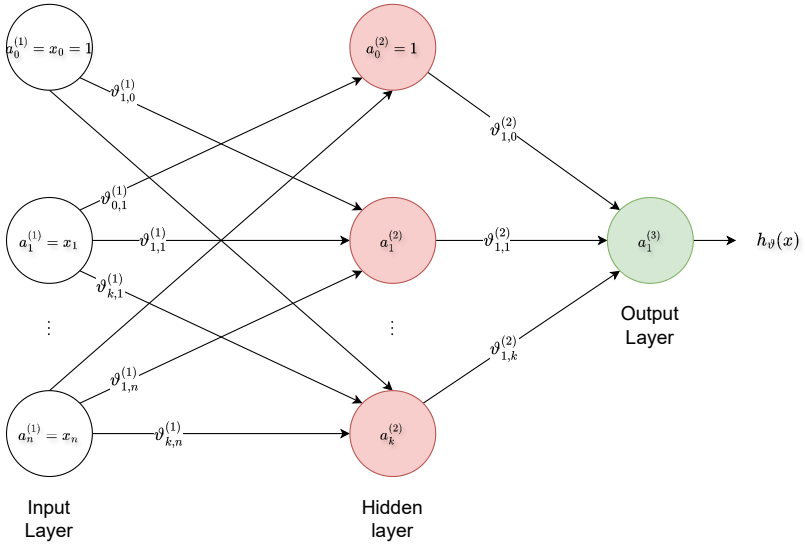


Figura 1.5. Un Multi-Layer Perceptron con un input layer (in bianco, $a_i^{(1)} = x_i$), un hidden layer con k neuroni (in rosso) e un output layer con un solo neurone (in verde) che restituisce $h_\vartheta(x)$. Le unità $a_0^{(1)}$ e $a_0^{(2)}$ sono i bias, sempre pari a 1; i pesi $\vartheta_{ji}^{(l)}$ sono riportati sugli archi.

Notazione. Indichiamo con:

- $a_i^{(l)}$ l'**attivazione** della i-esima unità del layer l; nell'input layer $a_i^{(1)} = x_i$.
- $\vartheta_{ji}^{(l)}$ il parametro che collega l'unità i del layer l (di partenza) all'unità j del layer $l + 1$ (di arrivo).
- $\vartheta^{(l)}$ la matrice dei parametri che mappa il layer l nel layer $l + 1$.

Ogni neurone calcola una combinazione lineare delle attivazioni del layer precedente e vi

applica una **funzione di attivazione** f:

$$z_j^{(l)} = \sum_i \vartheta_{ji}^{(l-1)}\, a_i^{(l-1)} = \vartheta_{j0}^{(l-1)} a_0^{(l-1)} + \vartheta_{j1}^{(l-1)} a_1^{(l-1)} + \dots, \qquad a_j^{(l)} = f\big(z_j^{(l)}\big)$$

o, in forma matriciale, $z^{(l)} = \vartheta^{(l-1)} a^{(l-1)}$ e $a^{(l)} = f(z^{(l)})$, dove f è applicata elemento per elemento. Nel caso della figura 1.5, l'output della rete è quindi:

$$\begin{aligned} h_\vartheta(x) = a_1^{(3)} &= f\big(\vartheta_{10}^{(2)} + \vartheta_{11}^{(2)} a_1^{(2)} + \dots + \vartheta_{1k}^{(2)} a_k^{(2)}\big) \\ &= f\Big(\vartheta_{10}^{(2)} + \vartheta_{11}^{(2)} \underbrace{f\big(\vartheta_{10}^{(1)} + \dots + \vartheta_{1n}^{(1)} a_n^{(1)}\big)}_{a_1^{(2)}} + \dots + \vartheta_{1k}^{(2)} \underbrace{f\big(\vartheta_{k0}^{(1)} + \dots + \vartheta_{kn}^{(1)} a_n^{(1)}\big)}_{a_k^{(2)}}\Big) \end{aligned}$$

Una rete neurale è quindi una **composizione di funzioni derivabili**.

Numero di parametri. Se la rete ha s_l unità nel layer l e s_{l+1} unità nel layer $l+1$, la matrice $\vartheta^{(l)}$ ha dimensione $s_{l+1} \times (s_l + 1)$, dove il $+1$ tiene conto dell'unità di bias. Ad esempio, una rete con 2 feature in input, un hidden layer con 3 neuroni e un output layer con 2 neuroni ha $\vartheta^{(1)} \in \mathbb{R}^{3\times 3}$ e $\vartheta^{(2)} \in \mathbb{R}^{2\times 4}$, per un totale di $9 + 8 = 17$ parametri.

1.6.2 Le due parti di una rete neurale

Una rete neurale per la classificazione è composta da due parti (figura 1.6):

- **Feature learning**: la prima parte della rete impara, in modo supervisionato e a seconda del task, come trasformare l'input x in una rappresentazione utile, nell'esempio $a^{(2)}$.
- **Classificatore**: l'ultima parte è un softmax (o una regressione logistica, nel caso binario) che prende in input le feature apprese.

Si possono aggiungere tutti gli hidden layer necessari per adattare la rete a un problema non lineare, rendendola **profonda** (*deep*). Hidden layer diversi possono avere un numero diverso di neuroni. La funzione di attivazione, invece, è in genere la stessa per tutti i neuroni (si possono usare funzioni diverse, ma si tratta di casi particolari). Al crescere della dimensione dell'input aumenta la probabilità che lo spazio sia non lineare, e può quindi servire aggiungere layer, neuroni per layer, o entrambi.

1.6.3 Funzioni di attivazione

Le funzioni di attivazione **devono essere derivabili**: in questo modo la rete, essendo una composizione di funzioni derivabili, è a sua volta derivabile, e la si può addestrare con la discesa del gradiente. Le più usate sono:

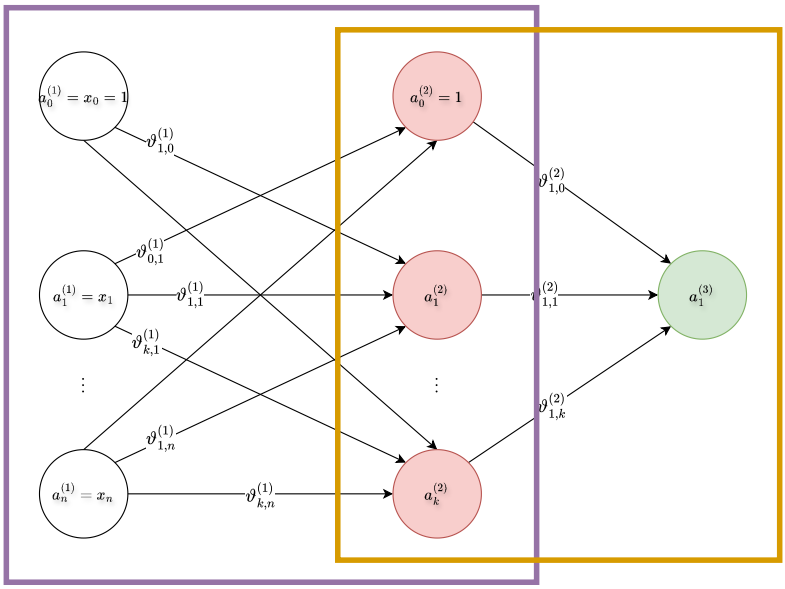


Figura 1.6. Le due parti di una rete neurale: il riquadro viola evidenzia la trasformazione appresa dell'input x nelle feature $a^{(2)}$, mentre il riquadro arancione evidenzia il classificatore (softmax o regressione logistica) che usa le feature apprese.

Nome	Funzione	Derivata
Identità	$f(x) = x$	$f'(x) = 1$
Sigmoide (logistica)	$f(x) = \dfrac{1}{1+e^{-x}}$	$f'(x) = f(x)\big(1 - f(x)\big)$
Tangente iperbolica	$f(x) = \tanh(x) = \dfrac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ReLU	$f(x) = \max(0, x)$	$f'(x) = \mathbf{1}\{x \geq 0\}$
Softplus	$f(x) = \log(1 + e^x)$	$f'(x) = \dfrac{1}{1+e^{-x}}$

1.6.4 Il teorema di approssimazione universale

Il motivo per cui le reti neurali funzionano così bene su problemi non lineari è formalizzato dal seguente risultato.

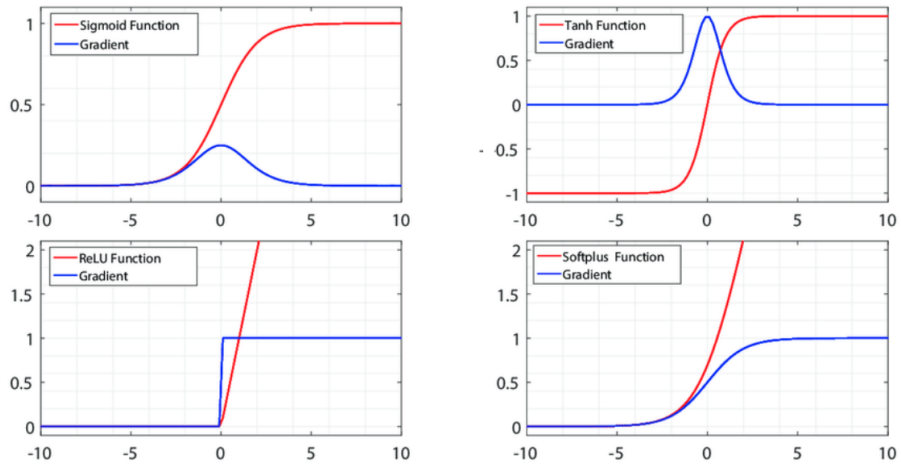


Figura 1.7. Le funzioni di attivazione più usate (in rosso) e le relative derivate (in blu): sigmoide (in alto a sinistra), tangente iperbolica (in alto a destra), ReLU (in basso a sinistra) e softplus (in basso a destra). Si noti come la derivata della softplus sia proprio la sigmoide.

Teorema 1.1 (Approssimazione universale). Una rete neurale feedforward[a] con un solo hidden layer, dotata di un numero finito ma sufficientemente grande di neuroni e di una funzione di attivazione non lineare e non polinomiale (ad esempio la sigmoide), è in grado di approssimare qualunque funzione continua $f : C \to \mathbb{R}$ definita su un insieme compatto $C \subset \mathbb{R}^n$ con errore arbitrariamente piccolo, ovvero per ogni $\varepsilon > 0$ esiste una rete che implementa $\hat{f}$ tale che

$$\sup_{x \in C} \left| f(x) - \hat{f}(x) \right| < \varepsilon.$$

[a] Per feedforward si intende una rete in cui ogni layer riceve input esclusivamente dal layer precedente e lo trasmette al successivo, senza cicli.

Il teorema garantisce quindi che un solo hidden layer è sufficiente a *rappresentare* qualunque funzione continua, ma non dice nulla su quanto questo sia praticabile: come osservato in [7], il layer potrebbe dover essere enormemente grande e la rete potrebbe non riuscire ad apprendere e generalizzare correttamente. Il teorema non fornisce infatti né un metodo per trovare i parametri, né un limite al numero di neuroni necessari. In pratica, l'uso di più hidden layer permette spesso di rappresentare le stesse funzioni in modo più compatto: questa è una delle motivazioni delle reti *deep*.

1.6.5 Classificazione multiclasse e regressione

Le reti neurali permettono di addestrare ipotesi complesse e non lineari sullo spazio delle feature, a seconda del task:

- **Classificazione binaria**: basta un solo neurone di output, con $y^{(i)} \in \{0, 1\}$.
- **Classificazione multiclasse**: con $K \geq 3$ classi si usano K unità di output e le etichette sono vettori one-hot, $y^{(i)} \in \{0, 1\}^K$, con un solo 1 nella posizione corrispondente alla classe. Come visto in figura 1.2, l'output layer è un softmax.

- **Regressione**: si eliminano logit e normalizzazione, si usa una regressione lineare nell'ultimo layer (ogni nodo di output è un regressore lineare) e si cambia la funzione di costo, usando la **MSE loss**, che deve tenere conto dell'output di tutti i regressori (figura 1.8).

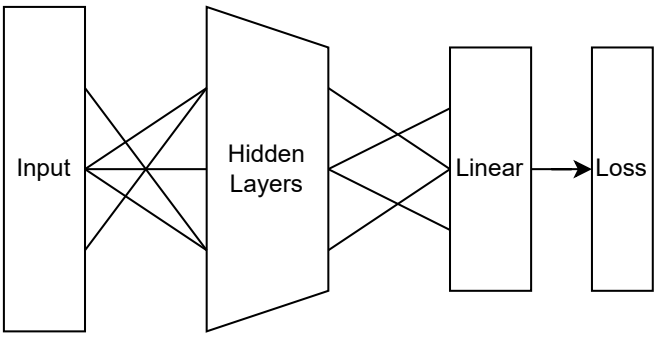


Figura 1.8. Struttura di una rete neurale per la regressione: dopo gli hidden layer l'ultimo blocco è un modello lineare, la cui uscita viene confrontata con il target tramite la MSE loss.

1.7 Training e inferenza

Supponiamo di aver definito un'architettura. Durante la fase di **training** l'obiettivo è imparare i parametri migliori per il modello, e per farlo servono una funzione di loss e un algoritmo che li apprenda. Una volta ottenuti i parametri, questi vengono usati per fare **inferenza** su nuovi pattern.

Funzione di costo. Per la classificazione vogliamo trovare $\hat{\vartheta}$ tale che

$$\hat{\vartheta} = \arg\min_{\vartheta} J(\vartheta), \qquad J(\vartheta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{c=0}^{K-1} \mathbf{1}\{y^{(i)} = c\} \log h_{\vartheta}^{(c)}(x^{(i)})$$

ovvero la somma delle cross-entropy di tutti i neuroni di output. Per raggiungere l'obiettivo dobbiamo essere in grado di calcolare in modo efficiente

$$\frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}} \qquad \forall j, i, l.$$

Overfitting. Le dimensioni della rete determinano un trade-off. Una rete "piccola" ha pochi parametri, è computazionalmente più economica ma è più soggetta a underfitting. Una rete "grande" ha molti parametri, è più soggetta a overfitting ed è computazionalmente più costosa; in questo caso si usa la regolarizzazione (λ) per contrastare l'overfitting.

Inizializzazione dei parametri. Come inizializzare la matrice dei parametri ϑ? Se si assegnassero gli stessi valori ai pesi che partono dallo stesso nodo, i nodi di arrivo calcolerebbero esattamente le stesse cose e riceverebbero gli stessi gradienti, comportandosi come un unico neurone. Per questo motivo si effettua il **symmetry breaking**: i parametri che dipendono dallo

stesso nodo devono essere inizializzati con valori diversi e piccoli, tipicamente scelti casualmente in un intervallo $\vartheta_{ji}^{(l)} \in [-\varepsilon, \varepsilon]$.

1.8 Come si apprendono i parametri?

Arriviamo così al problema centrale: come si apprendono i parametri di una rete neurale con un numero di hidden layer ≥ 1? Riepilogando quanto visto finora:

- **Un solo neurone**: si usa la discesa del gradiente, evoluzione dell'algoritmo di learning di Rosenblatt.
- **Softmax**: si usa la discesa del gradiente, perché la loss agisce su un solo layer di pesi.
- **Rete con hidden layer**: i valori dei nodi nascosti non sono noti in anticipo, ovvero il training set non fornisce alcuna ground truth per i layer intermedi.

Di conseguenza non possiamo più usare la discesa del gradiente nella forma classica, ma dobbiamo rivedere il modo in cui calcoliamo le derivate rispetto ai parametri. La soluzione si basa sul fatto che le reti neurali sono **grafi computazionali**, su cui si può usare la **chain rule** per spostare i gradienti all'indietro: è l'algoritmo di **backpropagation**, oggetto del prossimo capitolo.

In sintesi, il softmax risolve la classificazione multiclasse ma resta un modello lineare; le reti neurali imparano anche la rappresentazione dei dati e possono approssimare funzioni non lineari arbitrarie, al prezzo di dover calcolare i gradienti rispetto a parametri di layer di cui non conosciamo l'uscita attesa.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (13 ottobre 2026).
- Slide del corso *Softmax – Generalization of Logistic Regression* e *Neural Network – Multi-Layer Perceptron & Deep Learning*.
- Riassunto dei capitoli 2, 4 e 6 delle dispense di *Machine Learning*, da cui sono tratte anche tutte le figure di questo capitolo: `https://github.com/emanuelegaliano/Machine-Learning`.
- Teorema di approssimazione universale dal libro *Deep Learning* [7], sezione 6.4.1.

Capitolo 2

Backpropagation

Nel capitolo precedente abbiamo visto che una rete neurale con almeno un hidden layer non può essere addestrata con la discesa del gradiente nella forma classica, perché il training set non fornisce alcuna ground truth per i layer intermedi. Eppure la discesa del gradiente ha bisogno, per ogni parametro della rete, della derivata della funzione di costo rispetto a quel parametro. La **backpropagation** è l'algoritmo che permette di calcolare tutte queste derivate in modo efficiente, sfruttando il fatto che una rete neurale è un **grafo computazionale** e che sui grafi computazionali la chain rule si applica in modo del tutto meccanico. In questo capitolo introduciamo i grafi computazionali e il modo in cui si valutano in avanti e all'indietro, poi applichiamo questi strumenti a una rete multistrato per arrivare all'algoritmo di backpropagation vero e proprio.

2.1 Il problema dell'apprendimento

Consideriamo una rete come quella in figura 2.1, con 2 feature in input (estese con il bias), un hidden layer con 3 neuroni (più il bias) e un output layer con 2 neuroni. Indichiamo con ϑ l'insieme di tutti i parametri della rete, ovvero le matrici $\vartheta^{(1)} \in \mathbb{R}^{3\times 3}$ e $\vartheta^{(2)} \in \mathbb{R}^{2\times 4}$.

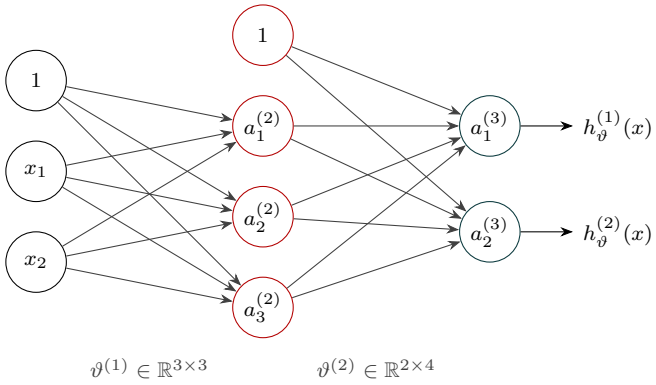


Figura 2.1. Una rete con 2 feature in input, un hidden layer con 3 neuroni (in rosso) e un output layer con 2 neuroni (in verde petrolio); i nodi con valore 1 sono le unità di bias. La matrice $\vartheta^{(1)}$ mappa l'input nell'hidden layer, la matrice $\vartheta^{(2)}$ mappa l'hidden layer nell'output.

Feedforward propagation. Prendendo ad esempio la figura 2.1, abbiamo una rete già trainata con cui fare inferenza (quindi con i pesi ϑ fissati). Dato un input $x = [x_1, x_2]^T$, l'output della rete si calcola propagando in avanti le attivazioni dei neuroni. Le attivazioni del layer di input sono $a_0^{(1)} = x_0 = 1$, $a_1^{(1)} = x_1$ e $a_2^{(1)} = x_2$. Per calcolare le attivazioni dei layer successivi, si moltiplica la matrice dei pesi per il vettore delle attivazioni del layer precedente e si applica la funzione di attivazione f:

$$a_j^{(2)} = f\Big(\sum_{i=0}^{2} \vartheta_{j,i}^{(1)} a_i^{(1)}\Big), \qquad j = 1, 2, 3$$

dove rappresentiamo con $a_i^{(1)}$ le attivazioni del layer di input.

Volendo generalizzare questa forma, dobbiamo considerare una rete con L layer, dove il layer l ha s_l unità. Quindi il layer $l+1$ ha s_{l+1} unità, e la matrice dei pesi $\vartheta^{(l)}$ ha dimensione $s_{l+1} \times (s_l + 1)$, dove il +1 è per il bias. Le attivazioni del layer $l+1$ si calcolano come:

$$a_j^{(l+1)} = f\Big(\sum_{i=0}^{s_l} \vartheta_{j,i}^{(l)} a_i^{(l)}\Big), \qquad j = 1, \dots, s_{l+1},$$

con $a_0^{(l)} = 1$ per ogni l (unità di bias). In forma vettoriale, indicando con $a^{(l)} \in \mathbb{R}^{s_l+1}$ il vettore delle attivazioni del layer l (bias incluso) e con f applicata elemento per elemento:

$$z^{(l+1)} = \vartheta^{(l)} a^{(l)}, \qquad a^{(l+1)} = \begin{bmatrix} 1 \\ f\big(z^{(l+1)}\big) \end{bmatrix}.$$

Il vettore $z^{(l+1)} \in \mathbb{R}^{s_{l+1}}$ contiene le somme pesate prima dell'attivazione. Il prodotto è ben definito perché $\vartheta^{(l)}$ ha $s_l + 1$ colonne, esattamente la lunghezza di $a^{(l)}$.

La feedforward propagation consiste quindi **nel ripetere** questo passo da $l = 1$ a $l = L - 1$, partendo dall'input:

1. $a^{(1)} = [1, x_1, \dots, x_n]^T$, dove $n = s_1$ è il numero di feature.
2. Per $l = 1, \dots, L-1$: calcolare $z^{(l+1)} = \vartheta^{(l)} a^{(l)}$ e $a^{(l+1)} = f\big(z^{(l+1)}\big)$, aggiungendo il bias $a_0^{(l+1)} = 1$ se $l + 1 < L$.
3. L'output della rete è $h_\vartheta(x) = a^{(L)} \in \mathbb{R}^{s_L}$.

Nell'ultimo layer non si aggiunge il bias, perché non c'è un layer successivo a cui trasmetterlo.

Learning. Il problema dell'apprendimento consiste nel trovare l'insieme ottimale di pesi per cui la rete h_ϑ approssima il più possibile una funzione F, rispetto a una funzione di loss. La funzione F non è data esplicitamente, ma solo implicitamente, attraverso gli esempi del training set. I pesi sono numeri reali piccoli, inizializzati casualmente e rispettando il symmetry breaking. Per determinarli bisogna usare la discesa del gradiente per ottenere il minimo della funzione di costo:

$$J(\vartheta) = \frac{1}{m} \sum_{i=1}^{m} L\big(\vartheta, x^{(i)}, y^{(i)}\big) + \underbrace{\frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} \big(\vartheta_{ji}^{(l)}\big)^2}_{\text{regolarizzazione}}$$

dove L è la loss function, ovvero la cross-entropy per la classificazione o la MSE per la regressione. La discesa del gradiente richiede le derivate di $J(\vartheta)$ rispetto a ogni parametro:

$$\vartheta_{ji}^{(l)} \leftarrow \vartheta_{ji}^{(l)} - \alpha \frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}}.$$

Calcolare a mano la derivata per ogni cammino della rete è inefficiente, perché molti cammini condividono gli stessi tratti e gli stessi calcoli verrebbero ripetuti più volte. Serve quindi un algoritmo più furbo, che sfrutti due osservazioni:

1. La rete è una composizione di funzioni derivabili.
2. Vale la proprietà della **chain rule**: se $F(x) = f(g(x))$, allora $F'(x) = g'(x)\, f'\big(g(x)\big)$.

Per trovare un modo efficiente di calcolare il gradiente su una rete si ricorre ai grafi computazionali.

2.2 Grafi computazionali

Definizione 2.1 (Grafo computazionale). Un **grafo computazionale** è un grafo orientato in cui:

- I **nodi** sono unità di calcolo: ogni nodo calcola una singola **funzione primitiva** del proprio input, ad esempio una somma, un prodotto o $1/x$.
- Gli **archi** trasmettono informazione numerica da un nodo all'altro, in avanti; un arco può essere **pesato**, e quando il peso non è indicato vale 1.

Le funzioni primitive devono essere **continue** e **derivabili**.

Una rete neurale feedforward rappresenta una catena di composizioni di funzioni che trasforma un input in un output, ed è quindi un grafo computazionale: i neuroni sono i nodi, i pesi $\vartheta_{ji}^{(l)}$ sono i pesi degli archi e le funzioni di attivazione sono le primitive dei nodi, assunte uguali per tutti i nodi.

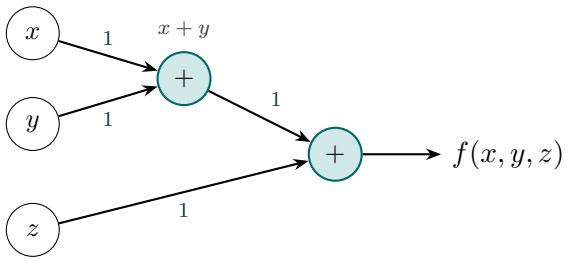


Figura 2.2. Esempio di Grafo Computazionale. Viene rappresentata la funzione $f(x, y, z) = (x + y) + z$: i nodi con lo sfondo verde petrolio calcolano la funzione primitiva somma, gli archi trasmettono i valori da sinistra verso destra e hanno tutti peso 1.

Esempio. Consideriamo la figura 2.2, che rappresenta un grafo computazionale per la funzione

$$f(x, y, z) = (x + y) + z.$$

La somma è una funzione primitiva, per cui la funzione si scompone in due nodi: il primo calcola $x + y$, il secondo somma z al risultato del primo.

2.2.1 B-diagram

Per poter calcolare le derivate, si dà a ogni nodo della rete una struttura composta, ottenendo un **backpropagation diagram** (B-diagram).

Definizione 2.2 (B-diagram). Un **B-diagram** è un grafo orientato in cui ogni nodo è diviso in due metà:

- la parte destra calcola la funzione primitiva del nodo;
- la parte sinistra calcola la derivata della funzione primitiva del nodo.

Gli archi non hanno più una freccia, perché il grafo può essere percorso sia in avanti sia all'indietro (figura 2.3).

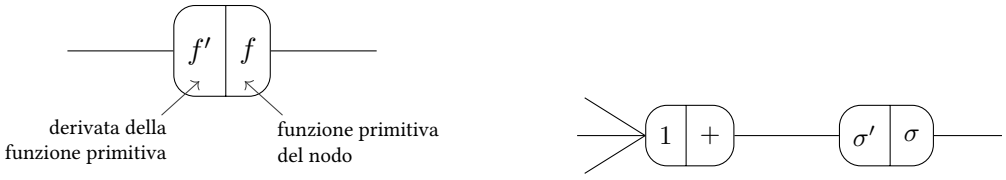


Figura 2.3. A sinistra, un nodo di un B-diagram: la metà destra contiene la funzione primitiva f, la metà sinistra la sua derivata f'. A destra, un neurone con attivazione sigmoide scomposto in due nodi: un nodo di integrazione (somma), la cui derivata rispetto a ogni input è 1, seguito dal nodo di attivazione σ.

Nodi di integrazione e di attivazione. Considerando le funzioni primitive, conviene separare la funzione di integrazione, ovvero la somma degli input di un neurone, dalla funzione di attivazione. Ad esempio, un neurone con attivazione sigmoide, il cui input è una combinazione lineare $z = \sum_i \vartheta_i x_i$, diventa la coppia di nodi a destra in figura 2.3: il nodo somma ha derivata 1 rispetto a ciascun input, mentre il nodo di attivazione calcola

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) = \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right).$$

Gli archi tra un nodo e l'altro possono essere pesati.

2.2.2 Valutazione in due passi

Per il training, la rete viene valutata in due passi:

1. **Feedforward step**: l'informazione va da sinistra verso destra. In ogni nodo vengono valutate sia le funzioni primitive (lato destro) sia le loro derivate (lato sinistro), e i risultati vengono memorizzati. I risultati del lato destro vengono inoltrati alle unità connesse sulla destra.

2. **Backward step**: serve a valutare gli errori e a propagarli all'indietro per aggiornare i parametri. In questo caso vengono usati i risultati del lato sinistro, ovvero le derivate. L'input dalla destra è sempre la costante 1.

Per capire come si combinano le informazioni nel backward step bisogna considerare tre casi: la composizione di funzioni, la somma di funzioni e gli archi pesati.

Caso 1: composizione di funzioni. Consideriamo due nodi in serie, il primo con primitiva g e il secondo con primitiva f, che calcolano quindi $f(g(x))$ (figura 2.4). Nel feedforward step il primo nodo riceve x, memorizza $g'(x)$ e trasmette $g(x)$; il secondo memorizza $f'(g(x))$ e restituisce $f(g(x))$. Nel backward step si immette 1 da destra: il secondo nodo moltiplica l'input per il valore memorizzato a sinistra e trasmette $1 \cdot f'(g(x))$; il primo fa lo stesso e restituisce

$$1 \cdot f'\big(g(x)\big)\, g'(x)$$

che è esattamente la derivata della piccola rete secondo la chain rule. La backpropagation fornisce quindi un modo per implementare il principio della chain rule.

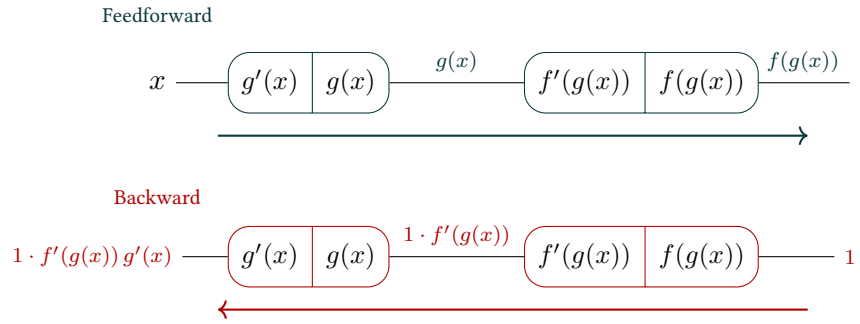


Figura 2.4. Caso 1, composizione di funzioni. In alto il feedforward step (in verde petrolio): ogni nodo memorizza la propria derivata a sinistra e trasmette il valore della primitiva a destra. In basso il backward step (in rosso): partendo dalla costante 1, ogni nodo moltiplica l'input proveniente da destra per la derivata memorizzata, ottenendo all'estremo sinistro la derivata $f'(g(x))\, g'(x)$ della composizione.

In generale, se due nodi in serie hanno memorizzato i valori B' e A' durante il feedforward, il valore che attraversa all'indietro il nodo A è $1 \cdot A'$ e il valore che esce da B è $1 \cdot A' \cdot B'$, ovvero la derivata. Si noti che le derivate $g'(x)$ e $f'(g(x))$ vengono calcolate nel feedforward e usate nel backward, mentre $g(x)$ e $f(g(x))$ vengono calcolate e usate nel solo feedforward.

Caso 2: somma di funzioni. Consideriamo ora un input x che viene inviato a due nodi f_1 e f_2, le cui uscite vengono sommate da un nodo $+$ (figura 2.5). Il nodo somma $F(x_1, x_2) = x_1 + x_2$ ha due derivate parziali, $\partial F/\partial x_1 = \partial F/\partial x_2 = 1$. Nel feedforward la rete calcola $f_1(x) + f_2(x)$. Nel backward step il nodo somma riceve 1 e lo trasmette, moltiplicato per 1, su entrambi i rami; ogni ramo lo moltiplica per la propria derivata, ottenendo $1 \cdot 1 \cdot f_1'(x)$ e $1 \cdot 1 \cdot f_2'(x)$. Andando all'indietro, **i valori che arrivano allo stesso nodo da rami diversi vengono sommati**, per cui all'estremo sinistro si ottiene

$$f_1'(x) + f_2'(x)$$

ovvero proprio la derivata di $f_1(x) + f_2(x)$. Un nodo il cui output viene inviato a più nodi è quindi equivalente a più copie dello stesso nodo, e i gradienti delle copie si sommano.

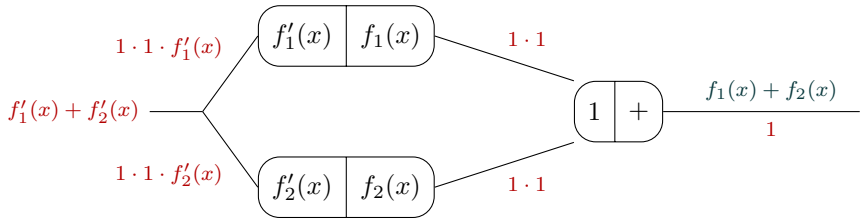


Figura 2.5. Caso 2, somma di funzioni. Il nodo somma restituisce $f_1(x) + f_2(x)$ nel feedforward (in verde petrolio); nel backward (in rosso) la costante 1 viene distribuita su entrambi i rami, ciascuno la moltiplica per la propria derivata e, nel punto in cui i rami si separano, i due contributi vengono sommati.

Caso 3: archi pesati. Infine, consideriamo due nodi A e B collegati da un arco di peso ϑ (figura 2.6). Nel feedforward il nodo A restituisce $A(x)$, l'arco la moltiplica per ϑ e il nodo B riceve $\vartheta A(x)$, restituendo $B(\vartheta A(x))$. Nel backward step il nodo B trasmette $B'(\vartheta A(x))$, l'arco lo moltiplica per ϑ e il nodo A lo moltiplica per la propria derivata, restituendo

$$\vartheta\, B'\big(\vartheta A(x)\big)\, A'(x)$$

che è la derivata di $B(\vartheta A(x))$ rispetto a x. I pesi modulano quindi i valori trasmessi **in entrambe le direzioni**.

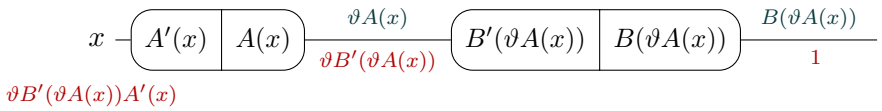


Figura 2.6. Caso 3, archi pesati. Sopra l'arco il valore trasmesso nel feedforward (in verde petrolio), sotto l'arco quello trasmesso nel backward (in rosso): in entrambe le direzioni il valore viene moltiplicato per il peso ϑ.

2.2.3 L'algoritmo sul grafo

Mettendo insieme i tre casi, data una rete F con input x, la derivata $F'(x)$ si calcola come segue:

1. **Forward pass**: l'input viene valutato da sinistra verso destra lungo la rete; i valori delle funzioni primitive e delle loro derivate vengono calcolati e memorizzati in ogni nodo.
2. **Backward pass**: la costante 1 viene imposta come input destro della rete, e la rete viene percorsa da destra verso sinistra. L'informazione che arriva a un nodo viene sommata e il risultato viene moltiplicato per il valore memorizzato nella parte sinistra del nodo e per il peso dell'arco. Il risultato finale è la derivata di F rispetto a x.

Osservazione. Qualunque grafo computazionale, ovvero qualunque composizione di funzioni, può essere valutato con la backpropagation per calcolarne le derivate. In questo senso la backpropagation non è un algoritmo specifico per le reti neurali, ma un modo generale di applicare la chain rule.

Esempio. Consideriamo la funzione

$$f(x, y, z) = (x + y) \cdot z$$

con input $x = -2$, $y = 5$, $z = -4$. Vogliamo calcolare $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$ e $\frac{\partial f}{\partial z}$.

Soluzione. Scomponiamo la funzione nelle due primitive $q = x + y$ e $f = q \cdot z$, di cui conosciamo le derivate parziali:

$$\frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1, \qquad \frac{\partial f}{\partial q} = z, \quad \frac{\partial f}{\partial z} = q.$$

Per la chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x} = z \cdot 1, \qquad \frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y} = z \cdot 1, \qquad \frac{\partial f}{\partial z} = q.$$

Sul grafo computazionale (figura 2.7) procediamo in due passi:

1. **Forward pass**: il nodo somma calcola $q = -2 + 5 = 3$, il nodo prodotto calcola $f = 3 \cdot (-4) = -12$.
2. **Backward pass**: si immette 1 a destra. Il nodo prodotto trasmette verso q il valore $1 \cdot z = -4$ e verso z il valore $1 \cdot q = 3$. Il nodo somma riceve -4 e lo trasmette, moltiplicato per 1, sia verso x sia verso y.

Si ottiene quindi $\frac{\partial f}{\partial x} = -4$, $\frac{\partial f}{\partial y} = -4$ e $\frac{\partial f}{\partial z} = 3$.

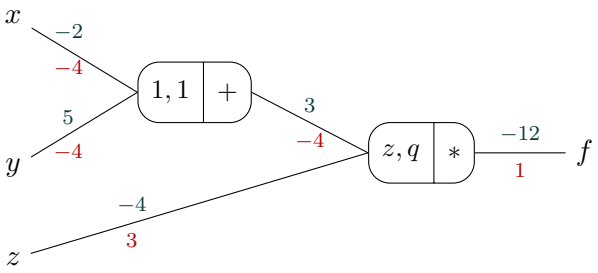


Figura 2.7. Forward e backward pass per $f(x, y, z) = (x + y) \cdot z$ con $x = -2$, $y = 5$, $z = -4$. Sopra gli archi i valori del forward pass (in verde petrolio), sotto gli archi i valori del backward pass (in rosso), che coincidono con le derivate parziali di f rispetto a ogni variabile. Nei nodi, a sinistra, sono riportate le derivate parziali della primitiva rispetto ai due input.

2.2.4 Il comportamento delle porte

L'esempio precedente mostra un comportamento ricorrente dei nodi, o *porte*, più comuni durante il backward pass. Indicando con A il gradiente che arriva da destra (figura 2.8):

- **Add gate**: *distribuisce* il gradiente, trasmettendo A invariato a tutti i propri input.
- **Mul gate**: *scambia* il gradiente, ovvero trasmette a ciascun input A moltiplicato per il valore dell'altro input: $A \cdot y$ verso x e $A \cdot x$ verso y.
- **Max gate**: *instrada* il gradiente, trasmettendo A solo all'input che era il massimo e 0 agli altri, perché $\dfrac{\partial \max(x,y)}{\partial x} = \mathbf{1}\{x \geq y\}$ e $\dfrac{\partial \max(x,y)}{\partial y} = \mathbf{1}\{y > x\}$.

Inoltre, come visto nel caso 2, quando l'output di un nodo viene inviato a più rami **i gradienti dei rami si sommano**.

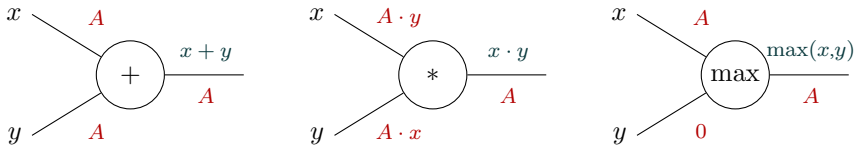


Figura 2.8. Comportamento delle porte durante il backward pass, con in verde petrolio l'output del forward pass e in rosso i gradienti: la add gate distribuisce il gradiente A, la mul gate lo scambia moltiplicandolo per l'altro input e la max gate lo instrada verso l'input massimo.

2.3 Backpropagation su una rete multistrato

Finora abbiamo usato la backpropagation per derivare funzioni qualsiasi, scritte come grafi computazionali. Torniamo ora al problema da cui siamo partiti: addestrare una rete con uno o più hidden layer per la **classificazione multiclasse**. In questa sezione mettiamo insieme i pezzi e arriviamo all'algoritmo che si usa in pratica.

2.3.1 La funzione di costo

Consideriamo un problema con K classi. Come visto nel capitolo 1, l'etichetta di ogni esempio è un vettore one-hot $y^{(i)} \in \mathbb{R}^K$, ad esempio con $K = 3$:

$$y \in \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}.$$

A seconda del task si usano due funzioni di costo. Nel caso **binario e multi-label** (un esempio può appartenere a più classi) ogni neurone di output è una regressione logistica indipendente, e si sommano le cross-entropy binarie:

$$J(\vartheta) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} \left[y_k^{(i)} \log \left(h_\vartheta(x^{(i)}) \right)_k + \left(1 - y_k^{(i)}\right) \log \left(1 - \left(h_\vartheta(x^{(i)}) \right)_k \right) \right]$$

dove $\left(h_\vartheta(x^{(i)})\right)_k$ è la k-esima componente dell'output della rete. Nel caso **categorico**, invece, ogni esempio ha una sola classe, l'output layer è un softmax e si usa la cross-entropy:

$$J(\vartheta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(h_\vartheta(x^{(i)})\right)_k.$$

In entrambi i casi l'obiettivo è minimizzare $J(\vartheta)$ con la discesa del gradiente, per cui ci servono le derivate

$$\frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}} \qquad \forall i, j, l.$$

2.3.2 La rete estesa

Per calcolare queste derivate con la backpropagation, la funzione di costo deve far parte del grafo. Si costruisce quindi una rete più grande.

> **Definizione 2.3 (Rete estesa).** La **rete estesa** (extended network) è la rete che include la funzione di loss ed è estesa a tutti gli esempi del training set, che sono noti in anticipo.

In pratica, come in figura 2.9, la rete viene replicata una volta per ogni esempio $x^{(i)}$, con $i = 1, \ldots, m$. Tutte le copie **condividono gli stessi pesi** ϑ. A valle dei K neuroni di output di ogni copia si aggiungono i nodi che calcolano l'errore.

Nella figura, che segue il libro di Rojas, l'errore è quello quadratico: ogni nodo calcola $\frac{1}{2}\left(o_k^{(i)} - y_k^{(i)}\right)^2$, dove $o_k^{(i)}$ è l'output della rete per la classe k e $y_k^{(i)}$ la k-esima componente del vettore one-hot. Un nodo somma raccoglie questi termini nell'errore E_i dell'esempio $x^{(i)}$, e un ultimo nodo somma restituisce l'errore complessivo:

$$E = \sum_{i=1}^{m} E_i = \frac{1}{2}\sum_{i=1}^{m}\sum_{k=1}^{K}\left(o_k^{(i)} - y_k^{(i)}\right)^2.$$

Gli input e le etichette sono fissati, per cui E è **una funzione dei soli parametri** della rete. Con la cross-entropy il discorso non cambia: cambiano soltanto i nodi che seguono l'output layer.

2.3.3 Il calcolo nel singolo nodo

Ogni nodo della rete estesa esegue un calcolo **locale**. Consideriamo un nodo con primitiva F che riceve due input x_1 e x_2 lungo archi di peso w_1 e w_2 (figura 2.10, a sinistra). Nel forward pass il nodo restituisce $z = F(w_1x_1, w_2x_2)$. Inoltre memorizza nella metà sinistra le derivate parziali di F rispetto a ciascun input[1]. Nel backward pass arriva da destra il gradiente $\frac{\partial L}{\partial z}$, e il nodo trasmette verso ogni input:

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial z}\,\frac{\partial F}{\partial x_1}\,w_1, \qquad \frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial z}\,\frac{\partial F}{\partial x_2}\,w_2$$

dove $\frac{\partial F}{\partial x_1}$ indica la derivata parziale di F rispetto al suo primo argomento. Sono esattamente i casi 1 e 3 visti prima: si moltiplica per la derivata memorizzata e per il peso dell'arco.

[1] Quando la primitiva ha più variabili, la metà sinistra del B-diagram contiene una derivata parziale per ogni input, e non più una sola derivata come nei casi visti finora.

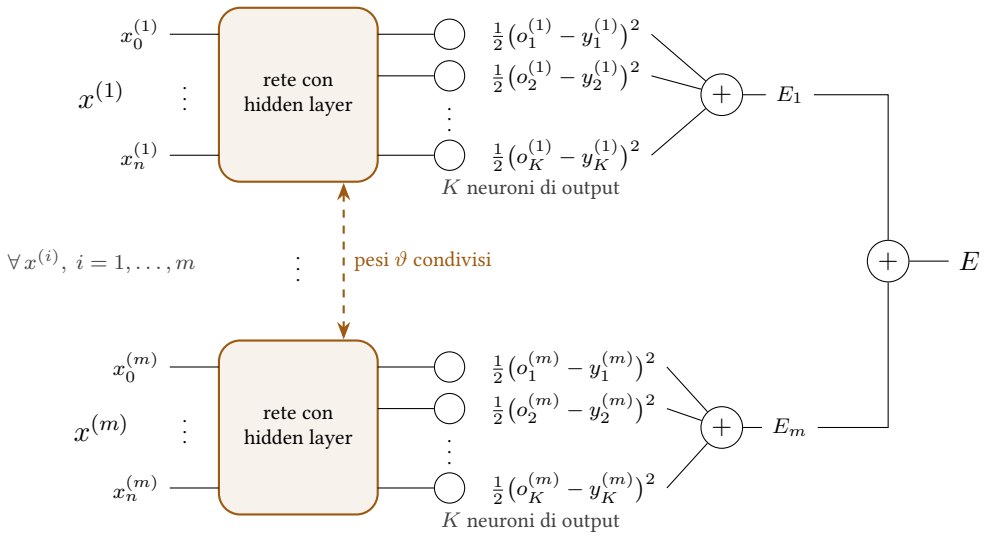


Figura 2.9. La rete estesa. La rete viene replicata per ogni esempio $x^{(i)}$ del training set (tutte le copie condividono i pesi ϑ). Dopo i K neuroni di output si calcolano gli errori $\frac{1}{2}(o_k^{(i)} - y_k^{(i)})^2$, che vengono sommati prima nell'errore E_i del singolo esempio e poi nell'errore complessivo E. Il risultato è una funzione dei soli parametri della rete.

Torniamo alla rete estesa. Vogliamo calcolare, per ogni parametro $\vartheta_{ji}^{(l)}$, la derivata

$$\frac{\partial E}{\partial \vartheta_{ji}^{(l)}}$$

dove l è il layer e il parametro collega il nodo i del layer l al nodo j del layer $l+1$. Indichiamo con $o_i^{(l)}$ l'output del nodo i. L'informazione che arriva al nodo j lungo questo arco è $o_i^{(l)}\vartheta_{ji}^{(l)}$, per cui il nodo j calcola nel backward pass la derivata $\frac{\partial E}{\partial o_i^{(l)}\vartheta_{ji}^{(l)}}$ (figura 2.10, a destra).

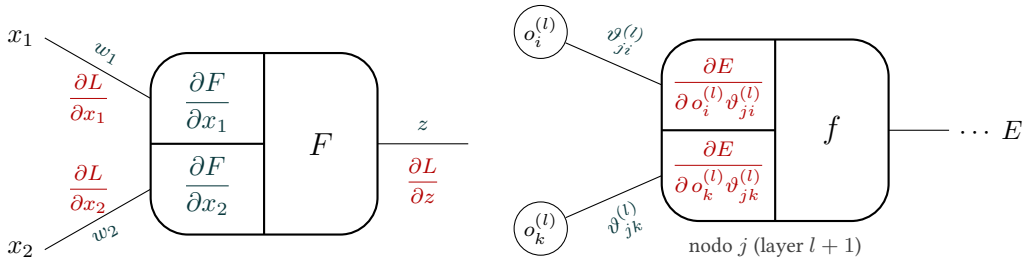


Figura 2.10. A sinistra, il calcolo locale in un nodo con primitiva F e due input pesati: in verde petrolio i valori del forward pass, in rosso i gradienti del backward pass. A destra, il nodo j della rete estesa: gli arrivano gli output $o_i^{(l)}$ e $o_k^{(l)}$ del layer precedente, moltiplicati per i pesi, e nel backward pass calcola la derivata dell'errore E rispetto a ciascuna di queste informazioni.

Il valore $o_i^{(l)}$ è già stato calcolato durante il forward pass, quindi nel backward pass è una

costante. Per la chain rule possiamo scrivere:

$$\frac{\partial E}{\partial \vartheta_{ji}^{(l)}} = \frac{\partial\, o_i^{(l)} \vartheta_{ji}^{(l)}}{\partial \vartheta_{ji}^{(l)}} \frac{\partial E}{\partial\, o_i^{(l)} \vartheta_{ji}^{(l)}} = o_i^{(l)}\, \tilde{\delta}_{ji}^{(l)}$$

dove $\tilde{\delta}_{ji}^{(l)} = \frac{\partial E}{\partial\, o_i^{(l)} \vartheta_{ji}^{(l)}}$ è l'**errore propagato all'indietro** (backpropagated error) fino al nodo j. Ogni nodo, quindi, durante il training conserva:

- f e f': la funzione primitiva e la sua derivata, calcolate sull'input $o_i^{(l)} \vartheta_{ji}^{(l)}$.
- a_{ji}: il valore di f calcolato nel forward pass.
- v_{ji}: il valore di f' calcolato nel forward pass.
- $\tilde{\delta}_{ji}^{(l)}$: il risultato cumulativo del backward pass, ovvero la derivata dell'errore rispetto a $o_i^{(l)} \vartheta_{ji}^{(l)}$.

Ne consegue che, per ogni nodo, il gradiente dell'errore rispetto al parametro associato si calcola in modo molto semplice, e si può usare direttamente nella discesa del gradiente:

$$\frac{\partial E}{\partial \vartheta_{ji}^{(l)}} = o_i^{(l)}\, \tilde{\delta}_{ji}^{(l)} \qquad \Longrightarrow \qquad \Delta \vartheta_{ji}^{(l)} = -\alpha\, o_i^{(l)}\, \tilde{\delta}_{ji}^{(l)}.$$

Questo si può fare per ogni parametro della rete.

> **Osservazione.** In una rete multistrato il nodo j prima somma i propri input e poi applica l'attivazione. La derivata della somma rispetto a ciascun input vale 1, per cui $\tilde{\delta}_{ji}^{(l)}$ è uguale per tutti gli archi che entrano nel nodo j: dipende solo da j e non da i. Per questo motivo, da qui in avanti, scriveremo un solo errore per nodo, $\delta_j^{(l+1)}$, e la formula diventa $\frac{\partial E}{\partial \vartheta_{ji}^{(l)}} = a_i^{(l)} \delta_j^{(l+1)}$, dove $a_i^{(l)} = o_i^{(l)}$ è l'attivazione del nodo i.

2.3.4 L'ultimo layer: softmax e cross-entropy

Per avviare il backward pass ci serve l'errore dell'ultimo layer. Nel caso categorico l'output è un softmax:

$$\hat{y}_i = \frac{e^{z_i}}{\Sigma_K} \qquad \forall i = 1, \dots, K, \qquad \text{dove } \Sigma_K = \sum_{k=1}^{K} e^{z_k}$$

e z è il vettore dei logit. Per usare il softmax in una rete dobbiamo calcolarne la derivata, ovvero $\frac{\partial \hat{y}_i}{\partial z_j}$ per ogni coppia i, j. Lo facciamo con la regola del quoziente:

$$f(x) = \frac{g(x)}{h(x)} \quad \Longrightarrow \quad f'(x) = \frac{g'(x)h(x) - h'(x)g(x)}{h^2(x)}$$

dove $g = e^{z_i}$ e $h = \Sigma_K$. Ricordando che $\frac{\partial \Sigma_K}{\partial z_j} = e^{z_j}$, ci sono due casi:

- **Se** $i = j$: anche il numeratore dipende da z_i, e

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i}\Sigma_K - e^{z_i}e^{z_i}}{\Sigma_K^2} = \frac{e^{z_i}}{\Sigma_K} \cdot \frac{\Sigma_K - e^{z_i}}{\Sigma_K} = \hat{y}_i(1 - \hat{y}_i).$$

- **Se** $i \neq j$: il numeratore non dipende da z_j, quindi la sua derivata è nulla, e

$$\frac{\partial \hat{y}_i}{\partial z_j} = \frac{0 - e^{z_i} e^{z_j}}{\Sigma_K^2} = -\frac{e^{z_i}}{\Sigma_K} \cdot \frac{e^{z_j}}{\Sigma_K} = -\hat{y}_i \hat{y}_j.$$

Consideriamo ora la cross-entropy su un singolo esempio[2], dove $\hat{y}$ è l'output del softmax e y l'etichetta one-hot del pattern x:

$$L = -\sum_{k=1}^{K} y_k \log(\hat{y}_k).$$

Deriviamo rispetto al logit z_i, separando il termine $k = i$ da tutti gli altri e usando le due derivate appena calcolate:

$$\begin{aligned}\frac{\partial L}{\partial z_i} &= -\sum_{k=1}^{K} y_k \frac{1}{\hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_i} = -\frac{y_i}{\hat{y}_i}\, \hat{y}_i(1 - \hat{y}_i) - \sum_{k \neq i} \frac{y_k}{\hat{y}_k}\big(-\hat{y}_k \hat{y}_i\big) \\ &= -y_i + y_i \hat{y}_i + \sum_{k \neq i} y_k \hat{y}_i = -y_i + \hat{y}_i \sum_{k=1}^{K} y_k = \hat{y}_i - y_i.\end{aligned}$$

L'ultimo passaggio è il punto in cui serve il one-hot encoding: le componenti di y sommano a 1, quindi $\sum_k y_k = 1$. Abbiamo così dimostrato il risultato già anticipato nella sezione 1.3: l'errore dell'ultimo layer è semplicemente la differenza tra la probabilità predetta e l'etichetta.

2.3.5 Forward e backward pass

Il training di una rete per la classificazione multiclasse segue questi passi:

1. Inizializzazione dei parametri, con il symmetry breaking.
2. Per ogni esempio $(x^{(i)}, y^{(i)})$ del training set:
   - forward propagation di $x^{(i)}$.
   - backpropagation, usando $y^{(i)}$, e calcolo delle derivate $\frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}}$.
3. Aggiornamento dei parametri con la discesa del gradiente.

Vediamo i due passi centrali sulla rete in figura 2.11, con 3 feature in input, due hidden layer da 3 neuroni e 3 classi.

Forward propagation. Si calcolano in ordine le attivazioni di tutti i layer, partendo dall'esempio $(x^{(i)}, y^{(i)})$:

$$\begin{aligned}&a^{(1)} = x^{(i)}, \\ &z^{(2)} = \vartheta^{(1)} a^{(1)}, \quad a^{(2)} = f\big(z^{(2)}\big), \\ &z^{(3)} = \vartheta^{(2)} a^{(2)}, \quad a^{(3)} = f\big(z^{(3)}\big), \\ &z^{(4)} = \vartheta^{(3)} a^{(3)}, \quad a^{(4)} = h_\vartheta\big(x^{(i)}\big) = f\big(z^{(4)}\big)\end{aligned}$$

[2] Il fattore $\frac{1}{m}$ e la somma sugli esempi non servono in questo calcolo: le derivate dei singoli esempi si sommano e si mediano alla fine, come vedremo nell'algoritmo.

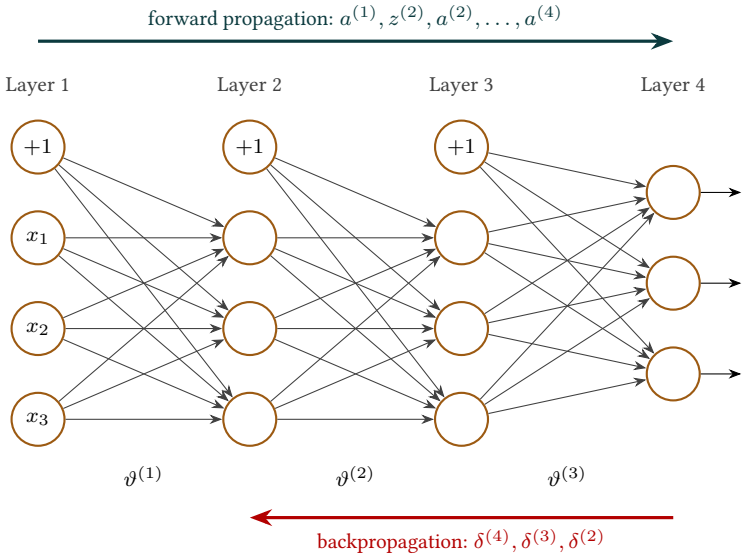


Figura 2.11. La rete usata nell'esempio: 3 feature in input, due hidden layer da 3 neuroni (più il bias, $+1$) e un output layer con 3 neuroni, uno per classe. In verde petrolio il verso del forward pass, in rosso quello del backward pass, che si ferma al layer 2: l'input non ha un errore da calcolare.

dove a ogni $a^{(l)}$, tranne l'ultimo, si aggiunge in testa il bias. Ad esempio, nel primo passo $a^{(1)} = [1, x_1, x_2, x_3]^T$ e $\vartheta^{(1)} \in \mathbb{R}^{3\times 4}$, per cui ogni componente di $z^{(2)}$ è una combinazione lineare dell'input:

$$z_j^{(2)} = \vartheta_{j0}^{(1)} + \vartheta_{j1}^{(1)} x_1 + \vartheta_{j2}^{(1)} x_2 + \vartheta_{j3}^{(1)} x_3, \qquad j = 1, 2, 3.$$

Nell'ultimo layer f è il softmax. Come nel B-diagram, durante il forward pass conviene memorizzare anche le derivate $f'\big(z^{(l)}\big)$, che serviranno nel backward pass.

Backpropagation. Si parte dall'errore dell'output layer, che abbiamo appena calcolato per softmax e cross-entropy, e si torna indietro un layer alla volta:

$$\begin{aligned}
\delta^{(4)} &= a^{(4)} - y^{(i)}, \\
\delta^{(3)} &= \big(\vartheta^{(3)}\big)^T \delta^{(4)} \mathbin{.*} f'\big(z^{(3)}\big), \\
\delta^{(2)} &= \big(\vartheta^{(2)}\big)^T \delta^{(3)} \mathbin{.*} f'\big(z^{(2)}\big)
\end{aligned}$$

dove $.*$ indica il prodotto elemento per elemento. Non si calcola $\delta^{(1)}$, perché l'input è un dato e non ha errore.

Controlliamo le dimensioni per $\delta^{(3)}$. La matrice $\vartheta^{(3)}$ mappa i 4 valori del layer 3 (bias compreso) nei 3 neuroni di output, quindi $\big(\vartheta^{(3)}\big)^T \in \mathbb{R}^{4\times 3}$. Moltiplicata per $\delta^{(4)} \in \mathbb{R}^3$ restituisce un vettore di 4 componenti. La prima corrisponde al bias, che non ha input da cui ricevere l'errore, e viene scartata: ne restano 3, una per neurone, compatibili con $f'\big(z^{(3)}\big)$.

La componente j-esima vale:

$$\Big[\big(\vartheta^{(3)}\big)^T \delta^{(4)}\Big]_j = \vartheta_{1j}^{(3)} \delta_1^{(4)} + \vartheta_{2j}^{(3)} \delta_2^{(4)} + \vartheta_{3j}^{(3)} \delta_3^{(4)}$$

ovvero la somma degli errori dei nodi a cui il neurone j è collegato, ciascuno moltiplicato per il

peso dell'arco. Sono di nuovo i casi 2 e 3 del B-diagram: i rami si sommano e i pesi moltiplicano. Il prodotto per $f'\big(z_j^{(3)}\big)$ corrisponde invece al caso 1, la derivata memorizzata nel nodo.

Una volta ottenuti gli errori, le derivate rispetto ai parametri si ricavano come visto per il singolo nodo:

$$\frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}} = a_i^{(l)}\, \delta_j^{(l+1)}, \qquad l = 1, 2, 3$$

ovvero, in forma matriciale, $\frac{\partial J}{\partial \vartheta^{(l)}} = \delta^{(l+1)} \big(a^{(l)}\big)^T$.[3] Qui stiamo considerando un solo esempio e trascurando la regolarizzazione: l'algoritmo completo aggiunge entrambe le cose.

2.3.6 L'algoritmo di backpropagation

Mettendo insieme forward e backward pass su tutto il training set si ottiene l'algoritmo 1. Per ogni esempio si accumulano le derivate in delle matrici $\Delta^{(l)}$, della stessa dimensione di $\vartheta^{(l)}$. Alla fine si fa la media e si aggiunge la regolarizzazione, ottenendo le matrici $D^{(l)}$.

Algorithm 1 Backpropagation

Require: training set $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$

1: $\Delta_{ji}^{(l)} \leftarrow 0 \quad \forall i, j, l$
2: **for** $t = 1, \dots, m$ **do**
3: $\quad a^{(1)} \leftarrow x^{(t)}$
4: $\quad$ forward propagation: calcolo di $a^{(2)}, a^{(3)}, \dots, a^{(L)}$
5: $\quad \delta^{(L)} \leftarrow a^{(L)} - y^{(t)}$
6: $\quad$ calcolo di $\delta^{(L-1)}, \delta^{(L-2)}, \dots, \delta^{(2)}$ con $\delta^{(l)} = \big(\vartheta^{(l)}\big)^T \delta^{(l+1)} .* f'\big(z^{(l)}\big)$
7: $\quad \Delta_{ji}^{(l)} \leftarrow \Delta_{ji}^{(l)} + a_i^{(l)} \delta_j^{(l+1)} \quad \forall l = 1, \dots, L-1$
8: **end for**
9: $D_{ji}^{(l)} \leftarrow \frac{1}{m} \Delta_{ji}^{(l)}$ **se** $i = 0$ (bias)
10: $D_{ji}^{(l)} \leftarrow \frac{1}{m} \big(\Delta_{ji}^{(l)} + \lambda \vartheta_{ji}^{(l)}\big)$ altrimenti

Le matrici ottenute sono proprio le derivate della funzione di costo[4]:

$$\frac{\partial J(\vartheta)}{\partial \vartheta_{ji}^{(l)}} = D_{ji}^{(l)}$$

e possono essere usate per aggiornare i parametri con la formula della discesa del gradiente. Il bias non viene regolarizzato[5], come già visto per la regressione logistica e il softmax.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (13 e 22 ottobre 2026).

[3] La stessa formula si trova anche in forma compatta come $a^{(l)} .* \delta^{(l+1)}$. Il prodotto $\delta^{(l+1)} \big(a^{(l)}\big)^T$ ha esattamente la dimensione di $\vartheta^{(l)}$, ovvero $s_{l+1} \times (s_l + 1)$.

[4] Con la funzione di costo regolarizzata nella forma $J(\vartheta) = \frac{1}{m} \sum_i L + \frac{\lambda}{2m} \sum_{l,i,j} \big(\vartheta_{ji}^{(l)}\big)^2$, la stessa usata per il softmax.

[5] In alcune formulazioni la condizione è scritta come $j = 0$. Con la nostra notazione, in cui $\vartheta_{ji}^{(l)}$ va dal nodo i al nodo j, il peso del bias è quello che parte dall'unità $i = 0$.

- Slide del corso *Backpropagation (via Computational Graphs)* e *Backpropagation Algorithm.*
- Capitolo 7, *The Backpropagation Algorithm*, del libro *Neural Networks: A Systematic Introduction* [14], in particolare la sezione 7.3 sulla rete estesa.

Capitolo 3

Convolutional Neural Network

Le **Convolutional Neural Network** (CNN) sono reti neurali pensate per elaborare immagini e, più in generale, dati organizzati su una griglia. In questo capitolo partiamo dai limiti di un MLP quando l'input è un'immagine, introduciamo l'operazione di convoluzione e vediamo come i suoi parametri (padding, stride e dilation) determinano la forma dell'output. Infine vediamo gli altri layer che compongono una CNN (attivazioni, normalizzazione, pooling e fully connected) e alcune regole pratiche per progettarla.

3.1 Dalle reti neurali alle immagini

3.1.1 Representation learning

Riprendiamo un MLP con un solo hidden layer, ad esempio con 4 input e 3 output. Rispetto al capitolo 1, qui scriviamo il bias b separato dai pesi ϑ. La rete calcola:

$$F_\vartheta(x) = \vartheta_2 \, f(\vartheta_1 x + b_1) + b_2$$

ed esegue quindi due trasformazioni:

1. trasforma l'input x in $a(x) = f(\vartheta_1 x + b_1)$.
2. trasforma il risultato intermedio $a(x)$ nell'output $h(x) = \vartheta_2 \, a(x) + b_2$.

Il risultato intermedio $a(x)$ è una **rappresentazione** di x, e non è scelto da noi ma appreso: l'MLP sta facendo **representation learning**[1].

In generale, una rete neurale implementa una funzione che mappa l'input x nell'output y, dati i parametri ϑ:

$$y = h(x; \vartheta) = F_\vartheta(x).$$

Se abbiamo un numero sufficiente di coppie $\langle x^{(i)}, y^{(i)} \rangle$ (il dataset), possiamo cercare i parametri ϑ ottimali, ovvero quelli per cui

$$\sum_i \operatorname{loss}\big(y^{(i)}, h(x^{(i)}; \vartheta)\big)$$

[1] Il termine *representation learning* si riferisce alla capacità di una rete neurale di apprendere automaticamente rappresentazioni utili dei dati di input.

è minima, dove la funzione di loss misura l'errore commesso da h nel predire y. La ricerca dei parametri ottimali è la procedura di **training**, e l'algoritmo più usato è la backpropagation vista nel capitolo 2: poiché h è derivabile rispetto a ϑ, funziona secondo il principio della discesa del gradiente.

Esempio: regressione. Vogliamo predire il consumo di un'automobile, in miglia per gallone (mpg), a partire da tre feature: il numero di cilindri c, la potenza p e il peso w. È ragionevole pensare che esista una funzione h tale che

$$\text{mpg} = h(c, p, w).$$

Come la definiamo matematicamente? Non lo facciamo: la impariamo dai dati. Il dataset *Auto MPG* contiene 398 auto, e possiamo usarlo per apprendere i pesi della rete a sinistra in figura 3.1. Trattandosi di regressione, come funzione di loss si usa la MSE.

Esempio: classificazione. Il dataset *Iris* contiene 150 fiori, descritti da 4 misure e divisi in 3 specie. Vogliamo classificare un fiore a partire dalle sue misure: gli input sono quindi 4, ma quanti sono gli output? Una tecnica comune è far calcolare alla rete **un punteggio per classe** e scegliere la classe con il punteggio più alto, per cui gli output sono 3 (figura 3.1, a destra).

In fase di training cerchiamo i pesi che assegnano un punteggio alto alla classe corretta e basso a tutte le altre. In fase di test, invece, facciamo passare il campione nella rete e scegliamo la classe con il punteggio massimo. Come loss si usa la cross-entropy, dopo aver trasformato i punteggi in probabilità con il softmax.

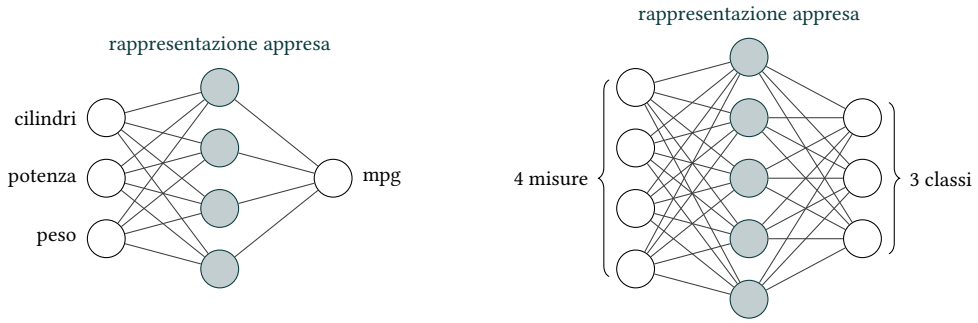


Figura 3.1. Due reti con un hidden layer. A sinistra, la regressione del consumo (mpg) a partire da cilindri, potenza e peso. A destra, la classificazione dei fiori del dataset Iris a partire da 4 misure, con un punteggio per ciascuna delle 3 classi. In entrambi i casi l'hidden layer (in verde petrolio) contiene la rappresentazione appresa dell'input.

Da shallow a deep. Possiamo aggiungere tutti gli hidden layer che vogliamo. Con due hidden layer, ad esempio, la rete diventa:

$$h(x, \vartheta) = \vartheta_h \, f_2\big(\vartheta_{f_2} f_1(\vartheta_{f_1} x + b_{f_1}) + b_{f_2}\big) + b_h$$

dove f_1 e f_2 sono le funzioni di attivazione dei due layer. Ogni layer costruisce la propria rappresentazione a partire da quella del layer precedente, per cui la rete impara delle **rappresentazioni gerarchiche**.

Serve davvero andare in profondità? In teoria no: il teorema di approssimazione universale (teorema 1.1) dice che un solo hidden layer, con un numero finito di neuroni, basta ad approssimare qualunque funzione continua su un compatto di $\mathbb{R}^n$. In pratica, però, il numero di neuroni necessari può diventare enorme, ed è più utile "scomporre" la rete introducendo altri hidden layer.

3.1.2 MLP e immagini

Gli MLP non sono molto adatti a elaborare immagini, per tre motivi:

- **Le immagini sono grandi**: anche una piccola immagine in scala di grigi di 100×100 pixel contiene 10000 valori.
- **Le immagini sono complesse**, e per elaborarle possono servire molti layer.
- Quando il numero di layer e il numero di input crescono, **il numero di parametri cresce di conseguenza**.

L'idea è quindi di ridurre il numero di parametri in ogni layer, così da poter costruire modelli più profondi.

Forniamo il seguente esempio, su cui torneremo più volte nel capitolo: il dataset **MNIST** (figura 3.2) contiene 70000 immagini di cifre scritte a mano, di 28×28 pixel, divise in 10 classi (le cifre da 0 a 9). Vogliamo classificare ogni immagine nella cifra corretta.

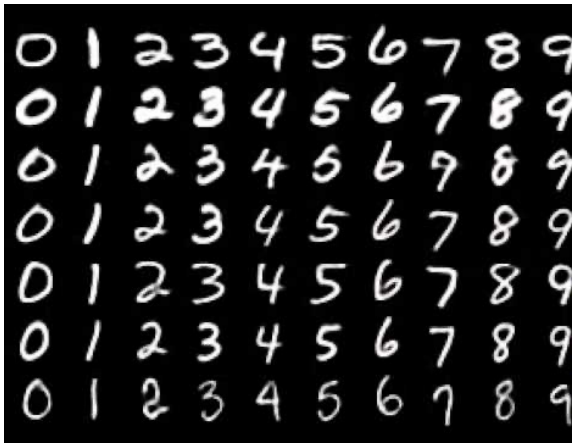


Figura 3.2. Alcune immagini del dataset MNIST: cifre scritte a mano da 0 a 9, in scala di grigi, di 28×28 pixel ciascuna.

L'approccio più semplice è usare un MLP. L'immagine viene "srotolata" da una matrice 28×28 a un vettore 784×1, che diventa l'input di una rete con un hidden layer da 500 neuroni e un output layer da 10 neuroni, uno per classe (figura 3.3). La rete calcola:

$$h(x, \vartheta) = \vartheta_h \, f(\vartheta_f x + b_f) + b_h$$

dove $\vartheta_f \in \mathbb{R}^{500\times784}$, $b_f \in \mathbb{R}^{500}$, $\vartheta_h \in \mathbb{R}^{10\times500}$ e $b_h \in \mathbb{R}^{10}$. Quanti parametri ha? Basta sommare le dimensioni:

$$500 \cdot 784 + 500 + 10 \cdot 500 + 10 = 397\,510.$$

Un MLP così costruito funziona, ma quasi 400 mila parametri per immagini di 28×28 pixel sono tanti. Inoltre la rete tratta ogni pixel come una feature indipendente: la struttura spaziale dell'immagine va persa quando la srotoliamo.

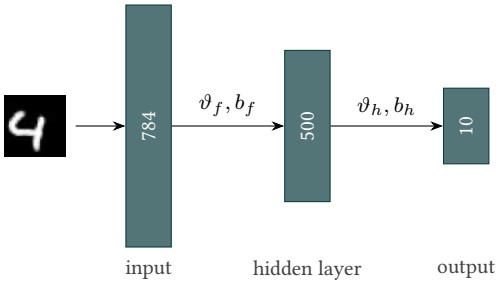


Figura 3.3. Un MLP per MNIST: l'immagine 28×28 viene srotolata in un vettore di 784 valori, seguito da un hidden layer con 500 neuroni e da un output layer con 10 neuroni, uno per cifra.

3.2 Convolutional Neural Network

Le Convolutional Neural Network sono progettate per far scalare le reti neurali a immagini molto grandi e a sequenze video. L'idea principale è **sostituire la moltiplicazione matriciale con la convoluzione** (figura 3.4).



Figura 3.4. L'idea alla base di una CNN: l'immagine in input attraversa una sequenza di convoluzioni, che prendono il posto delle moltiplicazioni matriciali di un MLP.

Osservazione (Invarianza alla traslazione). L'estrazione delle feature dovrebbe essere **invariante alla traslazione** lungo gli assi x e y. Pensiamo all'esempio di MNIST: un 7 resta un 7 sia che si trovi al centro dell'immagine, sia che sia spostato di qualche pixel. Una moltiplicazione matriciale imparerebbe pesi diversi in posizioni diverse, mentre la convoluzione, come vedremo, applica gli stessi pesi in tutte le posizioni e mantiene questa invarianza.

In altre parole, una CNN impara un certo numero di **filtri**, e li impara in modo che siano i migliori per risolvere il problema dato.

3.2.1 Architettura e componenti

Una CNN è una sequenza di layer, ognuno con i propri pesi W_l: prima alcuni layer convoluzionali, poi alcuni layer fully connected, che restituiscono la classe (figura 3.5).

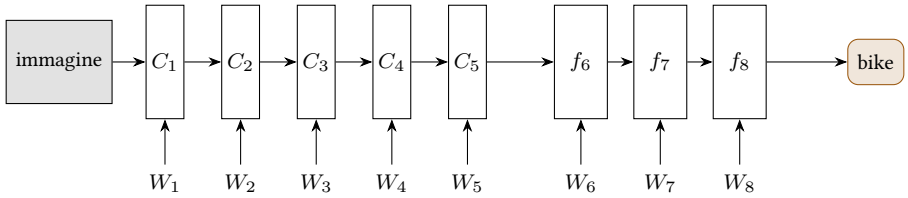


Figura 3.5. Architettura di una CNN (da [9]): cinque layer convoluzionali $C_1, \ldots, C_5$ e tre layer fully connected f_6, f_7, f_8, ciascuno con i propri pesi W_l. L'output è la classe dell'immagine in input, nell'esempio una bicicletta.

I layer convoluzionali sono costruiti combinando alcuni blocchi elementari (figura 3.6):

- **Filtri lineari 3D**: $y = F * x + b$, dove F è un banco di filtri e b il bias. È la convoluzione vera e propria, oggetto di questo capitolo.
- **ReLU**: $y = \max\{0, x\}$, la non linearità applicata dopo la convoluzione.
- **Downsampling**: riduce la dimensione spaziale dell'input.
- **Normalizzazione**: normalizza le risposte su gruppi di canali con una finestra scorrevole (sliding ℓ^2).
- **Spatial pooling**: $y_{ijk} = \max_{pq \in \Omega_{ij}} x_{pqk}$, ovvero per ogni canale k si prende il massimo in un intorno Ω_{ij} della posizione (i, j).

Gli ultimi quattro blocchi li vedremo nella sezione 3.4.

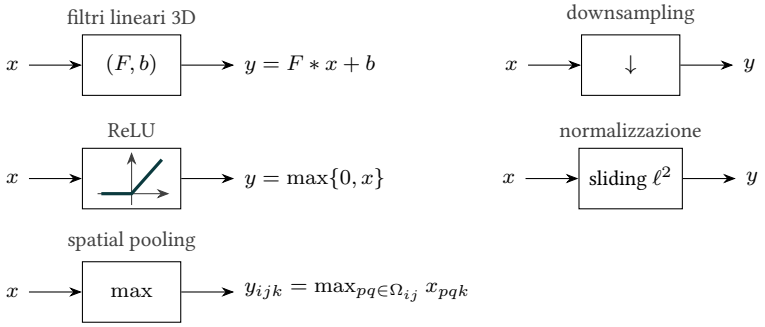


Figura 3.6. I blocchi elementari dei layer convoluzionali: filtri lineari 3D, downsampling, ReLU, normalizzazione e spatial pooling. Ciascun blocco riceve un input x e restituisce un output y.

3.2.2 La convoluzione 2D

Partiamo da un esempio (figura 3.7). Applichiamo a un'immagine in scala di grigi il kernel $[1, -1]$: ogni pixel dell'output è la differenza tra un pixel dell'input e il suo vicino a destra. Nelle zone uniformi la differenza è circa 0, e infatti l'output è grigio. Dove l'intensità cambia bruscamente in orizzontale, invece, la differenza è grande: l'output mette in evidenza i **bordi verticali** dell'immagine. Un kernel, quindi, risponde a una particolare caratteristica dell'input.

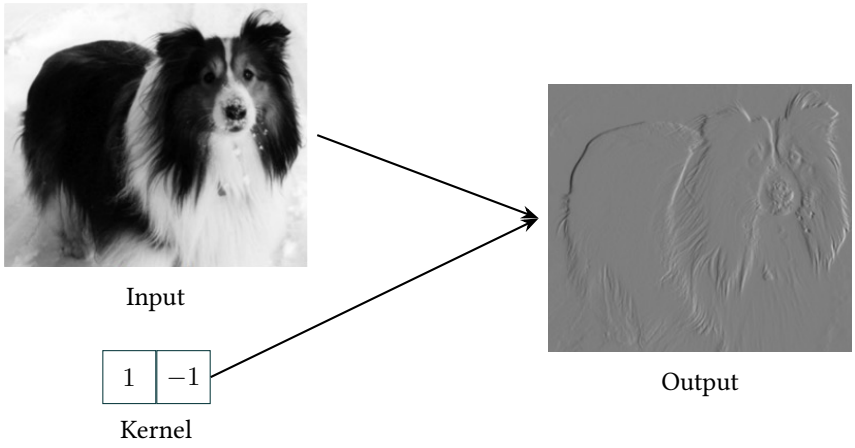


Figura 3.7. Esempio di convoluzione 2D (da [7]): il kernel $[1, -1]$ sottrae a ogni pixel il suo vicino a destra. Le zone uniformi diventano grigie, mentre i bordi verticali del cane vengono messi in evidenza.

In un layer possiamo imparare **più di un kernel**, in modo simile a quanto accade quando mettiamo più percettroni nello stesso layer di un MLP. Ogni kernel produce la propria uscita, e i layer successivi ricevono le uscite di tutti i kernel del layer precedente (figura 3.8).

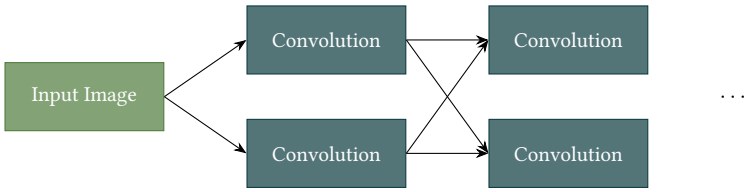


Figura 3.8. Più kernel per layer: ogni layer impara due convoluzioni, e ognuna di quelle del secondo layer riceve le uscite di entrambe quelle del primo.

Cross-correlation. I pesi da apprendere in un layer convoluzionale sono **i valori del kernel**. Consideriamo la figura 3.9: l'input è una matrice 3×4 e il kernel una matrice 2×2. Il kernel scorre sull'input e, in ogni posizione, moltiplica elemento per elemento i valori sovrapposti e li somma. Ad esempio, nella prima posizione l'output vale $aw + bx + ey + fz$. Il kernel ha solo 4 pesi, e questi restano 4 qualunque sia la dimensione dell'input.

Formalizziamo. Sia F l'immagine in input e h un kernel di dimensione $(2k+1) \times (2k+1)$, con indici che vanno da $-k$ a k. La **cross-correlation** si definisce come:

$$G = h \otimes F, \qquad G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} h[u,v]\, F[i+u, j+v]$$

mentre la **convoluzione** si definisce come:

$$G = h * F, \qquad G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} h[u,v]\, F[i-u, j-v]$$

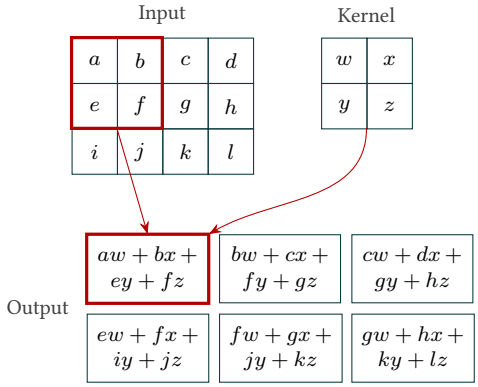


Figura 3.9. Cross-correlation 2D (da [7]): un kernel 2×2 scorre su un input 3×4 e produce un output 2×3. In rosso la prima posizione del kernel e il valore corrispondente dell'output. I pesi da apprendere sono soltanto i 4 valori del kernel.

dove G è l'output. L'unica differenza è il segno degli indici: la convoluzione equivale a una cross-correlation con il kernel **ribaltato** su entrambi gli assi, ovvero ruotato di $180°$ (figura 3.10).

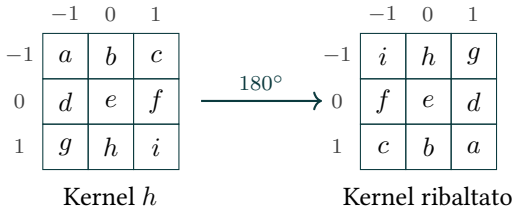


Figura 3.10. A sinistra un kernel 3×3 con indici da -1 a 1, a destra lo stesso kernel ribaltato su entrambi gli assi. Calcolare la convoluzione con il kernel di sinistra equivale a calcolare la cross-correlation con quello di destra.

Osservazione. Quella che nelle CNN chiamiamo convoluzione è in realtà, quasi sempre, una cross-correlation, come in figura 3.9. Nella pratica la differenza non conta: i valori del kernel vengono appresi, per cui una rete che usa la convoluzione imparerebbe semplicemente il kernel ribaltato.

Esempio. Consideriamo l'immagine 5×5 in scala di grigi della figura 3.11, divisa in una zona chiara (valore 2) e una scura (valore 8). Le applichiamo il kernel di *sharpening*:

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

Calcoliamo l'output della cross-correlation, facendo scorrere il kernel solo nelle posizioni in cui sta tutto dentro l'immagine.

Soluzione. Le posizioni possibili sono 3×3, quindi l'output è una matrice 3×3. Nella prima posizione il kernel copre una zona uniforme, tutta di valore 2, e restituisce $5 \cdot 2 - 4 \cdot 2 = 2$: le

zone uniformi restano invariate. Nella seconda posizione (in rosso in figura) il pixel centrale vale 2, mentre il suo vicino a destra vale 8:

$$G[0,1] = 5 \cdot 2 - (2 + 2 + 8 + 2) = -4.$$

Spostandoci di un'altra posizione il centro vale 8 e il vicino a sinistra 2, per cui $G[0,2] = 5 \cdot 8 - (8 + 2 + 8 + 8) = 14$. Il salto tra le due zone, che nell'input era da 2 a 8, nell'output diventa da -4 a 14: il kernel accentua i bordi, ed è proprio questo che rende l'immagine più nitida.

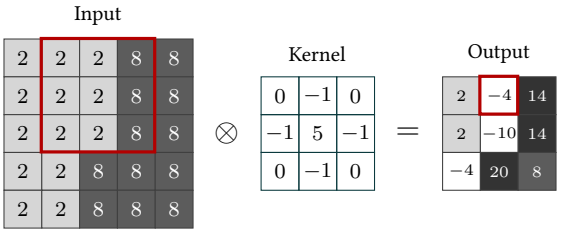


Figura 3.11. Il kernel di sharpening applicato a un'immagine 5×5. Il colore delle celle segue il valore (i valori negativi sono resi in bianco, come se fossero 0). In rosso la seconda posizione del kernel e il valore corrispondente dell'output, -4: lungo il bordo tra le due zone il contrasto aumenta.

3.2.3 Convoluzione 3D, padding, dilation e stride

La forma dell'output di un layer convoluzionale dipende dalla forma del suo input e da alcune scelte: la forma del kernel, lo **zero padding**, la **dilation** e lo **stride**. In ogni caso valgono tre cose:

- i pesi del layer corrispondono ai valori del kernel.
- i pesi sono **condivisi** all'interno di un canale (depth slice), ovvero lo stesso kernel viene usato in tutte le posizioni.
- stiamo di fatto imparando dei filtri che rispondono a parti, entità o indizi visivi dell'immagine.

Convoluzione 3D. Un'immagine a colori non è una matrice ma un **volume**: ad esempio un'immagine RGB ha 3 canali, per cui ha dimensione $W \times H \times 3$. In questo caso il kernel ha la stessa profondità dell'input: con un input di C canali, un filtro $F \times F$ ha dimensione $F \times F \times C$. In ogni posizione il filtro calcola la cross-correlation su ciascun canale, somma i risultati e aggiunge il bias, producendo **un solo valore**. Ogni filtro produce quindi una matrice 2D, e con K filtri l'output ha profondità K.

Esempio. Consideriamo un input $5 \times 5 \times 3$, a cui aggiungiamo un bordo di zeri ottenendo un volume $7 \times 7 \times 3$, e due filtri W_0 e W_1 di dimensione $3 \times 3 \times 3$, con bias $b_0 = 1$ e $b_1 = 0$. I filtri si spostano di 2 posizioni alla volta. Calcoliamo il primo valore dell'output del filtro W_0.

Soluzione. Il filtro W_0 si sovrappone all'angolo in alto a sinistra dei tre canali (in figura 3.12, i riquadri blu). Per ciascun canale calcoliamo la cross-correlation tra la porzione 3×3 dell'input e il canale corrispondente del filtro, ottenendo rispettivamente 2, 1 e 2. Sommando i tre contributi

e il bias:

$$o[0,0,0] = 2+1+2+b_0 = 2+1+2+1 = 6.$$

Ripetendo il calcolo in tutte le posizioni si ottiene una matrice 3×3. Facendo lo stesso con W_1 se ne ottiene un'altra. L'output ha quindi dimensione $3 \times 3 \times 2$, una matrice per filtro.

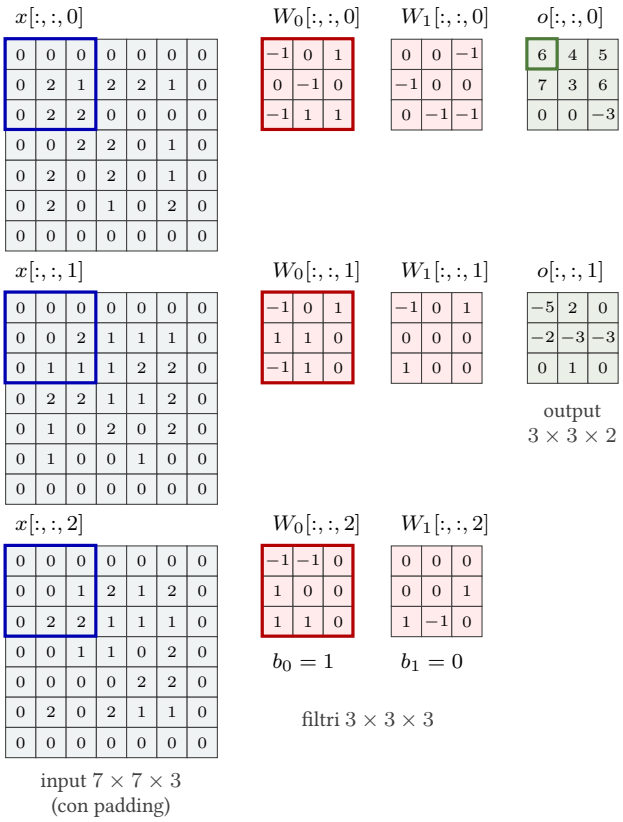


Figura 3.12. Una convoluzione 3D con due filtri e stride 2. A sinistra i tre canali dell'input, già circondati da un bordo di zeri. Al centro i tre canali dei filtri W_0 e W_1 con i rispettivi bias, a destra le due matrici dell'output, una per filtro. In blu la porzione di input coperta da W_0 (in rosso) nella prima posizione, in verde il valore corrispondente $o[0,0,0] = 6$.

Padding. Applicando un kernel 3×3 a un'immagine 5×5, come nell'esempio dello sharpening, l'output si è ridotto a 3×3: il kernel non può centrarsi sui pixel del bordo. Per evitarlo si usa lo **zero padding**, ovvero si aggiungono P bordi di zeri attorno all'input (figura 3.13). Ad esempio, un input di $32 \times 32 \times 3$ con due bordi di zeri ($P = 2$) diventa un volume di $36 \times 36 \times 3$. Applicando tre filtri $5 \times 5 \times 3$ con passo 1, si ottiene un output di $32 \times 32 \times 3$: la dimensione spaziale dell'input viene preservata.

Stride. Lo **stride** S è il passo con cui il kernel si sposta sull'input. Con $S = 1$ il kernel si sposta di una cella alla volta, mentre con $S = 2$ salta una posizione ogni volta, e l'output si riduce. Ad esempio, un kernel 3×3 su un input 7×7 produce un output 5×5 con $S = 1$, e un output 3×3 con $S = 2$ (figura 3.14).

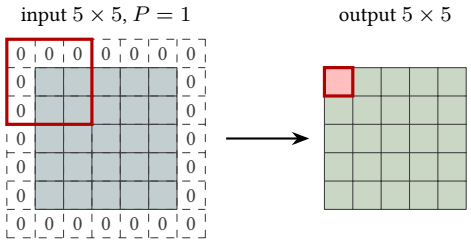


Figura 3.13. Zero padding (si veda [5]): un input 5×5 (in verde petrolio) viene circondato da un bordo di zeri ($P = 1$). Un kernel 3×3 con passo 1, di cui in rosso è riportata la prima posizione, produce un output 5×5, della stessa dimensione dell'input.

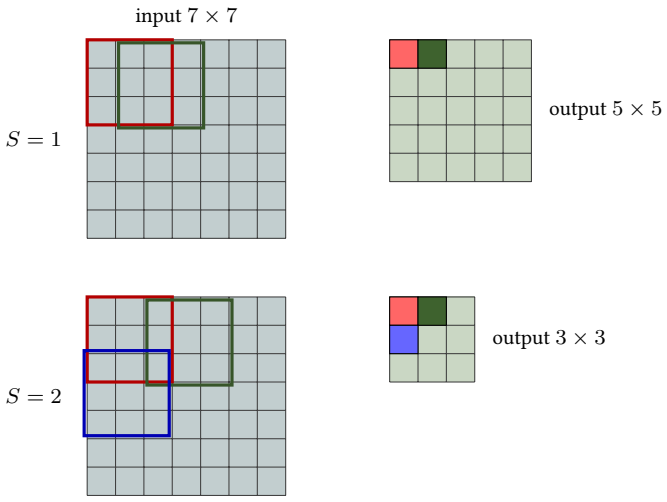


Figura 3.14. Lo stride, con un kernel 3×3 su un input 7×7. In alto, con $S = 1$, la seconda posizione del kernel (in verde) è spostata di una cella rispetto alla prima (in rosso) e l'output è 5×5. In basso, con $S = 2$, il kernel si sposta di due celle sia in orizzontale (in verde) sia in verticale (in blu), e l'output si riduce a 3×3. Ogni colore indica la posizione del kernel e il valore dell'output che produce.

Dilation. Con la **dilation** gli elementi del kernel non sono più adiacenti, ma distanziati: con un fattore di dilation d tra un elemento e l'altro restano $d-1$ celle libere. Un kernel 3×3 con $d = 2$, ad esempio, copre un'area di 5×5 pur avendo sempre 9 pesi (figura 3.15).

In generale, un kernel di lato F con dilation d copre un'area di lato

$$F + (F-1)(d-1).$$

Il punto più importante è che le convoluzioni dilatate permettono un'espansione esponenziale del receptive field (la porzione di input che influenza un valore dell'output) senza perdere risoluzione né copertura. Si ottiene quindi un receptive field più ampio con lo stesso costo in calcolo e in memoria.

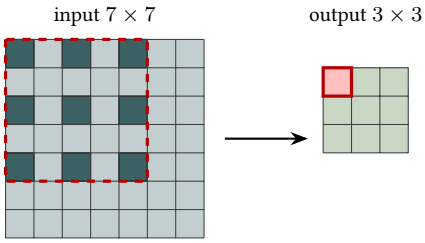


Figura 3.15. Convoluzione dilatata (si veda [5]): un kernel 3×3 con $d = 2$ applicato a un input 7×7. Le celle scure sono quelle moltiplicate per i 9 pesi del kernel nella prima posizione, mentre il riquadro tratteggiato è l'area 5×5 che coprono. Con stride 1 l'output è 3×3.

3.2.4 Local receptive field e shared weights

Vediamo ora da dove nasce il risparmio di parametri. In una rete neurale classica ogni nodo è **fully connected**, ovvero collegato a tutti i nodi del layer adiacente. Con un'immagine di 1000×1000 pixel, ogni neurone del primo hidden layer ha quindi un milione di parametri. Le CNN risolvono il problema con tre strumenti:

- **Local receptive field**.
- **Shared weights**.
- **Pooling layer**, che riducono la dimensionalità man mano che si va in profondità (li vedremo nella sezione 3.4.4).

Local receptive field. Ogni neurone di un hidden layer è collegato soltanto a una regione dell'input, detta **receptive field**, e non a tutta l'immagine. Ogni connessione serve ad apprendere un peso, per cui ogni nodo ha di fatto un proprio filtro. Ad esempio, con un receptive field di 3×3 ogni neurone ha 9 connessioni (figura 3.16, al centro).

Shared weights. Una feature può comparire in qualunque punto dell'immagine. Per questo motivo, i neuroni di uno stesso layer **condividono i pesi**: sono tutti sensibili alla stessa feature (la risposta del filtro), calcolata in posizioni diverse dell'input (figura 3.16, a destra). Una mappa di pesi applicata in posizioni diverse è esattamente una convoluzione.

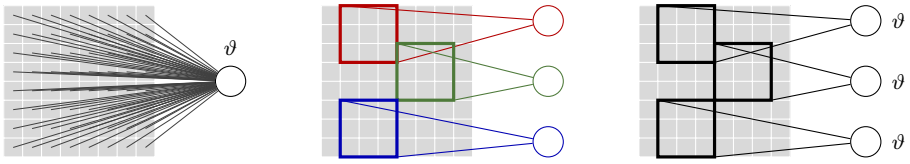


Figura 3.16. A sinistra, un neurone fully connected: ogni pixel dell'immagine è collegato al neurone con un proprio peso. Al centro, con il local receptive field ogni neurone vede solo una regione 3×3 dell'immagine, con i propri pesi (un colore per neurone). A destra, con gli shared weights tutti i neuroni usano gli stessi pesi ϑ in posizioni diverse: è una convoluzione.

Torniamo al nostro esempio di MNIST. Nell'MLP ogni neurone dell'hidden layer aveva 784 pesi, uno per pixel, e il solo primo layer contava $500 \cdot 784 + 500 = 392\,500$ parametri. Un filtro 5×5, invece, ha 25 pesi più un bias, e questo numero non dipende dalla dimensione dell'immagine. Anche imparando 6 filtri diversi, il layer convoluzionale ha appena $6 \cdot (25 + 1) = 156$ parametri.

3.2.5 Dimensione dell'output

Mettendo insieme tutto quello che abbiamo visto, possiamo calcolare la dimensione dell'output di un layer convoluzionale. Siano:

- $W \times H$ la dimensione dell'input.
- $F_w \times F_h$ la dimensione del kernel.
- P la dimensione del padding.
- S_w e S_h lo stride rispettivamente in larghezza e in altezza.

La dimensione dell'output è:

$$\text{output width} = \frac{W - F_w + 2P}{S_w} + 1, \qquad \text{output height} = \frac{H - F_h + 2P}{S_h} + 1.$$

Con una dilation $d > 1$ basta sostituire alla dimensione del kernel quella dell'area che copre, $F + (F - 1)(d - 1)$.

Possiamo verificare la formula sulle figure precedenti. Per lo stride vale $(7 - 3 + 0)/2 + 1 = 3$ e per il padding $(5 - 3 + 2)/1 + 1 = 5$. Per la convoluzione 3D otteniamo $(5 - 3 + 2)/2 + 1 = 3$, e per la dilation $(7 - 5 + 0)/1 + 1 = 3$. Su MNIST, un filtro 5×5 senza padding e con stride 1 produce un output di $(28 - 5)/1 + 1 = 24$ pixel di lato, mentre con $P = 2$ la dimensione resta 28.

Osservazione. Se il risultato della formula non è un numero intero, il valore dello stride non è corretto. Ad esempio, con $W = 10$, $P = 0$, $F = 3$ e $S = 5$:

$$\frac{W - F + 2P}{S} + 1 = \frac{10 - 3 + 0}{5} + 1 = \frac{7}{5} + 1$$

che non è intero: il kernel non riesce a coprire l'input con passi regolari. Senza padding, lo stride può valere solo 1 o 7, gli unici divisori di 7. In questi casi lo zero padding ci viene in aiuto: con $S = 3$, ad esempio, basta $P = 1$ per ottenere $(10 - 3 + 2)/3 + 1 = 4$.

3.2.6 Un esempio reale: AlexNet

Applichiamo la formula a una rete reale, **AlexNet** [9], ovvero la CNN di figura 3.5. L'input è un'immagine RGB di $227 \times 227 \times 3$ pixel[2]. Il primo layer convoluzionale ha 96 filtri di $11 \times 11 \times 3$, con stride $S = 4$ e senza padding. La dimensione dell'output è quindi:

$$\frac{227 - 11 + 0}{4} + 1 = 55$$

ovvero ogni filtro produce una mappa di 55×55. Il primo layer ha $96 \cdot 11 \cdot 11 \cdot 3 = 34\,848$ pesi, più 96 bias (uno per filtro), mentre le unità di output sono $55 \cdot 55 \cdot 96 = 290\,400$.

La tabella 3.1 riporta gli stessi conti per tutti i layer. Il dato interessante è come si distribuiscono i pesi. I cinque layer convoluzionali hanno circa 2.3 milioni di pesi, meno del 4% del totale, ma contano circa il 92% delle connessioni (e quindi delle operazioni). I tre layer fully connected, al contrario, hanno quasi 59 milioni di pesi: è lì che si concentra la memoria occupata dalla rete.

Layer	Unità	Pesi	Connessioni
L_1 (conv)	290 400	34 848	105 415 200
L_2 (conv)	186 624	307 200	223 948 800
L_3 (conv)	64 896	884 736	149 520 384
L_4 (conv)	64 896	663 552	112 140 288
L_5 (conv)	43 264	442 368	74 760 192
L_6 (dense)	4 096	37 748 736	37 748 736
L_7 (dense)	4 096	16 777 216	16 777 216
L_8 (dense)	1 000	4 096 000	4 096 000
Totale conv	650 080	2 332 704	665 784 864
Totale dense	9 192	58 621 952	58 621 952
Totale	659 272	60 954 656	724 406 816

Tabella 3.1. Unità, pesi e connessioni dei layer di AlexNet [9]. I layer *dense* sono i layer fully connected, e i pesi non includono i bias.

[2] Il paper riporta un input di 224×224, ma con questa dimensione la formula non restituisce un numero intero. I conti tornano con 227×227, che è la dimensione usata nelle implementazioni della rete.

3.3 Sparse connections e parameter sharing

Riprendiamo le idee di local receptive field e shared weights con lo schema di [7], in cui input e output sono vettori 1D. In questo modo è più semplice vedere quali connessioni ci sono e quali parametri vengono usati.

3.3.1 Sparse connections

In una rete fully connected ogni output dipende da tutti gli input. In un layer convoluzionale, invece, ogni output dipende solo dagli input coperti dal kernel: si parla quindi di **sparse connections** (connessioni sparse). Possiamo guardare la cosa da due punti di vista (figura 3.17):

- **dal basso**: fissato un input, ad esempio x_3, con un kernel di larghezza 3 questo influenza solo tre output (s_2, s_3 e s_4). Con la moltiplicazione matriciale, invece, li influenza tutti.
- **dall'alto**: fissato un output, ad esempio s_3, questo dipende solo da tre input (x_2, x_3 e x_4). Questi input formano il **receptive field** di s_3.

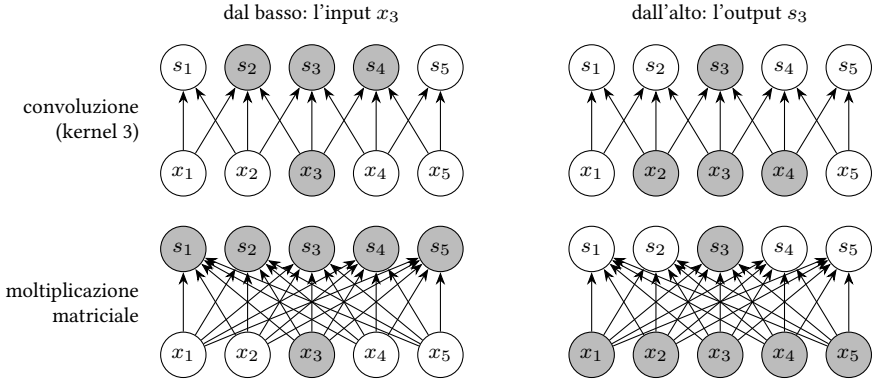


Figura 3.17. Sparse connections (da [7]). A sinistra, le unità di output influenzate dall'input x_3: con un kernel di larghezza 3 sono solo tre, con la moltiplicazione matriciale sono tutte. A destra, le unità di input che influenzano l'output s_3, ovvero il suo receptive field.

Il receptive field cresce man mano che si va in profondità (figura 3.18). Un'unità del secondo layer, ad esempio g_3, è collegata direttamente solo a tre unità del layer precedente. Ognuna di queste, però, dipende a sua volta da tre input, per cui g_3 dipende indirettamente da tutti e cinque gli input. L'effetto aumenta se la rete usa lo stride o il pooling (sezione 3.4.4). Quindi, anche se le connessioni dirette sono sparse, le unità dei layer più profondi possono essere collegate indirettamente a tutta l'immagine (o quasi).

3.3.2 Parameter sharing

La convoluzione usa gli stessi parametri in tutte le posizioni spaziali, mentre la moltiplicazione matriciale non condivide nessun parametro. Lo vediamo in figura 3.19. Nel caso della convoluzione, le frecce nere indicano tutti gli usi dell'elemento centrale di un kernel di 3 elementi: grazie al **parameter sharing**, quel singolo parametro viene usato in tutte le posizioni dell'input. Nel modello fully connected, invece, il peso che collega x_3 a s_3 viene usato una sola volta.

Usare le convoluzioni ha quindi tre effetti:

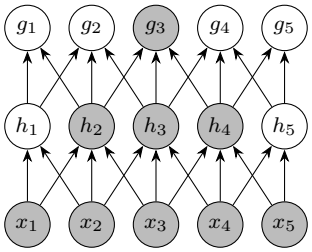


Figura 3.18. Il receptive field delle unità nei layer profondi (da [7]). L'unità g_3 è collegata direttamente a h_2, h_3 e h_4, ma indirettamente a tutti gli input.

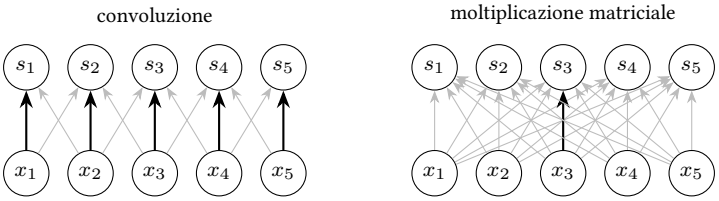


Figura 3.19. Parameter sharing (da [7]). A sinistra, le frecce nere sono tutti gli usi dell'elemento centrale del kernel: lo stesso parametro viene usato in ogni posizione. A destra, la freccia nera è l'unico uso dell'elemento della matrice dei pesi che collega x_3 a s_3.

- il numero di parametri di un layer **non dipende dalla dimensione dell'input**.
- il numero di parametri di un layer **si riduce**.
- l'estrazione delle feature diventa **invariante alle traslazioni** lungo x e y.

Poiché i parametri sono pochi, possiamo costruire modelli molto profondi, ad esempio con 150 layer.

3.3.3 Convoluzione efficiente

Ci sono due modi per rendere più veloce il calcolo di una convoluzione. Il primo sfrutta il fatto che una convoluzione nel dominio del tempo (o dello spazio) equivale a una moltiplicazione nel dominio della frequenza. Per alcune dimensioni del problema conviene quindi portare i segnali nel dominio della frequenza, moltiplicarli e riportare il risultato indietro.

Il secondo riguarda i **kernel separabili**. Un kernel si dice separabile se si può scrivere come prodotto di due vettori, ad esempio:

$$\frac{1}{4}\begin{bmatrix}1\\2\\1\end{bmatrix} * \frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix} = \frac{1}{16}\begin{bmatrix}1 & 2 & 1\\2 & 4 & 2\\1 & 2 & 1\end{bmatrix}.$$

Un kernel separabile occupa meno memoria, perché basta salvare i due vettori. Inoltre convolvere l'input prima con un vettore e poi con l'altro è più veloce della convoluzione ingenua: per un kernel $F \times F$ servono $2F$ moltiplicazioni per pixel invece di F^2.

3.4 I layer di una CNN

Torniamo ai blocchi elementari di figura 3.6. Dei filtri lineari abbiamo già parlato. In questa sezione vediamo le funzioni di attivazione, la normalizzazione, il pooling (che si occupa anche del downsampling) e infine i layer fully connected che chiudono la rete.

In letteratura si usano due terminologie per descrivere questi blocchi (figura 3.20). Nella prima, un layer convoluzionale è un blocco complesso composto da più *stage*: la convoluzione (una trasformazione affine), il *detector stage* (la non linearità) e il pooling. Nella seconda, ogni passo è un layer a sé, per cui non tutti i layer hanno dei parametri.

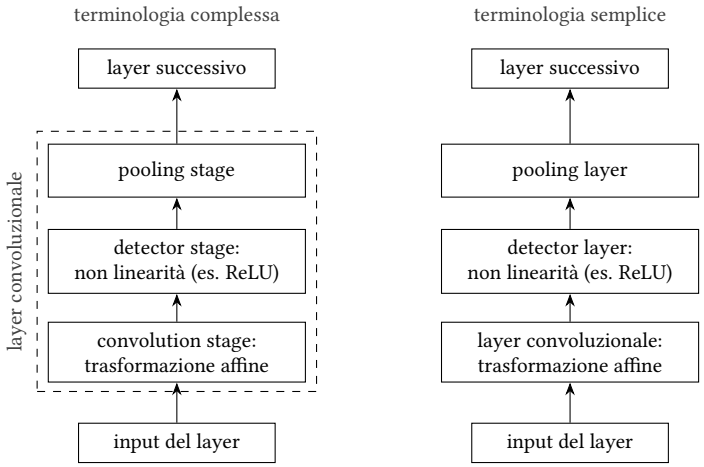


Figura 3.20. Le due terminologie per descrivere un layer di una CNN (da [7]). A sinistra, il layer convoluzionale è un blocco con tre stage. A destra, ogni passo è un layer a sé.

3.4.1 Funzioni di attivazione

Una combinazione di funzioni lineari è a sua volta una funzione lineare. Anche la convoluzione è lineare, per cui una sequenza di convoluzioni equivale a un'unica trasformazione lineare. Abbiamo quindi bisogno di una funzione non lineare dopo ogni convoluzione.

Le funzioni di attivazione le abbiamo già viste nella sezione 1.6. Nelle CNN la scelta più comune è la **ReLU** con le sue varianti, per tre motivi: è più veloce da calcolare, semplifica la backpropagation (la derivata vale 0 oppure 1) ed evita i problemi di saturazione. La sigmoide e la tangente iperbolica, infatti, hanno una derivata quasi nulla per valori grandi di $|x|$, e il gradiente si spegne.

La ReLU ha però un difetto quando i dati sono centrati in 0. In questo caso circa metà degli input è negativa, e la ReLU restituisce 0 con gradiente nullo: per quegli input il neurone non impara. Le varianti della ReLU mantengono quindi una piccola risposta anche per i valori negativi (figura 3.21):

- **Leaky ReLU**: $f(x) = \max(0.1x, x)$, o più in generale $f(x) = \epsilon x + (1 - \epsilon)\max\{0, x\}$ con ϵ piccolo.
- **Parametric ReLU**: come la Leaky ReLU, ma la pendenza α della parte negativa, $f(x) = \max(\alpha x, x)$, viene appresa durante il training.

- **ELU** (Exponential Linear Unit):

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases} \qquad f'(x) = \begin{cases} 1 & x > 0 \\ f(x) + \alpha & x \leq 0. \end{cases}$$

Tra le varianti c'è anche la Soft ReLU, $f(x) = \log(1 + e^x)$, ovvero la softplus già vista nella sezione 1.6.

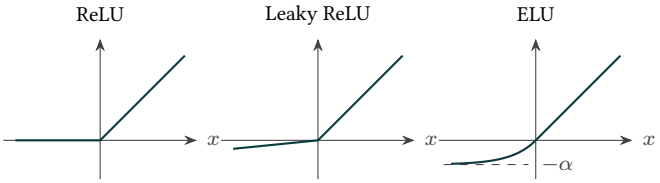


Figura 3.21. La ReLU e due sue varianti: la Leaky ReLU, con pendenza 0.1 per $x < 0$, e la ELU con $\alpha = 1$, che per $x \to -\infty$ tende a $-\alpha$.

3.4.2 Feature map

Un layer convoluzionale è quindi una trasformazione lineare (la convoluzione) seguita da una funzione di attivazione. Ogni filtro cattura una specifica feature del layer precedente, e la CNN impara molti filtri per ogni layer. L'output di ogni filtro, dopo la funzione di attivazione, si chiama **feature map** (figura 3.22). Le feature map di un layer, impilate lungo la profondità, diventano l'input del layer successivo.

Torniamo al nostro esempio di MNIST: con 6 filtri 5×5, senza padding e con stride 1, il primo layer produce 6 feature map di 24×24. Il layer successivo riceve quindi un volume di $24 \times 24 \times 6$, e i suoi filtri avranno profondità 6.

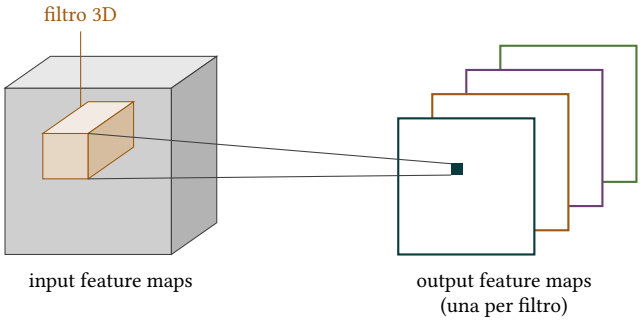


Figura 3.22. Un layer convoluzionale. Ogni filtro 3D ha la stessa profondità dell'input e, dopo la funzione di attivazione, produce una feature map. Le feature map dei diversi filtri (una per colore) formano l'output del layer.

3.4.3 Normalizzazione

Un altro blocco è la **normalizzazione locale** delle feature, usata ad esempio in AlexNet. Normalizza le risposte dei neuroni lungo i canali (*feature channel*) e non spazialmente:

$$y_{ijk} = x_{ijk}\Big(\kappa + \alpha \sum_{q \in G(k)} x_{ijq}^2\Big)^{-\beta}$$

dove i, j indicano la posizione spaziale, k il canale, $G(k)$ è un gruppo di canali vicini a k, mentre κ, α e β sono iperparametri. La normalizzazione lavora su ogni posizione spaziale in modo indipendente, e i gruppi $G(k)$ sono definiti con una finestra scorrevole lungo i canali (è il blocco "sliding ℓ^2" di figura 3.6). È utile soprattutto quando le attivazioni non sono limitate, come nel caso della ReLU. Nel capitolo 4 vedremo la batch normalization, che normalizza invece le attivazioni sul mini-batch.

3.4.4 Pooling

Le CNN hanno anche dei **pooling layer**. Un pooling layer prende delle regioni di una feature map e per ognuna seleziona un unico valore rappresentativo. I due tipi più comuni sono il **max pooling**, che prende il massimo, e l'**average pooling**, che prende la media. Il pooling serve a tre cose:

- rende le feature **più robuste rispetto alla posizione**, ovvero a piccoli spostamenti dell'input.
- riduce la dipendenza da una posizione specifica.
- riduce la **complessità computazionale**, perché riduce la dimensione delle feature map.

Il pooling si applica a ogni canale in modo indipendente, quindi la profondità dell'output resta la stessa dell'input. Inoltre l'operazione deve essere derivabile, perché la rete va addestrata con la backpropagation (vedremo tra poco come si deriva il massimo).

Max pooling. Vorremmo che le nostre reti fossero invarianti a piccoli spostamenti lungo x e y: ad esempio, l'immagine potrebbe non essere perfettamente centrata sull'oggetto. Per ottenere questo si usa di solito il max pooling, che nella forma più semplice sottocampiona la feature map selezionando i massimi locali (figura 3.23). Il max pooling è ispirato all'**inibizione laterale**: un neurone eccitato può ridurre l'attività dei suoi vicini.

Il pooling può essere *overlapping* oppure *non-overlapping*, a seconda che le regioni si sovrappongano o meno. Un esempio di pooling non-overlapping è il max pooling con filtro 2×2 e stride 2. Le dimensioni dell'output si calcolano con la stessa formula vista per la convoluzione, senza padding: un input 4×4 diventa $(4 - 2)/2 + 1 = 2$, ovvero 2×2.

Max pooling e average pooling. Entrambi comprimono le feature estratte, ma in modo diverso (figura 3.24). Se un blocco ha per lo più attivazioni piccole e qualche attivazione grande, il max pooling tiene solo quella grande, e l'informazione sulle attivazioni piccole va persa. L'average pooling, invece, conserva qualcosa delle attivazioni basse. Tuttavia, se in un blocco grandi attivazioni positive sono bilanciate da attivazioni negative, la media è vicina a 0 e l'output sembra non attivo.

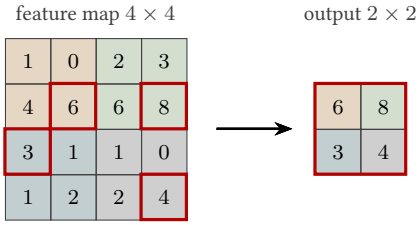


Figura 3.23. Max pooling non-overlapping con filtro 2×2 e stride 2. Ogni regione colorata dell'input viene sostituita dal suo massimo (in rosso).

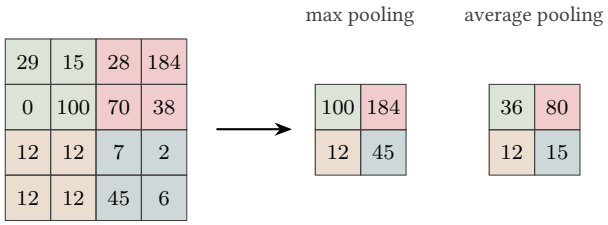


Figura 3.24. Max pooling e average pooling 2×2 con stride 2 sulla stessa feature map. Nella regione in basso a sinistra i valori sono tutti uguali e i due pooling coincidono, nelle altre il max pooling perde l'informazione sui valori piccoli.

Invarianza ad altre trasformazioni. Il max pooling sulle posizioni spaziali rende la rete invariante alle traslazioni. Se invece facciamo pooling sugli output di convoluzioni diverse, la rete può imparare a essere invariante anche ad altre trasformazioni, ad esempio alle rotazioni (figura 3.25). Supponiamo di avere tre filtri, ognuno dei quali riconosce un 5 scritto a mano con un'inclinazione diversa. Quando in input arriva un 5 ruotato, il filtro con l'inclinazione giusta risponde con un'attivazione alta. L'unità di pooling, che prende il massimo dei tre, ha quindi una risposta alta qualunque sia il filtro che si è attivato.

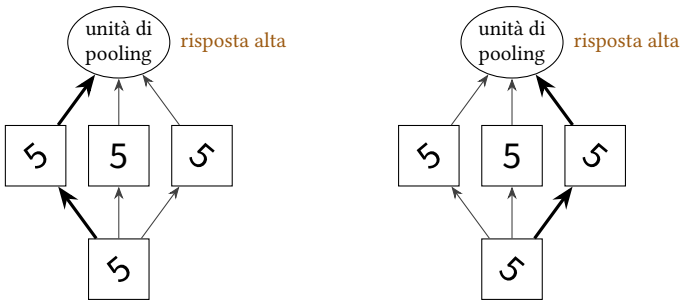


Figura 3.25. Invarianza appresa con il pooling su più filtri (da [7]). I tre filtri riconoscono un 5 con tre inclinazioni diverse. A sinistra si attiva il primo filtro, a destra il terzo, ma in entrambi i casi l'unità di pooling ha una risposta alta.

Pooling e downsampling. Il pooling può anche ridurre la dimensione della rappresentazione. Basta usare uno stride tra le regioni di pooling maggiore di 1 (figura 3.26). In questo modo si riduce il carico computazionale e statistico del layer successivo.

Per molti task, inoltre, il pooling è indispensabile per gestire **input di dimensione variabile**. Se vogliamo classificare immagini di dimensioni diverse, l'input del layer di classificazione deve comunque avere una dimensione fissa. Per ottenerla si varia la dimensione delle regioni di pooling (e la distanza tra loro), in modo che il layer di classificazione riceva sempre lo stesso numero di valori. Ad esempio, l'ultimo pooling layer potrebbe restituire quattro valori, uno per ogni quadrante dell'immagine, qualunque sia la dimensione dell'immagine.

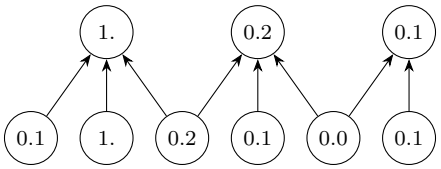


Figura 3.26. Max pooling con regioni di larghezza 3 e stride 2 (da [7]). La rappresentazione si dimezza circa. L'ultima regione è più piccola, ma va inclusa per non ignorare alcune unità.

Backpropagation nel pooling. Come si deriva il massimo? Ricordiamo la derivata di $f(x, y) = \max(x, y)$:

$$\nabla f = \big[\mathbf{1}\{x \geq y\},\ \mathbf{1}\{y > x\}\big].$$

Il gradiente vale 1 per l'input più grande e 0 per l'altro. Nel backward pass, quindi, il max pooling instrada il gradiente solo verso l'input che aveva il valore più grande nel forward pass. Nell'esempio di figura 3.23, il gradiente che arriva sull'output 6 passa tutto alla cella che conteneva il 6, mentre le altre tre celle della regione ricevono 0. Per farlo in modo efficiente, durante il forward pass si salva l'indice del massimo di ogni regione (a volte questi indici sono chiamati *switches*).

3.4.5 Fully connected layer

In cima alla rete ci sono uno o più **layer fully connected** (FC), che mappano le risposte delle feature nelle etichette delle classi. L'output dell'ultimo layer convoluzionale (o di pooling) viene srotolato in un vettore, come facevamo con l'MLP per le immagini di MNIST, e diventa l'input del primo FC.

L'ultimo layer è fully connected e ha un numero di neuroni pari al numero di classi. Il suo output è il punteggio di probabilità rispetto alle classi, ad esempio calcolato con il softmax. Nel caso di MNIST, l'ultimo FC ha 10 neuroni, uno per cifra. La figura 3.27 mostra una CNN completa, dall'immagine in input fino ai punteggi delle classi.

3.5 Regole pratiche

Chiudiamo il capitolo con alcune regole empiriche (*rules of thumb*) per progettare una CNN.

- **Input layer**: la dimensione dell'input dovrebbe essere divisibile per 2, meglio se è una potenza di 2 (ad esempio 256 o 512). In questo modo i pooling che dimezzano l'input restituiscono sempre dimensioni intere.

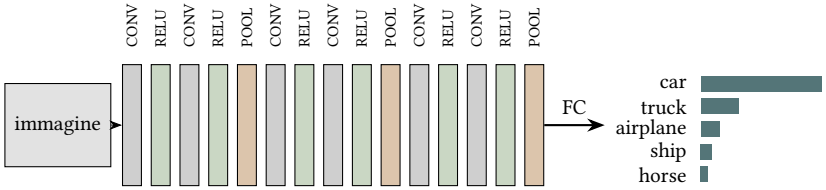


Figura 3.27. Una CNN completa: tre blocchi CONV, RELU, CONV, RELU, POOL estraggono le feature, e il layer fully connected in cima restituisce un punteggio per ogni classe. Nell'esempio l'immagine è un'automobile (*car*).

- **Layer convoluzionali**: si usano filtri piccoli con stride 1, e di solito lo zero padding per lasciare invariata la dimensione dell'input. In generale, fissato F, scegliendo $P = (F-1)/2$ con $S = 1$ si ha:

$$\frac{W - F + 2P}{S} + 1 = W - F + (F-1) + 1 = W.$$

  È il caso di MNIST con un filtro 5×5 e $P = 2$.

- **Pooling layer**: sono comuni il max pooling con $F = 3$ e $S = 2$ oppure con $F = 2$ e $S = 2$. Un pooling con un receptive field F più grande può essere distruttivo, perché butta via troppa informazione.

Downsampling. A un certo punto della rete bisogna ridurre la dimensione. Se i layer convoluzionali non la riducono, se ne occupano solo i pooling layer. Se invece la riducono anche i layer convoluzionali, bisogna scegliere con attenzione stride e simili. I requisiti sulle dimensioni di tutti i layer devono essere soddisfatti (la formula deve restituire numeri interi), e i layer devono incastrarsi correttamente. In pratica $S = 1$ funziona bene, ma per input più grandi si possono provare stride maggiori (come nel primo layer di AlexNet, con $S = 4$).

Inizializzazione dei pesi. Come si inizializzano i pesi? La prima opzione è inizializzarli in modo casuale, e in letteratura si è visto che funziona bene. La seconda è ottenere i filtri altrove e usarli come pesi iniziali: ad esempio addestrandoli su un altro problema, oppure con un pre-training non supervisionato su patch (finestre) di immagini. Così la ricerca dei parametri parte da una posizione migliore, e si risparmiano interi forward e backward pass. Si può addirittura evitare di addestrare i layer convoluzionali. È l'idea alla base del transfer learning, che vedremo nel capitolo 4.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (22 e 27 ottobre 2026).
- Slide del corso *Convolutional Neural Net*, fino all'inizializzazione dei pesi.

- Capitolo 9, *Convolutional Networks*, del libro *Deep Learning* [7], da cui sono tratti l'esempio del kernel $[1, -1]$, la figura della cross-correlation e gli schemi su sparse connections, parameter sharing e pooling.
- L'architettura in figura 3.5 e la tabella 3.1 da «ImageNet Classification with Deep Convolutional Neural Networks» [9]. Le figure su padding, stride e dilation seguono le convenzioni di *A guide to convolution arithmetic for deep learning* [5].

Capitolo 4

Addestrare una CNN

Nel capitolo 3 abbiamo visto come è fatta una CNN. Addestrarla, però, non è banale: le reti profonde hanno milioni di parametri (AlexNet ne ha circa 61 milioni), e il rischio di overfitting è alto, soprattutto quando i dati sono pochi. In questo capitolo vediamo in breve alcune tecniche usate in pratica: tre tecniche di regolarizzazione (dropout, batch normalization e data augmentation) e il transfer learning, che permette di riutilizzare una rete già addestrata.

4.1 Dropout

Il **dropout** [16] è un modo semplice per ridurre l'overfitting. In fase di training, a ogni passo, ogni neurone viene "spento" in modo casuale: resta attivo con probabilità p, altrimenti viene rimosso insieme alle sue connessioni. Si ottiene così una rete **assottigliata** (*thinned*), diversa a ogni passo (figura 4.1).

Perché funziona? Spegnendo a caso alcuni neuroni, la rete non può affidarsi a un unico percorso per riconoscere qualcosa. Il significato semantico di quello che sta imparando viene quindi distribuito su più percorsi, e la rete generalizza meglio.

In fase di test, invece, i neuroni sono sempre attivi. Per compensare, i pesi in uscita da ogni neurone vengono moltiplicati per p: se in training un neurone aveva pesi w, al test usa pw. In questo modo l'output al test è uguale all'output atteso in fase di training.

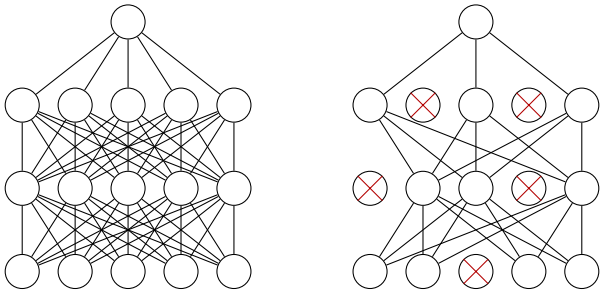


Figura 4.1. Dropout (da [16]). A sinistra, una rete standard con due hidden layer. A destra, una rete assottigliata ottenuta applicando il dropout alla rete di sinistra: i neuroni barrati sono stati spenti, insieme alle loro connessioni.

4.2 Batch normalization

La **batch normalization** [8] normalizza l'input e gli output delle funzioni di attivazione, così da accelerare il training delle reti profonde. Dato un mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$ con i valori di un'attivazione x, calcoliamo:

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{media del mini-batch}$$

$$\sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{varianza del mini-batch}$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{normalizzazione}$$

$$y_i = \gamma\hat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{scale and shift}$$

dove ϵ è una piccola costante che evita la divisione per zero, mentre γ e β sono parametri appresi insieme agli altri pesi della rete. Grazie a γ e β la rete può riscalare e traslare le attivazioni normalizzate, e se serve può anche annullare la normalizzazione.

In fase di test non c'è un mini-batch su cui calcolare media e varianza. Si usano quindi la media e la varianza stimate sull'intero training set, calcolate come medie su più mini-batch.

La batch normalization ha due vantaggi:

- **si possono usare learning rate più alti**, perché la normalizzazione evita che qualche attivazione diventi troppo grande o troppo piccola. Il training è quindi più veloce.
- **riduce l'overfitting**, perché ha un leggero effetto di regolarizzazione: come il dropout, aggiunge un po' di rumore alle attivazioni di ogni hidden layer.

Con la batch normalization possiamo quindi usare meno dropout, e perdere meno informazione. Tuttavia non conviene affidarsi solo alla batch normalization per la regolarizzazione: è meglio usarla insieme al dropout.

4.3 Data augmentation

La **data augmentation** è anch'essa uno schema di regolarizzazione. L'idea è aumentare la varianza dei pattern del training set generando nuovi esempi a partire da quelli che abbiamo. Si applicano trasformazioni affini ai dati di training: per ogni immagine di input si crea un "duplicato" traslato, ingrandito o rimpicciolito, ruotato, ribaltato, distorto o con una tonalità di colore diversa. L'immagine originale e il duplicato entrano nella rete con la stessa etichetta.

Torniamo al nostro esempio di MNIST (figura 4.2). Da un'unica immagine di un 4 possiamo ottenere diversi esempi, tutti etichettati come 4. Le trasformazioni vanno però scelte in modo opportuno: un 4 ribaltato orizzontalmente non è più una cifra valida, mentre per la foto di un animale il ribaltamento orizzontale va benissimo. In letteratura sono stati proposti anche altri schemi di augmentation, più sofisticati.



Figura 4.2. Data augmentation su un'immagine di MNIST. Dall'immagine originale otteniamo nuovi esempi con rotazioni, zoom e traslazioni, che entrano nella rete con la stessa etichetta.

4.4 Transfer learning

Addestrare una rete profonda *from scratch*, ovvero partendo da pesi inizializzati in modo casuale, è difficile, perché è raro avere un dataset abbastanza grande. Di solito, quindi, si parte da una rete **pre-addestrata** su un dataset molto grande (ad esempio ImageNet, EPIC-Kitchens o Ego4D). La rete viene poi usata come inizializzazione, oppure come feature extractor fisso, per il task che ci interessa. Questa idea si chiama **transfer learning**: si riutilizzano le informazioni (soprattutto quelle di basso livello) apprese da una rete per estrarre feature su un problema diverso.

Le strategie principali sono tre (figura 4.3).

Fixed feature extractor. Prendiamo un modello pre-addestrato su un task (ad esempio object recognition), rimuoviamo l'ultimo layer fully connected e usiamo il resto della rete come feature extractor fisso per il nuovo dataset. Una volta estratte le feature dal nuovo dataset, addestriamo su di esse un classificatore, ad esempio un softmax. I parametri della rete originale restano fissi durante il training del classificatore. Come classificatore si può usare anche un metodo basato sulle distanze, come il KNN che vedremo nel capitolo 5.

Fine-tuning. Sostituiamo e riaddestriamo il classificatore sul nuovo dataset, ma aggiorniamo anche i pesi della rete pre-addestrata continuando la backpropagation. Si possono aggiornare tutti i layer, oppure tenere fissi i primi (per evitare l'overfitting) e aggiornare solo la parte più alta della rete. In pratica, con il nostro dataset addestriamo gli ultimi k layer, e la backpropagation aggiorna solo i loro parametri.

Perché tenere fissi i primi layer? I primi layer di una rete profonda contengono feature generiche, come rilevatori di bordi (*edge detector*) o di macchie di colore (*color blob detector*), utili per molti task. I layer successivi, invece, diventano via via più specifici per le classi del dataset originale.

Modelli pre-addestrati. I ricercatori rilasciano spesso i *checkpoint* finali dei loro modelli, in modo che altri possano usarli per il fine-tuning. Ad esempio, la libreria PyTorch ha un sotto-pacchetto (`torchvision.models`) con le definizioni di diverse architetture e i relativi pesi.

4.5 Quando conviene il fine-tuning

La strategia da scegliere dipende da due fattori: la dimensione del nuovo dataset e quanto è simile al dataset originale su cui la rete è stata addestrata. Abbiamo quindi quattro casi (figura 4.4):

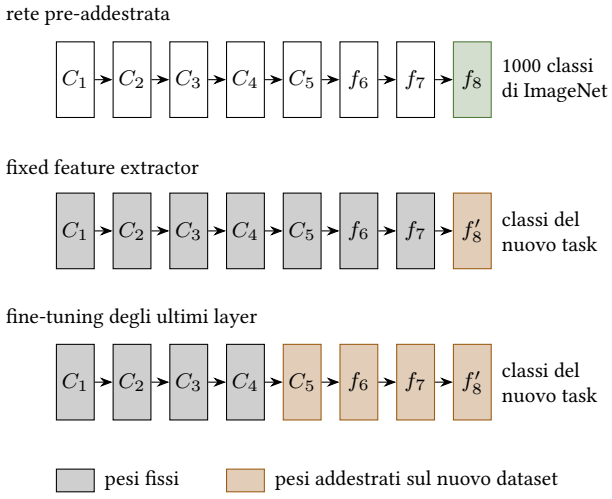


Figura 4.3. Strategie di transfer learning a partire da una rete pre-addestrata su ImageNet. Con il fixed feature extractor si addestra solo il nuovo classificatore f'_8. Con il fine-tuning si addestrano anche gli ultimi layer della rete, mentre i primi restano fissi.

1. **Dataset piccolo e simile all'originale**: fare fine-tuning di tutta la rete può portare all'overfitting. Conviene addestrare solo il layer appena aggiunto.
2. **Dataset grande e simile all'originale**: più dati abbiamo, meglio è. Possiamo addestrare tutta la rete.
3. **Dataset piccolo e diverso dall'originale**: addestrare tutta la rete non è una buona idea. Tuttavia il nuovo layer non va aggiunto in cima, perché gli ultimi layer sono molto specifici per il dataset (e il problema) originale. Conviene aggiungerlo a una parte precedente della rete.
4. **Dataset grande e diverso dall'originale**: possiamo fare fine-tuning di tutta la rete. Equivale a un nuovo training, in cui però i pesi sono inizializzati con quelli di un'altra rete.

In tutti i casi, partire da pesi pre-addestrati conviene: hanno già un senso, mentre dei pesi inizializzati in modo casuale no. Il vantaggio è massimo quando il dataset è piccolo ma simile all'originale.

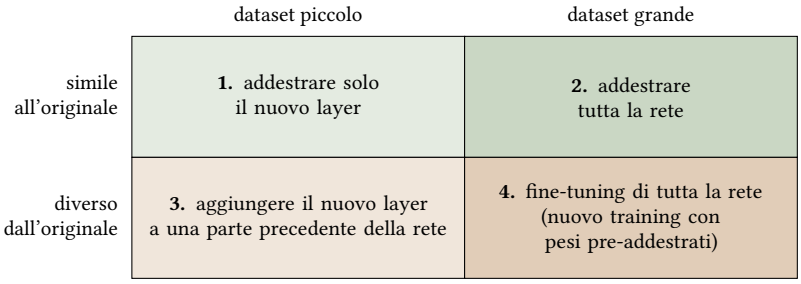


Figura 4.4. La strategia di fine-tuning in base alla dimensione del nuovo dataset (colonne) e alla sua somiglianza con il dataset originale (righe).

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (27 ottobre 2026).
- Slide del corso *Convolutional Neural Net*, dal dropout al fine-tuning.
- «Dropout: A Simple Way to Prevent Neural Networks from Overfitting» [16], il paper originale sul dropout, da cui è tratta la figura 4.1.
- «Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift» [8], il paper originale sulla batch normalization.

Capitolo 5

Metric Learning

Molti algoritmi di machine learning si basano su una misura di distanza tra i dati: due esempi vicini nello spazio vengono considerati simili. In questo capitolo partiamo da due algoritmi di questo tipo, il KNN e il K-means, e dalle misure di distanza più usate. Poi vediamo il **metric learning**, ovvero come imparare dai dati una rappresentazione in cui una distanza fissata a priori funziona bene. Lo faremo prima con una trasformazione lineare e poi con una rete neurale (*deep metric learning*).

5.1 Algoritmi basati sulle distanze

Diversi approcci di machine learning richiedono una misura di distanza tra punti in uno spazio $\mathbb{R}^d$, i cui punti rappresentano i dati del problema. Nel capitolo 4 abbiamo visto che una rete pre-addestrata, senza l'ultimo layer, è un feature extractor. Possiamo quindi calcolare le distanze non sui dati grezzi, ma sulle loro rappresentazioni $\phi(x)$, dove ϕ è una **representation function** (ad esempio appresa da una rete).

5.1.1 K-Nearest Neighbor

Il classificatore **nearest neighbor** si basa su un'idea semplice: dato un nuovo esempio x', cerchiamo l'esempio del training set più vicino e gli assegniamo la stessa classe. L'ipotesi è che elementi vicini nello spazio della rappresentazione siano probabilmente simili. Pensiamo a MNIST: se l'immagine di una cifra è molto vicina a un'immagine del training set etichettata come 7, è probabile che sia anch'essa un 7.

Per misurare quanto due esempi sono simili ci serve una funzione di distanza $D(x, x')$, che deve essere piccola per esempi simili. La scelta più comune è la distanza euclidea:

$$D(x, x') = \|x - x'\|_2 = \sqrt{\sum_{i=1}^{d} (x_i - x'_i)^2}.$$

Il classificatore assegna a x' la classe dell'esempio del training set più vicino:

$$h(x') = y^{(j)}, \qquad j = \arg\min_i D\big(x', x^{(i)}\big)$$

dove $x^{(j)}$ è il **nearest neighbor** di x' e $y^{(j)}$ la sua etichetta. Questo algoritmo si chiama nearest neighbor, o **1-NN**. L'algoritmo 2 lo riporta in una versione in cui la distanza è calcolata sulle rappresentazioni ϕ.

Algorithm 2 Classificazione 1-NN

Require: matrice X ($m \times d$) dei campioni $\{x^{(i)}\}_{i=1}^{m}$, representation function ϕ, vettore y ($m \times 1$) delle etichette, con $y^{(i)} \in C$ (l'insieme delle classi), nuovo pattern P da classificare
Ensure: classe del pattern P
1: **for** $i = 1, \ldots, m$ **do**
2: $\quad \text{distanze}(i) \leftarrow D\big(\phi(x^{(i)}), \phi(P)\big)$
3: **end for**
4: $\text{min_index} \leftarrow$ indice della distanza minima in distanze
5: **return** $y^{(\text{min_index})}$

Da 1-NN a K-NN. Il nearest neighbor funziona bene se gli esempi della stessa classe sono vicini tra loro nello spazio della rappresentazione. Spesso, però, il training set contiene degli **outlier**, ovvero esempi che non seguono la distribuzione degli altri. Può succedere perché i dati non sono puliti (un esempio etichettato male) oppure perché la rappresentazione non è ideale. Ad esempio, in MNIST potremmo avere un 1 scritto in modo da sembrare un 7. Se il nearest neighbor di un nuovo 7 è proprio quell'1, il 1-NN sbaglia, anche se tutti gli altri esempi vicini sono dei 7.

Conviene quindi guardare un intorno del punto, invece del solo punto più vicino. Fissare un raggio per l'intorno non è semplice: se lo spazio non è uniformemente denso, intorni con lo stesso raggio possono contenere numeri di elementi molto diversi, a volte anche nessuno. Si considerano quindi i K esempi più vicini, dove K è un iperparametro. Il classificatore **K-Nearest Neighbor** (K-NN) assegna la classe più frequente tra questi:

$$h(x') = \text{mode}\big\{y^{(i)} : x^{(i)} \in N_K(x')\big\}$$

dove $N_K(x')$ è l'insieme dei K esempi del training set più vicini a x', e mode è la moda statistica, ovvero restituisce l'elemento più frequente di un insieme. Il 1-NN visto prima è il caso $K = 1$. In figura 5.1, ad esempio, con $K = 3$ il pattern da classificare viene assegnato alla classe 1.

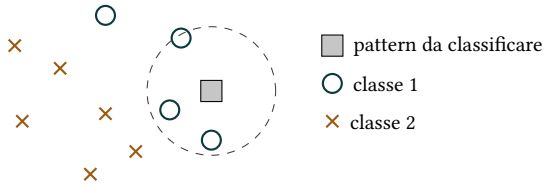


Figura 5.1. Classificazione K-NN con $K = 3$. I tre vicini del pattern da classificare (dentro il cerchio tratteggiato) appartengono tutti alla classe 1, che è quindi la classe assegnata.

Il valore di K si può fissare in modo arbitrario oppure ottimizzare con un validation set o con la cross-validation. K controlla come viene partizionato lo spazio: con K piccolo il decision boundary segue ogni singolo punto (outlier compresi), mentre con K grande le regioni diventano più regolari. È il solito trade-off tra bias e varianza.

Il KNN è un algoritmo supervisionato, perché usa le etichette del training set. Non ha però una vera fase di training: è un metodo **con memoria**, che salva tutto il training set e calcola le distanze solo quando deve classificare un nuovo esempio.

Curse of dimensionality. Il KNN funziona molto bene su dati con poche feature, ma soffre della **curse of dimensionality** (maledizione della dimensionalità). Al crescere del numero di dimensioni i dati diventano sparsi, e la distanza dal punto più vicino e quella dal punto più lontano diventano quasi uguali. Se tutti i punti sono "ugualmente lontani", il concetto di vicino perde di significato. Per questo, su dati ad alta dimensionalità come le immagini, conviene calcolare le distanze su una rappresentazione $\phi(x)$ compatta e non sui pixel.

5.1.2 K-means

Il **K-means** sfrutta la stessa idea per il **clustering**. In questo caso non abbiamo etichette (è un algoritmo non supervisionato), e vogliamo dividere automaticamente i dati in K gruppi (o cluster) di elementi vicini tra loro. L'output è quindi una partizione dello spazio di input. L'algoritmo 3 ne riporta lo pseudocodice.

Algorithm 3 Clustering K-means

Require: matrice X ($m \times d$) dei campioni, representation function ϕ, numero di cluster K
Ensure: K cluster (una partizione dello spazio di input)

scegli in modo casuale K campioni di X e usa le loro rappresentazioni come centroidi $\{c_1, \ldots, c_K\}$
repeat
    for ogni campione $x^{(i)}$ **do**
        trova il centroide c_j più vicino, con la distanza D tra c_j e $\phi(x^{(i)})$
        assegna $x^{(i)}$ al cluster j
    end for
    for ogni cluster j **do**
        ricalcola c_j come media dei $\phi(x^{(i)})$ dei punti assegnati al cluster j
    end for
until convergenza, oppure per un numero fissato di iterazioni

Ogni iterazione ha due passi. Nel primo, ogni punto viene assegnato al cluster del centroide più vicino, secondo la distanza D. Nel secondo, ogni centroide viene ricalcolato come la media delle rappresentazioni dei punti che gli sono stati assegnati. Tornando a MNIST, con $K = 10$ vorremmo che ogni cluster raccogliesse le immagini di una sola cifra. Anche qui, quanto bene funziona dipende dalla distanza e dalla rappresentazione ϕ che usiamo.

5.1.3 Misure di distanza

In pratica si usano metriche standard quando si hanno conoscenze a priori sul dominio. Per gli spazi euclidei le più usate sono la distanza euclidea e la similarità del coseno:

$$E(x^{(1)}, x^{(2)}) = \sqrt{\sum_i \left(x_i^{(1)} - x_i^{(2)}\right)^2}, \qquad C(x^{(1)}, x^{(2)}) = \frac{\sum_i x_i^{(1)} x_i^{(2)}}{\sqrt{\sum_i \left(x_i^{(1)}\right)^2} \sqrt{\sum_i \left(x_i^{(2)}\right)^2}}.$$

Per le distribuzioni di probabilità si usano invece la KL divergence (Kullback-Leibler)[1] e il coefficiente di Bhattacharyya:

$$D_{KL}(x^{(1)}, x^{(2)}) = \sum_i x_i^{(1)} \log \frac{x_i^{(1)}}{x_i^{(2)}}, \qquad B(x^{(1)}, x^{(2)}) = \sum_i \sqrt{x_i^{(1)} x_i^{(2)}}.$$

Attenzione all'interpretazione dei valori (tabella 5.1). La distanza euclidea e la KL divergence valgono 0 per esempi simili, e crescono per esempi dissimili. Il coseno e il coefficiente di Bhattacharyya, invece, sono misure di similarità: valgono 1 per esempi simili e diminuiscono per esempi dissimili.

Misura	Dominio	Intervallo	Simili	Dissimili
Euclidea E	spazi euclidei	$[0, +\infty)$	0	$\to +\infty$
Coseno C	spazi euclidei	$[-1, 1]$	1	-1
KL divergence D_{KL}	distribuzioni	$[0, +\infty)$	0	$\to +\infty$
Bhattacharyya B	distribuzioni	$[0, 1]$	1	0

Tabella 5.1. Le misure di distanza (e di similarità) più usate, con l'intervallo dei valori e i valori tipici per esempi simili e dissimili.

Tuttavia è spesso difficile stabilire quale sia la metrica più adatta per uno specifico problema o dataset. Qui entra in gioco il metric learning.

5.2 Metric learning

Definizione 5.1 (Metric learning). Il **metric learning** ha l'obiettivo di costruire algoritmi che creano automaticamente misure di distanza specifiche per un task (*task-specific*), a partire da dati supervisionati o weakly supervised.

La misura di similarità appresa potrà poi essere usata per vari problemi, ad esempio per la classificazione con il KNN o per il clustering con il K-means. In realtà, la procedura di apprendimento agisce sullo spazio di input: lo trasforma in modo che possa essere "usato" con una metrica definita a priori. In altre parole, l'algoritmo riceve i dati e una metrica, e restituisce una rappresentazione dei dati in cui quella metrica produce i risultati attesi (figura 5.2).

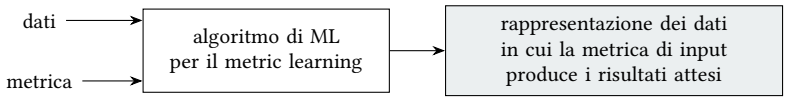


Figura 5.2. Lo schema del metric learning. L'algoritmo riceve i dati e una metrica definita a priori, e trasforma i dati in modo che la metrica funzioni.

[1] La KL divergence non è simmetrica, ovvero in generale $D_{KL}(x^{(1)}, x^{(2)}) \neq D_{KL}(x^{(2)}, x^{(1)})$. A rigore, quindi, non è una distanza.

5.2.1 Supervised e weakly supervised

Ci sono due categorie principali di metric learning:

- **Supervised**: l'algoritmo ha accesso a dati etichettati, in cui a ogni dato è associata un'etichetta di classe. L'obiettivo è trasformare lo spazio dei dati di input in modo che i punti con la stessa etichetta risultino vicini nel nuovo spazio, e i punti con etichette diverse risultino lontani.
- **Weakly supervised**: l'algoritmo ha accesso a dati in cui la supervisione è solo a livello di tuple (coppie, triplette, eccetera), ad esempio coppie simili e coppie dissimili. L'obiettivo è trasformare lo spazio in modo che le tuple simili siano vicine nello spazio di output, e quelle dissimili lontane.

Un esempio di weakly supervised sono le facce sorridenti. Non abbiamo un'etichetta di classe per ogni faccia, ma sappiamo che due facce sorridenti formano una coppia simile, mentre una faccia sorridente e una non sorridente formano una coppia dissimile.

Il problema del metric learning si formula quindi come un problema di ottimizzazione: bisogna trovare i parametri di una trasformazione dello spazio di input, rispetto a una misura di distanza. Si minimizza una funzione obiettivo sui dati di input.

5.2.2 Mahalanobis distance learning

Il caso più semplice è una trasformazione lineare. Nel **Mahalanobis distance learning** (implementato ad esempio in scikit-learn) la distanza tra due esempi è:

$$D(x^{(1)}, x^{(2)}) = \sqrt{\left(Lx^{(1)} - Lx^{(2)}\right)^T \left(Lx^{(1)} - Lx^{(2)}\right)}$$

dove $x \in \mathbb{R}^d$ è il vettore delle feature e $L \in \mathbb{R}^{m \times d}$ è la matrice dei parametri. Si cerca una trasformazione lineare L dello spazio delle feature che soddisfi l'obiettivo del metric learning (supervised o weakly supervised). Ad esempio, vogliamo che i pattern simili abbiano distanza 0 e quelli dissimili distanza 1. Il nome viene dal fatto che, ponendo $M = L^T L$, la distanza si può riscrivere come $\sqrt{(x^{(1)} - x^{(2)})^T M\,(x^{(1)} - x^{(2)})}$, che è proprio la forma della distanza di Mahalanobis.

L trasforma un vettore di d componenti in uno di m componenti (figura 5.3). Se $m < d$, quindi, otteniamo anche una riduzione di dimensionalità. Se invece L è la matrice identità, D non è altro che la distanza euclidea.

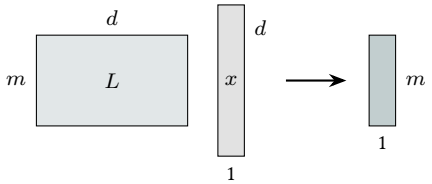


Figura 5.3. La trasformazione lineare del Mahalanobis distance learning: la matrice L ($m \times d$) trasforma il vettore delle feature x ($d \times 1$) in un vettore Lx ($m \times 1$). Con $m < d$ si ha una riduzione di dimensionalità.

Restano due domande. Come possiamo trasformare lo spazio di input, ovvero come troviamo L? E se volessimo trasformazioni non lineari? La risposta è la stessa: con un'architettura neurale.

5.3 Deep metric learning

5.3.1 L'idea di partenza

Nel **deep metric learning** la trasformazione è una rete neurale L_ϑ (figura 5.4). Due esempi $x^{(1)}$ e $x^{(2)}$ passano attraverso la stessa rete, che li trasforma in $\tilde{x}^{(1)}$ e $\tilde{x}^{(2)}$:

$$\tilde{x} \equiv L_\vartheta(x).$$

Sulle rappresentazioni trasformate calcoliamo la distanza D, e con questa una funzione di loss. Le coppie $(x^{(1)}, x^{(2)})$ sono etichettate come simili (1) oppure dissimili (0). Nel caso supervised, ad esempio, sono simili due campioni della stessa classe e dissimili due campioni di classi diverse. La distanza D stabilisce l'errore utile per aggiornare i parametri dell'architettura L_ϑ.

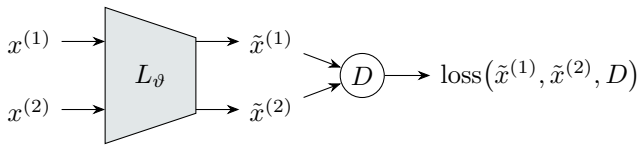


Figura 5.4. L'idea alla base del deep metric learning. La rete L_ϑ trasforma i due esempi, e la loss confronta le rappresentazioni trasformate tramite la distanza D.

Ci servirà capire tre cose:

1. chi è L_ϑ: una rete neurale.
2. chi è D, ovvero come scrivere una funzione di loss che tenga conto di D, al fine di ottenere una buona L_ϑ.
3. come campionare le tuple per ottimizzare i parametri, sia nel caso supervised che in quello weakly supervised.

Il nuovo spazio $\tilde{X}$, ottenuto dopo la trasformazione, viene generalmente chiamato **embedding space**.[2] In fase di test usiamo L_ϑ per trasformare il nuovo pattern, e D per cercare il pattern più vicino nello spazio trasformato del training set.

5.3.2 Approcci contrastive

Gli approcci **contrastive** cercano una loss function che spinga direttamente gli elementi simili a essere vicini nello spazio, e quelli dissimili a essere lontani. La misura di distanza generalmente

[2] Il termine *embedding* si usa anche per i transformer e gli LLM, ma con un significato diverso. Lì l'embedding è la tabella appresa che associa a ogni token un vettore, e le rappresentazioni dei layer successivi dipendono dal contesto. Questi vettori sono addestrati per un altro task (predire il token successivo), quindi la loro distanza non è per forza una misura di similarità. Nel metric learning, invece, l'embedding space viene appreso proprio perché la distanza D abbia un significato. Non a caso i modelli che confrontano frasi, come Sentence-BERT [13], sono addestrati con reti siamesi e con la triplet loss.

usata per costruire l'embedding space negli approcci contrastive è la distanza euclidea ℓ_2:

$$D(p^{(1)}, p^{(2)}) = \|p^{(1)} - p^{(2)}\|_2 = \sqrt{\sum_i \left(p_i^{(1)} - p_i^{(2)}\right)^2}.$$

L'obiettivo è ottimizzare, con una loss function opportuna, un'architettura L_ϑ tale che la distanza dopo la trasformazione

$$D\big(L_\vartheta(p^{(1)}), L_\vartheta(p^{(2)})\big)$$

rispetti i requisiti: simili vicini e dissimili lontani.

In genere L_ϑ è un'architettura nota, che ottiene buoni risultati in problemi di classificazione (ad esempio VGG o ResNet) o di regressione (ad esempio PoseNet). Si può anche partire da un modello pre-addestrato, come abbiamo visto con il transfer learning.

5.3.3 Contrastive loss

La **contrastive loss** è una loss function classica, proposta da Chopra et al. nel 2005 [2]. Siano $x^{(1)}$ e $x^{(2)}$ due campioni del dataset, con etichette $y^{(1)}$ e $y^{(2)}$. Sia inoltre $\mathbf{1}_A$ la funzione indicatrice, che restituisce 1 se il predicato A è vero e 0 se è falso.

Definizione 5.2 (Contrastive loss). La contrastive loss, da minimizzare, è definita come:

$$\begin{aligned}\text{loss}_{\text{contrastive}}(x^{(1)}, x^{(2)}, D) = \frac{1}{2}\Big[&\mathbf{1}_{y^{(1)}=y^{(2)}}\, D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(2)})\big)^2 \\ &+ \mathbf{1}_{y^{(1)}\neq y^{(2)}} \max\big(0,\ \alpha - D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(2)})\big)\big)^2\Big]\end{aligned}$$

dove $\alpha > 0$ è detto **margine**.

Per brevità scriviamo D al posto di $D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(2)})\big)$. Dei due termini ne resta sempre solo uno:

- **Caso 1**, campioni simili: si considera $\frac{1}{2}D^2$. Quindi minimizziamo la distanza euclidea tra i simili.
- **Caso 2**, campioni dissimili: si considera $\frac{1}{2}\max(0, \alpha - D)^2$.

Vediamolo graficamente (figura 5.5, a sinistra). Supponiamo che il cerchio e le stelle siano simili, mentre il cerchio e i quadrati siano dissimili. Vogliamo massimizzare le distanze **inter-class** (frecce tratteggiate) e minimizzare le distanze **intra-class** (frecce continue). Per ogni punto, quindi, vogliamo che la massima distanza intra-class sia più piccola della minima distanza inter-class.

Il margine α traduce questa richiesta (figura 5.5, a destra). Dopo la trasformazione dello spazio, rispetto al punto preso in considerazione (il cerchio), tutte le stelle devono avere una distanza minore di α, mentre tutti i quadrati devono avere una distanza maggiore. Se riusciamo a ottenere questo, la ricerca del vicino diventa semplice: se un nuovo punto da classificare sta a una distanza minore di α da un altro, allora i due sono simili, altrimenti sono dissimili.

Questo concetto è espresso proprio dal termine per i dissimili:

$$\frac{1}{2}\max\big(0,\ \alpha - D\big)^2.$$

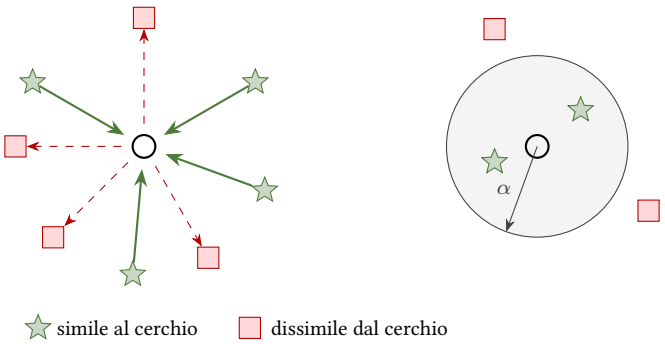


Figura 5.5. A sinistra, la contrastive loss avvicina i campioni simili (frecce continue) e allontana quelli dissimili (frecce tratteggiate). A destra, dopo il training: i simili stanno entro il margine α dal cerchio, i dissimili fuori.

La formula penalizza i punti dissimili la cui distanza D è minore di α: in questo caso $\alpha - D$ è positivo, e abbiamo un "errore" $(\alpha - D)^2$ da usare per aggiornare i pesi. Se invece la distanza è maggiore o uguale ad α, $\alpha - D$ è negativo o nullo, e il massimo restituisce 0. Il gradiente è quindi nullo e i pesi non vengono aggiornati: i punti dissimili sono già a una distanza adeguata, e non li spingiamo ancora più lontano (figura 5.6).

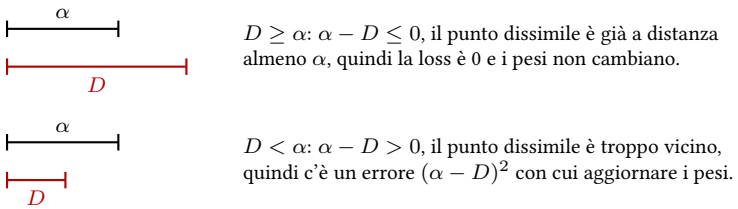


Figura 5.6. I due casi del termine $\frac{1}{2}\max(0, \alpha - D)^2$ per una coppia di punti dissimili.

Uno dei motivi per cui usiamo il margine è evitare che l'ottimizzazione prenda delle "scorciatoie". Senza un vincolo sui dissimili, la rete potrebbe mappare tutti i punti su un unico punto, con distanza zero tra ogni coppia dopo il training. Il termine con il margine lo impedisce, perché obbliga le coppie dissimili a restare almeno a distanza α.

5.3.4 Il caso weakly supervised

Nella definizione della loss abbiamo supposto il caso supervised, in cui ogni pattern x è associato a un'etichetta y. Nel caso weakly supervised non abbiamo etichette di classe, ma possiamo associare delle etichette alle coppie di punti (simili o dissimili).

Supponiamo, ad esempio, di voler costruire un algoritmo che, data un'immagine in input, cerchi in un dataset l'immagine più simile. Le immagini del dataset sono associate a delle coordinate, ovvero alla posizione e all'orientamento rispetto alla mappa di un ambiente (figura 5.7, a sinistra). A una nuova immagine, acquisita nello stesso ambiente, vorremmo associare una posizione (x, y) e un orientamento (supponiamo che tutte le immagini siano acquisite alla stessa altezza z). Un caso concreto è la localizzazione di un carrello della spesa all'interno di un supermercato.

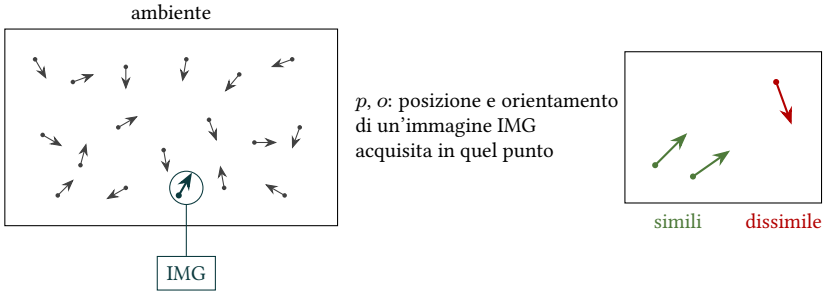
Per farlo possiamo usare un KNN con $K = 1$: dopo aver addestrato la rete con una contrastive loss, cerchiamo l'immagine più simile usando la sua rappresentazione e la distanza euclidea ℓ_2.

> **Osservazione.** Le immagini di training hanno delle etichette relative alla posizione e all'orientamento, ma non sono etichette di classe. Non possiamo quindi valutare le condizioni della loss ($\mathbf{1}_{y^{(1)}=y^{(2)}}$ oppure $\mathbf{1}_{y^{(1)}\neq y^{(2)}}$): poiché campioniamo le immagini nell'ambiente, verosimilmente avremo $y^{(1)} \neq y^{(2)}$ per quasi tutte le coppie.

Possiamo comunque sfruttare l'algoritmo con un **relabeling**: associamo a ogni coppia di immagini una similarità o una dissimilarità, cambiando la funzione indicatrice. Ad esempio:

$$\mathbf{1}(y^{(1)}, y^{(2)}) = \begin{cases} 1 & \text{se } \|p^{(1)} - p^{(2)}\| < t \text{ e } |o^{(1)} - o^{(2)}| < g \\ 0 & \text{altrimenti} \end{cases}$$

dove p è la posizione dell'immagine, o il suo orientamento, mentre t e g sono parametri scelti dall'utente. Consideriamo quindi simile una coppia di immagini vicine nello spazio e con un orientamento simile (figura 5.7, a destra).



> **Figura 5.7.** A sinistra, le immagini di training sono acquisite in diversi punti dell'ambiente, ognuna con una posizione e un orientamento (le frecce). A destra, il relabeling: due immagini vicine e con orientamento simile formano una coppia simile, mentre un'immagine lontana o con orientamento diverso è dissimile.

Nella loss usiamo la nuova definizione $\mathbf{1}(y^{(1)}, y^{(2)})$ per la parte che riguarda le coppie simili, e $1 - \mathbf{1}(y^{(1)}, y^{(2)})$ per la parte che riguarda le coppie dissimili. La loss function diventa quindi:

$$\begin{aligned} \text{loss}_{\text{contrastive}}(x^{(1)}, x^{(2)}, D) = \frac{1}{2}\Big[& \mathbf{1}(y^{(1)}, y^{(2)})\, D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(2)})\big)^2 \\ & + \big(1 - \mathbf{1}(y^{(1)}, y^{(2)})\big) \max\big(0,\ \alpha - D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(2)})\big)\big)^2 \Big]. \end{aligned}$$

5.3.5 Siamese network

Un altro esempio weakly supervised sono le facce sorridenti, in cui la similarità si basa sul contenuto delle immagini. L'architettura usata in questi casi è la **siamese network** (figura 5.8). In realtà le architetture sono due, una per ogni elemento della coppia, ma condividono i pesi (*shared weights*): sono quindi la stessa rete L_ϑ applicata due volte. Il backward pass attraversa i due rami separatamente, ma poiché i pesi sono condivisi, i due contributi del gradiente aggiornano gli stessi parametri.

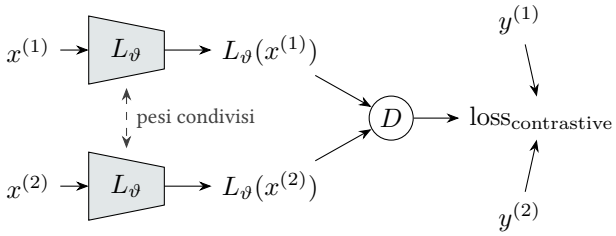


Figura 5.8. Una siamese network. I due elementi della coppia passano attraverso due copie della stessa rete L_ϑ, con pesi condivisi. La loss usa la distanza D tra le due rappresentazioni e le etichette $y^{(1)}$ e $y^{(2)}$ (o la loro similarità).

Il training procede così:

0. si fa il relabeling delle coppie di facce (simili o dissimili).
1. si campionano delle coppie. Bisogna fare in modo che il campionamento consideri sia casi simili che casi dissimili, e anche casi "difficili".
2. ogni pattern della coppia viene rappresentato tramite L_ϑ, dove L_ϑ è un'architettura neurale (ad esempio VGG o ResNet).
3. si valuta la funzione di loss usando la distanza D.
4. si aggiornano i pesi di L_ϑ con la backpropagation.

5.3.6 Triplet loss

Nella contrastive loss ogni coppia è simile oppure dissimile. Perché non guardare contemporaneamente un caso simile e uno dissimile? È l'idea della **triplet loss** (Schroff et al., 2015 [15]), una delle loss di metric learning più usate.

Siano $x^{(1)}$, $x^{(2)}$ e $x^{(3)}$ tre campioni del dataset tali che $y^{(1)} = y^{(2)}$ e $y^{(1)} \neq y^{(3)}$. In letteratura $x^{(1)}$ è detto **ancora** (*anchor*), mentre $x^{(2)}$ è chiamato **campione positivo**, perché ha la stessa etichetta dell'ancora (è simile). $x^{(3)}$, infine, è detto **campione negativo**, perché ha un'etichetta diversa dall'ancora (è dissimile). Si considerano quindi triplette (ancora, positivo, negativo) per addestrare la rete L_ϑ che trasforma lo spazio delle feature.

Definizione 5.3 (Triplet loss). La triplet loss è definita come:

$$\text{loss}_{\text{triplet}}(x^{(1)}, x^{(2)}, x^{(3)}, D) = \max\big(0,\ D_P - D_N + \alpha\big)$$

dove α è il margine, mentre

$$D_P = D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(2)})\big)^2, \qquad D_N = D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(3)})\big)^2$$

sono le distanze (al quadrato) dell'ancora dal positivo e dal negativo.

Anche qui ci sono due casi:

- **Caso 1**: $D_P - D_N + \alpha \leq 0$, ovvero $D_P + \alpha \leq D_N$. La distanza tra i dissimili è già maggiore della distanza tra i simili più il margine. Il massimo vale 0, e non è necessario aggiornare i pesi.
- **Caso 2**: $D_P - D_N + \alpha > 0$, ovvero $D_P + \alpha > D_N$. Il negativo è troppo vicino all'ancora, e vogliamo che venga allontanato con un errore pari a $D_P - D_N + \alpha$ (figura 5.9).

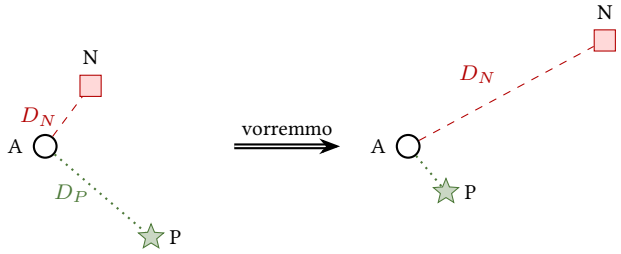


Figura 5.9. Il caso 2 della triplet loss. A sinistra il negativo N è più vicino all'ancora A del positivo P. Vorremmo invece che il positivo fosse vicino all'ancora e il negativo lontano, almeno di un margine α in più.

Nei modelli basati sulla triplet loss conta molto il **campionamento dei negativi**. A ogni training step vogliamo campionare triplette $(x^{(1)}, x^{(2)}, x^{(3)})$ tali che $D_N < D_P + \alpha$, ovvero triplette che la nostra rete L_ϑ non mappa ancora nel modo voluto. Le altre hanno loss nulla, e non contribuiscono all'apprendimento.

La differenza principale tra la contrastive loss classica e la triplet loss sta nel fatto che, con quest'ultima, si ottimizza considerando contemporaneamente campioni positivi e negativi.

5.3.7 Quadruplet loss

Negli anni sono state proposte diverse altre varianti degli approcci contrastive. Tra queste c'è la **quadruplet loss** (Chen et al., 2017 [1]), che cerca di massimizzare la variazione inter-class delle feature e di minimizzare quella intra-class. La triplet loss vista prima, infatti, non tiene conto della variazione delle feature all'interno della classe.

Data una quadrupla di campioni $x^{(1)}$, $x^{(2)}$, $x^{(3)}$, $x^{(4)}$ tali che $y^{(1)} = y^{(2)} = y^{(4)}$ e $y^{(1)} \neq y^{(3)}$, la loss è definita come segue[3]:

$$\begin{aligned}\text{loss}_{\text{quadruplet}} = &\max\left(0,\ D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(2)})\big)^2 - D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(4)})\big)^2 + \alpha_1\right) \\ &+ \max\left(0,\ D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(4)})\big)^2 - D\big(L_\vartheta(x^{(1)}), L_\vartheta(x^{(3)})\big)^2 + \alpha_2\right)\end{aligned}$$

dove α_1 e α_2 sono due margini. Il primo termine chiede che $x^{(4)}$ sia più lontano dall'ancora di $x^{(2)}$, almeno di α_1. Il secondo chiede che il negativo $x^{(3)}$ sia più lontano di $x^{(4)}$, almeno di α_2 (figura 5.10). In questo modo la minima distanza inter-class deve essere maggiore della massima distanza intra-class.

[3] Questa formulazione è diversa da quella del paper originale [1], in cui la quadrupla è formata da un'ancora x_i, un positivo x_j e due negativi x_k e x_l, di classi diverse tra loro e diverse da quella dell'ancora. Il primo termine è quello della triplet loss, $\max(0, D(x_i, x_j)^2 - D(x_i, x_k)^2 + \alpha_1)$. Il secondo, $\max(0, D(x_i, x_j)^2 - D(x_l, x_k)^2 + \alpha_2)$, confronta la distanza tra ancora e positivo con la distanza tra i due negativi, che non coinvolge l'ancora (per brevità, qui $D(a, b)$ indica $D(L_\vartheta(a), L_\vartheta(b))$).

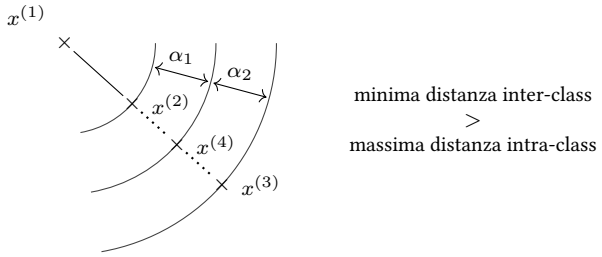


Figura 5.10. La quadruplet loss: rispetto all'ancora $x^{(1)}$, il campione $x^{(4)}$ deve stare oltre $x^{(2)}$ di almeno α_1, e il negativo $x^{(3)}$ oltre $x^{(4)}$ di almeno α_2.

5.3.8 Limiti degli approcci contrastive

Sebbene siano molto semplici e in diversi casi efficaci, questi metodi presentano due limiti:

1. è molto difficile assicurare che campioni con etichette simili vengano trasformati e messi in una regione comune del nuovo spazio. La quadruplet loss prova a migliorare la variabilità intra-class e inter-class, ma in ogni caso cercare di migliorare i risultati con un vincolo globale di questo tipo non porta a grandi miglioramenti.

2. tutti i metodi di deep metric learning che provano a minimizzare o massimizzare direttamente le distanze ℓ_2 tra campioni dipendono pesantemente da sofisticati metodi di **mining** delle tuple. A ogni training step vanno campionate le tuple (o i batch di tuple) "utili" per l'apprendimento. Questo limita lo sfruttamento pieno della GPU, e può diventare problematico in sistemi con GPU distribuite.

5.4 Center loss

Abbiamo capito che il deep metric learning si "riduce" alla scelta di un'architettura L_ϑ e di una loss function, con una distanza D definita a priori. Possiamo allora usare una loss classica dei problemi di classificazione per un problema di metric learning? Ad esempio, possiamo usare la **softmax loss** per apprendere un buon embedding space, su cui poi usare una metrica come quella euclidea? La risposta è sì, ma i risultati saranno meno soddisfacenti di quelli che si ottengono con i metodi contrastive visti prima o con altri metodi di metric learning.

Ricordiamo la softmax loss (sezione 1.3), scritta con la notazione del paper sulla center loss [18]:

$$\text{loss}_{\text{softmax}} = -\sum_{i=1}^{N} \log \frac{e^{W_{y_i}^T z_i + b_{y_i}}}{\sum_{j=1}^{K} e^{W_j^T z_i + b_j}}$$

dove N è il numero di campioni (ad esempio di un mini-batch), K il numero di classi e $z_i = L_\vartheta(x_i)$ la feature del campione i. W_j e b_j sono invece i pesi e il bias del layer softmax per la classe j.

Quando addestriamo un classificatore formato da un'architettura L_ϑ a cui abbiamo "attaccato" alla fine un layer softmax, questa loss ottimizza anche i parametri di L_ϑ. La parte che si occupa della rappresentazione dei dati impara quindi a mappare l'input in un nuovo spazio, "meglio organizzato" e più discriminativo. L'obiettivo è avere pattern con la stessa etichetta vicini nel nuovo spazio, e ben separati dai pattern con etichetta diversa.

La softmax loss funziona in molti casi in un framework di classificazione. Tuttavia, se osserviamo lo spazio delle feature prodotto dalla backbone L_ϑ collegata al layer softmax, ci accorgiamo che non è così discriminativo come ci si aspetta (figura 5.11, a sinistra). Le classi sono separabili, ma la trasformazione ha ancora troppa variazione intra-class.

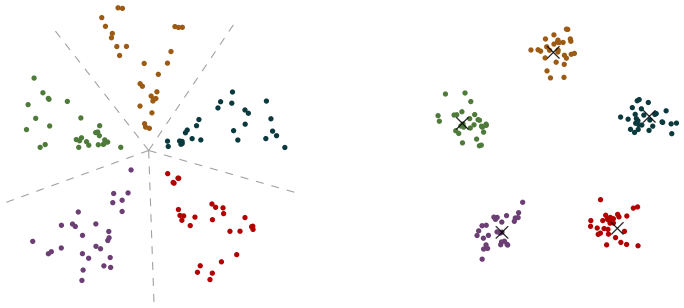


Figura 5.11. Schema delle feature 2D di cinque classi (si veda [18]). A sinistra, con la sola softmax loss le classi sono separabili (le linee tratteggiate), ma le feature di una classe sono disperse lungo un raggio. A destra, la center loss raccoglie le feature attorno al centro della propria classe ($\times$).

Possiamo però modificare la softmax loss per renderla più adatta al metric learning. L'idea è aggiungere un termine che "spinga" le feature dei vari pattern a raggrupparsi attorno a dei **prototipi**, uno per classe. La **center loss** [18] cattura proprio questa idea.

Definizione 5.4 (Center loss). La center loss è definita come:

$$\text{loss} = \text{loss}_{\text{softmax}} + \lambda\, \text{loss}_{\text{center}}, \qquad \text{loss}_{\text{center}} = \frac{1}{2} \sum_{i=1}^{N} \left\| z_i - c_{y_i} \right\|_2^2$$

dove $z_i \equiv L_\vartheta(x_i)$, c_j è il prototipo (o centro) della classe j nello spazio delle feature, e λ bilancia i due termini.

Anche i centri c_j vanno aggiornati durante il training. Si possono ricalcolare dopo ogni aggiornamento dei parametri, come le nuove medie delle feature, oppure aggiornare con la discesa del gradiente usando il gradiente della $\text{loss}_{\text{center}}$. Ad esempio, a ogni iterazione:

$$c_j \leftarrow c_j - \alpha\, \Delta c_j, \qquad \Delta c_j = \frac{\sum_{i=1}^{N} \delta(y_i = j)\,(c_j - z_i)}{1 + \sum_{i=1}^{N} \delta(y_i = j)}$$

dove $\delta(y_i = j)$ vale 1 se il campione i appartiene alla classe j e 0 altrimenti. α è invece il learning rate dei centri, da non confondere con il margine delle loss contrastive[4].

Il numeratore somma le differenze tra il centro e le feature dei campioni della classe j presenti nel mini-batch, e il denominatore le divide per il numero di questi campioni. L'1 al denominatore evita la divisione per zero quando la classe non compare nel mini-batch. In pratica, quindi, Δc_j è la differenza media tra il centro e le feature della sua classe.

[4] La stessa formula si trova spesso con la somma al numeratore sull'indice j e le feature indicate con x_i. Qui la somma è sui campioni i e le feature sono le z_i, in modo coerente con la formula della loss.

La center loss rende la softmax loss più adatta al metric learning (figura 5.11, a destra), e risolve entrambi i limiti degli approcci contrastive:

1. forza i campioni a raggrupparsi attorno al prototipo della classe a cui appartengono (il primo limite).
2. non richiede alcun campionamento di tuple per l'apprendimento (il secondo limite).

Anche la center loss, però, ha dei limiti. Negli ultimi anni sono apparse diverse altre loss di metric learning che migliorano lo stato dell'arte, tra cui:

- SphereFace (Liu et al., 2017 [10]).
- CosFace (Wang et al., 2018 [17]).
- ArcFace (Deng et al., 2019 [3]).
- Sub-center ArcFace (Deng et al., 2020 [4]).

Non le tratteremo in dettaglio, ma si possono approfondire nei paper citati.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (27 ottobre 2026).
- Appunti del corso *Deep Metric Learning*, da cui sono tratti gli algoritmi 1-NN e K-means, le formule e gli schemi del capitolo.
- Capitolo *Classification, evaluation measures and KNN* di [6], per la parte sul K-Nearest Neighbor.
- I paper originali delle loss: «Learning a Similarity Metric Discriminatively, with Application to Face Verification» [2], «FaceNet: A Unified Embedding for Face Recognition and Clustering» [15], «Beyond Triplet Loss: A Deep Quadruplet Network for Person Re-identification» [1] e «A Discriminative Feature Learning Approach for Deep Face Recognition» [18].

Capitolo 6

Modelli Generativi

TODO: scrivere introduzione al capitolo

6.1 Supervised vs Unsupervised Learning

Esistono diversi modi di allenare un modello di ML/DL ma alla base di tutti i metodi di allenamento ci sono i dati. Spesso troviamo i dati in varie forme e i dataset possono essere di diversissima qualità. Ipotizziamo di avere un dataset di immagini di gatti e di cani, e vogliamo allenare un modello per distinguere tra le due classi. La prima ipotesi che uno si può fare è: alleniamo un modello di classificazione che fa un'ipotesi sulla classe di appartenenza dell'immagine, ovvero l'**etichetta**. Questo è un esempio di **supervised learning**, in cui il dataset contiene le immagini e le etichette corrispondenti e possiamo descrivere questo modello, dati i dati X e le etichette Y, come un modello che approssima la distribuzione condizionata

$$P(Y \mid X)$$

In questo caso, la categoria di modelli viene chiamata **Discriminativi** ed impara a predire l'etichetta Y data l'immagine X.

La classificazione non è l'unico task supervisionato. Oltre alla regressione, in cui l'etichetta è un numero reale, l'etichetta può avere una struttura più complessa:

Object detection Per ogni oggetto di interesse (ad esempio, ogni persona) il modello deve trovare la classe e il **bounding box**, ovvero il rettangolo che lo contiene. Il bounding box è descritto da quattro numeri (x, y, w, h), ovvero la posizione di un vertice, la larghezza e l'altezza[1]. Trovarlo è quindi un problema di regressione.

Semantic segmentation Il modello deve assegnare una classe a ogni pixel dell'immagine, ad esempio erba, gatto, albero o cielo.

Image captioning L'etichetta è una frase che descrive l'immagine, ad esempio "un gatto seduto su una valigia".

In tutti questi casi l'obiettivo è lo stesso: imparare una funzione che mappa l'input x nell'etichetta y.

[1] Qui x e y indicano le coordinate del rettangolo nell'immagine, non il dato e l'etichetta.

In generale, il supervised learning è utile quando abbiamo un dataset etichettato, ovvero quando abbiamo a disposizione le etichette per ogni dato di input. Tuttavia, in molti casi non abbiamo dataset etichettati, o le etichette si possono ottenere ma sono lunghe e costose. Da questo è nato il concetto di **unsupervised learning**, in cui il dataset contiene solo i dati X che formano una certa distribuzione di probabilità. Esistono diverse categorie di modelli di unsupervised learning:

Clustering In questo caso, il modello cerca di raggruppare i dati in base a delle caratteristiche simili per formare delle "etichette" in modo automatico (non supervisionato). Un esempio di clustering è il K-means, che raggruppa i dati in K cluster in base alla distanza tra i punti.

Dimensionality Reduction Qui il modello cerca di ridurre la dimensionalità dei dati, mantenendo le caratteristiche più importanti. Un esempio di dimensionality reduction è il PCA (Principal Component Analysis), che cerca di trovare le componenti principali dei dati e proiettarli in uno spazio a bassa dimensionalità.

Feature Learning Questa è una variante interessante di unsupervised learning, dove il modello non cerca di imparare a predire un'etichetta o a raggruppare i dati, ma cerca di imparare delle rappresentazioni dei dati che possano essere utili per altri compiti. Un esempio di feature learning è l'autoencoder, che cerca di comprimere i dati in una rappresentazione più compatta e poi ricostruirli.

Density Estimation In questo caso, il modello cerca di stimare la distribuzione di probabilità dei dati. Un esempio di density estimation è il GMM (Gaussian Mixture Model), che cerca di modellare la distribuzione dei dati come una combinazione di distribuzioni gaussiane.

Un esempio che verrà studiato in questo capitolo sono i **modelli generativi**. A differenza dei modelli discriminativi non apprendo l'etichetta a partire dai dati, ma cercano di "imitare" la distribuzione dei dati stessi. In altre parole, un modello generativo cerca di imparare la distribuzione di probabilità dei dati

$$P(X)$$

Riprendendo l'esempio dei gatti e dei cani, un modello generativo cerca di imparare la distribuzione delle immagini di gatti e cani e ogni immagine compete tra loro all'interno della distribuzione $P(x)$. In altre parole, poiché il modello deve imitare una distribuzione e quindi deve valere $\int_x P(x)dx = 1$, non può assegnare più probabilità a un'immagine irrealistica, perché questo significherebbe che la probabilità delle immagini realistiche sarebbe più bassa.

Questo rende $P(X)$ molto più difficile da imparare di $P(Y \mid X)$: per distribuire la massa di probabilità in modo sensato, il modello deve avere una comprensione profonda delle immagini. Ad esempio, è più probabile un cane seduto o un cane in piedi? E un cane con tre zampe o una scimmia con tre braccia? Un modello discriminativo non deve porsi nessuna di queste domande, perché gli basta capire se nell'immagine c'è un gatto o un cane.

Esiste inoltre un'altra categoria di modelli generativi, chiamati **modelli generativi condizionati**, che cercano di imparare la distribuzione condizionata dei dati rispetto a un'etichetta, ovvero

$$P(X \mid Y)$$

6.1.1 Regola di Bayes applicata ai modelli generativi

La regola di Bayes è un teorema fondamentale della probabilità che permette di calcolare la probabilità condizionata di un evento dato un altro evento. In particolare, la regola di Bayes afferma che:

$$P(x \mid y) = \frac{P(y \mid x)P(x)}{P(y)}$$

Nei modelli generativi la regola di Bayes viene usata per classificare: si modellano la likelihood $P(x \mid y)$ e la prior $P(y)$, e da queste si ricava la posterior $P(y \mid x)$ [6]. Nei modelli generativi possiamo leggere la stessa formula al contrario, perché a destra compaiono le distribuzioni viste finora (figura 6.1). In particolare, $P(y \mid x)$ è un modello discriminativo, $P(x)$ è un modello generativo e $P(y)$ è la prior, che si stima contando quante volte compare ogni etichetta nel dataset. L'equazione mostra che è possibile derivare la capacità di generare campioni appartenenti a una specifica classe, ovvero $P(x \mid y)$, combinando la conoscenza su come classificare i dati con la conoscenza della struttura dei dati stessi. Il problema, quindi, sta nel riuscire a modellare efficacemente i componenti necessari.

La Figura 6.1 mostra come la massa di probabilità compete nei tre tipi di modelli, con lo spazio delle immagini ridotto per semplicità a quattro immagini. Nel modello discriminativo competono le etichette di ogni immagine: le tre barre di un'immagine sommano a 1, anche per il rumore. Nel modello generativo competono tutte le immagini. Nel modello generativo condizionato, fissata l'etichetta $Y = \text{cane}$, competono di nuovo tutte le immagini, e le probabilità si ricavano dalle altre due con il teorema di Bayes.

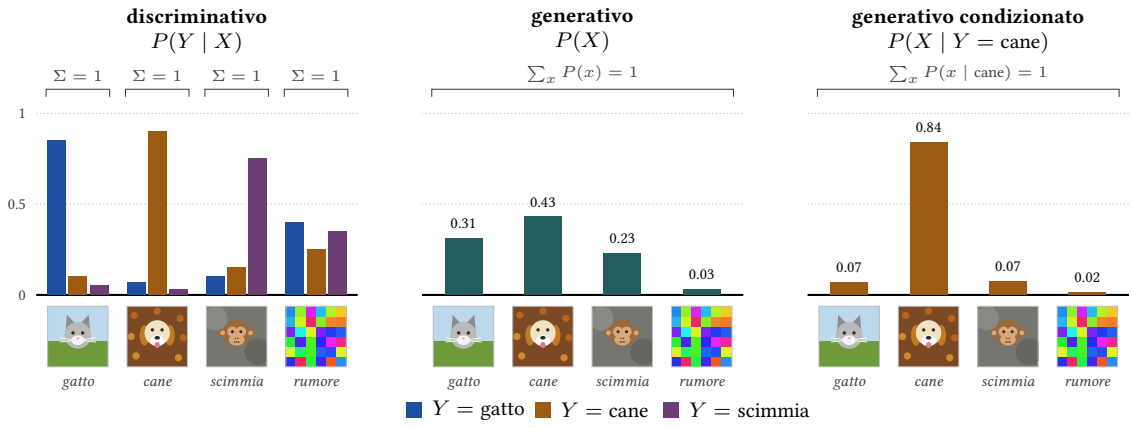


Figura 6.1. Cosa compete per la massa di probabilità nei tre modelli, con lo spazio delle immagini ridotto per semplicità a quattro immagini. Nel modello discriminativo competono le etichette di ogni immagine: le tre barre di un'immagine sommano a 1, anche per il rumore. Nel modello generativo competono tutte le immagini. Nel modello generativo condizionato, fissata l'etichetta $Y = \text{cane}$, competono di nuovo tutte le immagini, e le probabilità si ricavano dalle altre due con il teorema di Bayes.

6.1.2 Tassonomia dei modelli generativi

Un modello generativo può servire a generare nuovi dati: se ha imparato $P(x)$, può campionare immagini che non sono nel dataset ma che le imitano. Un modello generativo condizionato, invece,

è utile anche per assegnare etichette rigettando gli outlier. Un'immagine con $P(x \mid y)$ bassa per tutte le etichette, come il rumore della figura 6.1, non appartiene a nessuna classe e può essere scartata.

Esistono diverse famiglie di modelli generativi, che si distinguono per il modo in cui trattano la densità $p(x)$ (figura 6.2). I modelli a **densità esplicita** definiscono $p(x)$ in modo esplicito. In alcuni casi la densità si calcola direttamente (tractable density), come nei modelli autoregressivi. In altri si può solo approssimare (approximate density), come nei Variational Autoencoder e nei Diffusion Models.

I modelli a **densità implicita**, invece, non definiscono mai $p(x)$: imparano soltanto a campionare da essa. Il campionamento può essere diretto, come nelle Generative Adversarial Network (GAN), oppure passare da una catena di Markov, come nelle Generative Stochastic Network (GSN).

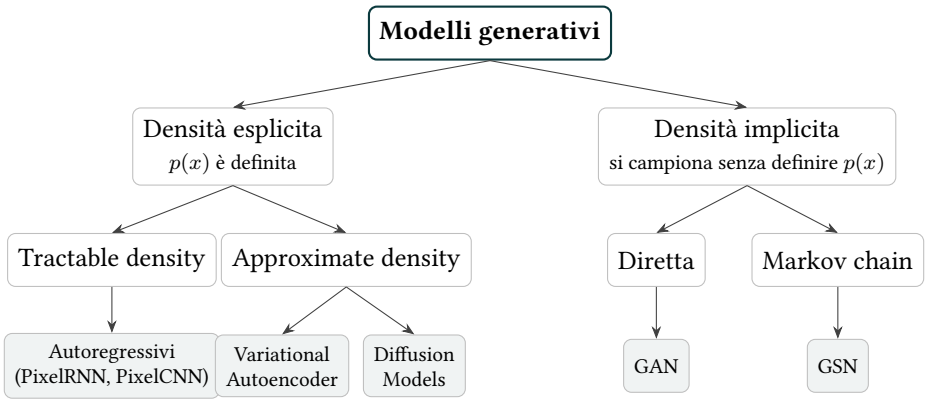


Figura 6.2. Tassonomia dei modelli generativi. I modelli a densità esplicita definiscono $p(x)$, e la calcolano direttamente o la approssimano. I modelli a densità implicita imparano solo a campionare da $p(x)$.

6.2 Modelli autoregressivi

I modelli autoregressivi sono modelli a densità esplicita: l'obiettivo è definire una funzione parametrizzata $f_W(x)$ che rappresenti la distribuzione $p(x)$, dove W è l'insieme dei parametri (pesi) del modello.

Il training avviene con la **Maximum Likelihood Estimation** (MLE): cerchiamo i pesi che rendono più probabili i dati del training set. Dato un dataset $X = \{x^{(i)}\}_{i=1}^{m}$, risolviamo il seguente problema di ottimizzazione:

$$\begin{aligned} W^* &= \arg\max_W \prod_i p(x^{(i)}) \\ &= \arg\max_W \sum_i \log p(x^{(i)}) \\ &= \arg\max_W \sum_i \log f_W(x^{(i)}) \end{aligned}$$

dove il passaggio al logaritmo non cambia il massimo, in quanto il logaritmo è una funzione crescente, e trasforma il prodotto in una somma, più semplice da derivare.

Resta il problema di come calcolare $p(x)$. Pensiamo a un'immagine: è un vettore di pixel $x = (x_1, \ldots, x_n)$, e $p(x)$ è la probabilità congiunta di tutti i suoi pixel. I modelli autoregressivi la rendono trattabile scomponendola con la chain rule:

$$p(x) = p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1})$$

dove ogni fattore è la probabilità di un pixel dati tutti i pixel che lo precedono. In pratica, l'immagine viene generata un pixel alla volta, e ogni nuovo pixel dipende da quelli già generati. Ad esempio, un'immagine 28×28 in scala di grigi ha 784 pixel, quindi 784 fattori.

Ogni fattore $p(x_i \mid x_1, \ldots, x_{i-1})$ dipende da un numero di pixel che cresce con i: il primo pixel non dipende da niente, l'ultimo dipende da tutti gli altri 783. Un MLP non è opportuno, perché richiede un input di dimensione fissa. Si potrebbe pensare di utilizzare un MLP diverso per ogni pixel, ma servirebbero 784 reti, ognuna con i propri pesi e addestrata su un solo pixel per immagine. Serve quindi un'architettura che **condivida i pesi** tra tutti i pixel, in modo che una sola rete modelli tutti i fattori. Le RNN e le CNN hanno proprio questa proprietà.

PixelRNN e PixelCNN. PixelRNN e PixelCNN sono due modi per realizzare f_W: la rete riceve i pixel già generati e restituisce la distribuzione del pixel successivo, ad esempio con una softmax sui 256 valori di intensità. Il prodotto di queste probabilità è $p(x)$, e massimizzarne il logaritmo equivale a una cross-entropy su ogni pixel.

La **PixelRNN** [11] utilizza una RNN (Recurrent Neural Network), ovvero una rete che viene applicata in modo ricorrente: l'output di un passo diventa l'input del passo successivo. A ogni passo utilizza gli stessi pesi, quindi gestisce senza problemi sequenze di lunghezza variabile.

L'immagine viene generata un pixel alla volta, partendo dall'angolo in alto a sinistra. Per ogni pixel la rete calcola uno stato nascosto (hidden state), che dipende dallo stato del pixel a sinistra e da quello del pixel sopra (oltre che dai loro colori):

$$h_{i,j} = f(h_{i,j-1}, h_{i-1,j}, W)$$

dove i e j sono la riga e la colonna del pixel, e f è una LSTM (un tipo particolare di RNN) con pesi W.

Lo stato del pixel sopra dipende a sua volta dagli stati sopra e a sinistra di quel pixel, e così via. Ogni pixel dipende quindi **implicitamente** da tutti i pixel sopra e a sinistra.

A partire dallo stato nascosto, la rete predice il colore del pixel un canale alla volta: prima il rosso, poi il verde (che dipende dal rosso) e infine il blu (che dipende da entrambi), ognuno con una softmax sui valori da 0 a 255.

Il problema è la lentezza. Per calcolare $h_{i,j}$ servono gli stati a sinistra e sopra, quindi si possono calcolare insieme solo i pixel sulla stessa antidiagonale, ovvero con la stessa somma $i + j$ (figura 6.3). Un'immagine $N \times N$ ha $2N - 1$ antidiagonali, e richiede quindi $2N - 1$ passi sequenziali, sia in training che in generazione. Ad esempio, per un'immagine 32×32 servono 63 passi, ognuno dei quali deve aspettare il precedente.

La **PixelCNN** [12] sostituisce la RNN con delle convoluzioni, che guardano una regione di pixel già generati attorno a quello da predire (con un'opportuna maschera, in modo che ogni pixel veda solo quelli precedenti). Lo stesso kernel scorre su tutte le posizioni, come visto per il

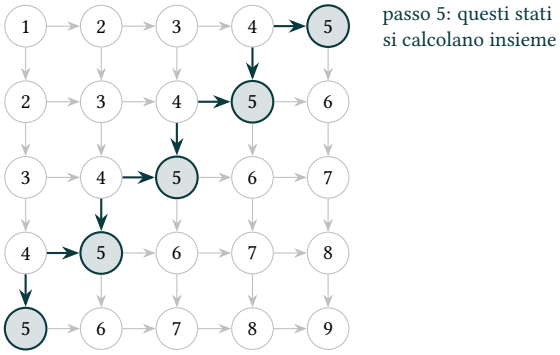


Figura 6.3. Ordine di calcolo della PixelRNN su un'immagine 5×5. Ogni cerchio è lo stato $h_{i,j}$ di un pixel, e il numero indica il passo in cui viene calcolato. Ogni stato dipende da quello a sinistra e da quello sopra, quindi gli stati sulla stessa antidiagonale si calcolano insieme: sono evidenziati quelli del passo 5, con le frecce da cui dipendono. In totale servono $2N - 1 = 9$ passi.

parameter sharing nel capitolo 3, e sfrutta il fatto che in un'immagine i pixel vicini sono i più correlati. Il prezzo è che ogni pixel vede solo un intorno, e non tutti i pixel precedenti come nella RNN.

In cambio, il training è più veloce: in fase di training l'immagine è nota per intero, quindi le convoluzioni si calcolano in parallelo su tutti i pixel. La generazione, invece, resta sequenziale, perché per generare un pixel servono quelli precedenti.

6.3 Autoencoder

Un **autoencoder** è un'architettura neurale che impara delle feature dai dati grezzi senza utilizzare etichette. L'idea è banale: prendiamo l'input, lo comprimiamo in una rappresentazione più piccola (perdendo informazione) e proviamo a ricostruirlo a partire da questa.

Perché passare dalla ricostruzione? Vorremmo delle feature utili per qualche task, ma queste feature non si possono osservare direttamente: non abbiamo un target con cui confrontarle. Possiamo risolvere il problema utilizzando come target l'input stesso. L'architettura è divisa in due parti (figura 6.4):

- **Encoder** f_W: trasforma l'input x nel codice $h = f_W(x)$, ovvero la sua rappresentazione nello spazio latente.
- **Decoder** g_{W^*}: a partire dal codice, prova a ricostruire l'input.

Encoder e decoder hanno ciascuno i propri parametri, W e W^*. La ricostruzione di x è quindi:

$$\hat{x} = g_{W^*}(h) = g_{W^*}(f_W(x))$$

dove $\hat{x}$ deve essere il più possibile simile a x. Se il codice è più piccolo dell'input, l'encoder è costretto a tenere solo le informazioni che servono davvero per la ricostruzione, e queste sono le feature che cerchiamo. Encoder e decoder sono di solito simmetrici, e possono essere dei semplici layer fully connected oppure dei layer convoluzionali.

Dopo il training, il decoder si può buttare via. L'encoder ha imparato una rappresentazione dei dati, e quindi si può utilizzare come **feature extractor**: ad esempio, possiamo mettere un

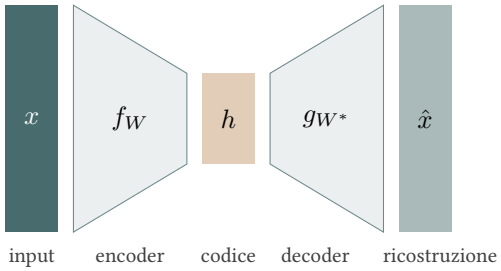


Figura 6.4. Schema di un autoencoder: l'encoder f_W comprime l'input x nel codice h, e il decoder g_{W^*} prova a ricostruire l'input a partire dal codice.

classificatore sopra l'encoder (figura 6.5). Il vantaggio è che le feature si imparano senza etichette. Per il fine-tuning di encoder e classificatore basta quindi un dataset etichettato molto più piccolo di quello che servirebbe partendo da zero. È la stessa idea del transfer learning (capitolo 4), con la differenza che qui il pre-training non utilizza etichette.

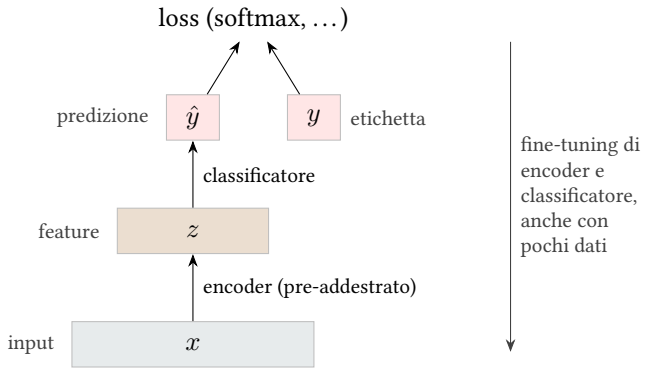


Figura 6.5. Un classificatore costruito sopra l'encoder di un autoencoder, che fa da feature extractor. Encoder e classificatore vengono poi addestrati insieme sul task finale, anche con pochi dati etichettati.

Gli autoencoder hanno diverse applicazioni pratiche, in cui l'input viene trasformato in una versione "migliore" di sé stesso:

- **Miglioramento della qualità** di un'immagine, ad esempio togliendo il rumore.
- **Inpainting**: ricostruire le parti mancanti o coperte di un'immagine, ad esempio togliendo delle scritte.
- **Recolorization**: colorare immagini in bianco e nero.
- **Semantic segmentation**: assegnare una classe a ogni pixel, con un'architettura encoder-decoder che produce una mappa grande quanto l'input.

Un'altra applicazione è l'**anomaly detection**, ovvero il riconoscimento degli input anomali (outlier). L'autoencoder viene addestrato solo su dati normali, quindi impara a ricostruire bene gli input simili a quelli del training set, mentre un input anomalo viene ricostruito male. L'errore di ricostruzione fa da punteggio di anomalia: se supera una certa soglia, l'input viene segnalato come anomalo.

Ad esempio, un autoencoder addestrato su foto di pezzi prodotti correttamente da una fabbrica ricostruisce male le foto dei pezzi difettosi. In alternativa si può lavorare sui codici: si raggruppano in cluster i codici dei dati, e un input con un codice lontano da tutti gli altri viene considerato un outlier.

6.3.1 Un esempio: l'autoencoder lineare

Consideriamo un input $x \in \mathbb{R}^D$ e un singolo hidden layer, il cui output è il codice $h \in \mathbb{R}^k$. Il caso più semplice è un autoencoder **lineare**:

$$h = f_W(x) = Wx + b, \qquad \hat{x} = g_{W^*}(h) = W^*h + c = W^*(Wx + b) + c$$

dove $W \in \mathbb{R}^{k \times D}$ e $b \in \mathbb{R}^k$ sono i parametri dell'encoder, mentre $W^* \in \mathbb{R}^{D \times k}$ e $c \in \mathbb{R}^D$ sono quelli del decoder. Le dimensioni tornano: W porta un vettore di D componenti in uno di k, e W^* fa il viaggio inverso.

Vediamo un esempio con $D = 3$ e $k = 2$ (figura 6.6). L'encoder ha una matrice W di dimensione 2×3 e un bias b con 2 componenti:

$$\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

dove w_{jd} è il peso dell'arco che collega l'input x_d al neurone h_j del codice. La riga j di W contiene quindi i pesi degli archi che entrano in h_j, e la colonna d quelli che escono da x_d. Il bias b_j, invece, è il peso dell'arco che arriva in h_j da un'unità sempre uguale a 1, come visto nel capitolo 1.

Il decoder ha invece una matrice W^* di dimensione 3×2 e un bias c con 3 componenti:

$$\begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \\ \hat{x}_3 \end{bmatrix} = \begin{bmatrix} w^*_{11} & w^*_{12} \\ w^*_{21} & w^*_{22} \\ w^*_{31} & w^*_{32} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} + \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

dove w^*_{dj} è il peso dell'arco che collega h_j all'output $\hat{x}_d$, e c_d quello dell'arco che arriva in $\hat{x}_d$ dall'unità di bias. In totale l'encoder ha $6 + 2 = 8$ parametri e il decoder $6 + 3 = 9$.

Se addestriamo encoder e decoder minimizzando l'errore quadratico $\|\hat{x} - x\|_2^2$, si può dimostrare che la soluzione ottima si ottiene con i **pesi legati** (tied weights), ovvero $W^* = W^T$. Questo autoencoder è equivalente alla **PCA** (Principal Component Analysis): le righe di W generano lo stesso sottospazio delle prime k componenti principali dei dati. Nell'esempio con $D = 3$ e $k = 2$, i pesi legati significano $w^*_{dj} = w_{jd}$: l'arco da x_d a h_j e quello da h_j a $\hat{x}_d$ hanno lo stesso peso.

Perché i pesi legati. L'encoder proietta x su un sottospazio di dimensione k, generato dalle righe di W. Minimizzando l'errore quadratico, la ricostruzione migliore è la **proiezione ortogonale** di x su questo sottospazio. Se le righe di W sono una base ortonormale, come le componenti principali della PCA, la trasposta W^T inverte la proiezione: riporta il codice nello spazio di partenza senza deformarlo. In altre parole, encoder e decoder lavorano nello stesso spazio vettoriale.

La figura 6.7 mostra un autoencoder lineare con pesi legati, addestrato su immagini di cifre scritte a mano di 8×8 pixel (quindi $D = 64$).

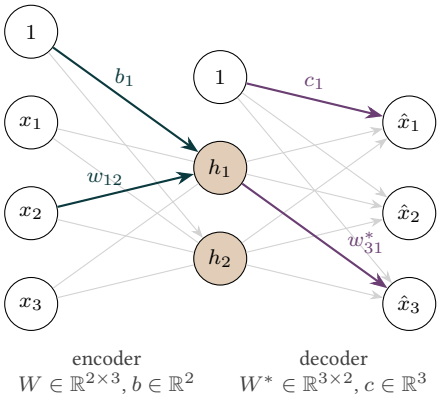


Figura 6.6. Autoencoder lineare con $D = 3$ e $k = 2$. I nodi con valore 1 sono le unità di bias: l'arco che va da 1 a h_j ha peso b_j, quello che va da 1 a $\hat{x}_d$ ha peso c_d. Il peso w_{12} collega l'input x_2 al neurone h_1 del codice, e si trova nella riga 1 e nella colonna 2 di W. Il peso w^*_{31} collega h_1 all'output $\hat{x}_3$, e si trova nella riga 3 e nella colonna 1 di W^*.

Il codice h ha $k = 8$ componenti, e ogni componente dice quanto una delle feature (le righe di W) è presente nell'input. Il decoder parte dal bias c, che è l'immagine media del dataset[2], e somma le feature pesate dal codice:

$$\hat{x} = c + W^T h = c + \sum_{j=1}^{k} h_j w_j$$

dove w_j è la j-esima riga di W. Più k è grande, più la ricostruzione è fedele: con $k = 64$ è perfetta, ma il codice non comprime più niente.

Autoencoder non lineare. Come in un MLP, possiamo aggiungere una funzione di attivazione non lineare dopo il layer dell'encoder, ad esempio una sigmoide:

$$h = f_W(x) = \sigma(Wx + b)$$

Sul decoder bisogna invece fare attenzione. Se l'input ha valori reali, non possiamo applicare una sigmoide all'output, perché la ricostruzione sarebbe limitata tra 0 e 1. Se invece l'input è binario, ovvero $x \in \{0, 1\}^D$, la sigmoide è proprio la scelta opportuna:

$$\hat{x} = g_{W^*}(h) = \sigma(W^* h + c)$$

dove $\sigma(z) \in [0, 1]$ si può interpretare come la probabilità che ogni componente valga 1.

6.3.2 Deep autoencoder

Finora encoder e decoder avevano un solo layer. Come in un MLP, possiamo utilizzarne di più: si parla allora di **deep autoencoder**, o **stacked autoencoder**[3]. L'encoder riduce la dimensione un

[2] In questo caso $b = -Wc$, quindi $h = W(x - c)$: l'encoder misura quanto l'input si discosta dall'immagine media lungo ogni feature.

[3] Il nome viene dal fatto che, inizialmente, questi autoencoder venivano addestrati un layer alla volta, impilando (stacking) degli autoencoder con un solo hidden layer.

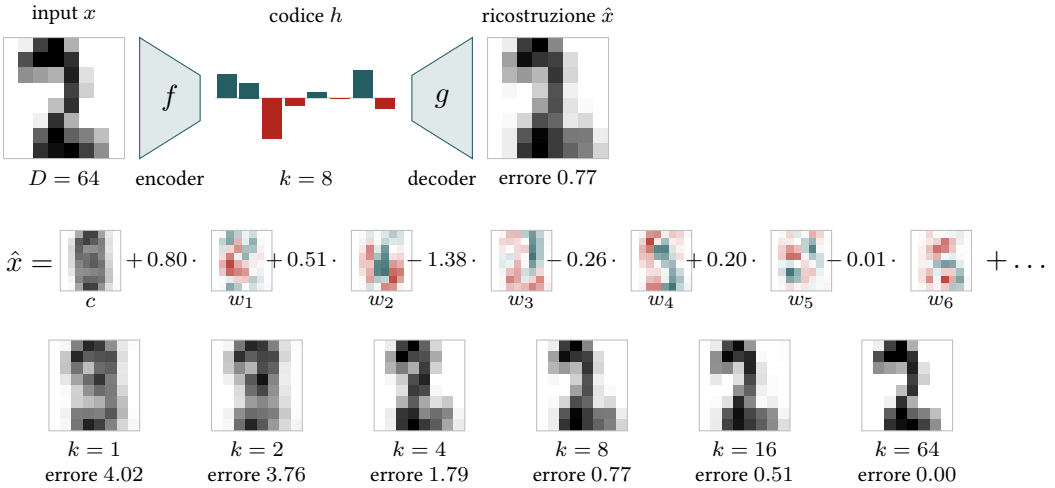


Figura 6.7. Un autoencoder lineare con pesi legati sul dataset *digits* di scikit-learn (1797 cifre di 8×8 pixel), con i pesi ottimi, ovvero le componenti principali del dataset. La cifra 2 viene codificata in $k = 8$ numeri (in rosso i valori negativi) e ricostruita dal decoder come l'immagine media c più le feature w_j pesate dal codice. Al crescere di k la ricostruzione migliora, fino a diventare esatta con $k = D = 64$.

layer alla volta, fino al codice, e il decoder fa il percorso inverso, di solito in modo simmetrico.

Ad esempio, la figura 6.8 mostra un deep autoencoder per documenti di testo. L'input è un vettore con il numero di occorrenze di 2000 parole nel documento, e l'encoder lo comprime in 500, 250 e infine 10 numeri. A partire da questi 10 numeri, il decoder deve ricostruire tutti i 2000 conteggi. Il codice è quindi una rappresentazione molto compatta del documento, che si può utilizzare, ad esempio, per confrontare due documenti.

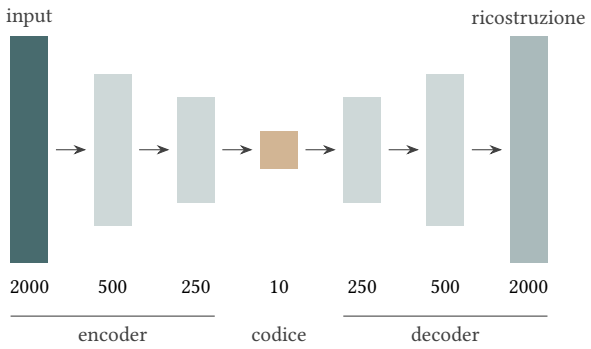


Figura 6.8. Un deep autoencoder per documenti di testo. L'input contiene i conteggi di 2000 parole, e l'encoder lo comprime fino a un codice di 10 numeri passando da layer di 500 e 250 neuroni. Il decoder, simmetrico all'encoder, ricostruisce i 2000 conteggi a partire dal codice.

Con più layer, e funzioni di attivazione non lineari, encoder e decoder possono rappresentare trasformazioni molto più complesse di quella lineare vista prima. Questo, però, aumenta la capacità della rete, e come vedremo nella sezione 6.3.4 una rete troppo potente rischia di imparare a ricostruire l'input senza imparare feature utili.

6.3.3 Training dell'autoencoder

Per trovare i pesi W e W^* serve una loss function $L(x, \hat{x})$ che misuri l'errore di ricostruzione, ovvero la differenza tra l'input x e la sua ricostruzione $\hat{x}$. Il problema di ottimizzazione è:

$$\min_{W,W^*} L(x, \hat{x}) = \min_{W,W^*} L\big(x, g_{W^*}(f_W(x))\big)$$

La scelta di L dipende dal tipo di input, come per il decoder. Con input a valori reali si utilizza la squared loss:

$$L(x, \hat{x}) = \|\hat{x} - x\|_2^2 = \sum_{d=1}^{D} (\hat{x}_d - x_d)^2$$

dove D è la dimensione dell'input. Con input binari si utilizza invece la cross-entropy:

$$L(x, \hat{x}) = -\sum_{d=1}^{D} \big(x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d)\big)$$

che è la loss della regressione logistica, applicata a ogni componente dell'input.

In pratica, si minimizza la loss media sul training set, $\frac{1}{m} \sum_{i=1}^{m} L(x^{(i)}, \hat{x}^{(i)})$, con la backpropagation e la discesa del gradiente. Osserviamo che non servono etichette: il target di ogni esempio è l'esempio stesso.

6.3.4 Undercomplete e overcomplete

Progettare un autoencoder significa bilanciare la capacità di encoder e decoder con la dimensione k del codice. Dato un input di dimensione D, si distinguono due casi (figura 6.9): autoencoder **undercomplete**, con $k < D$, e **overcomplete**, con $k \geq D$.

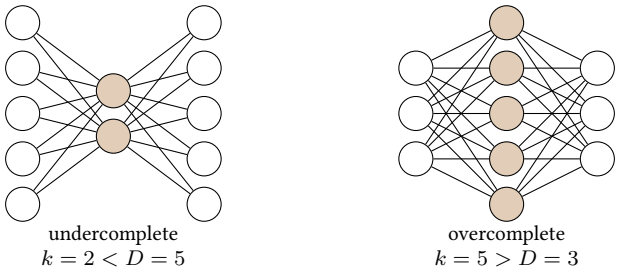


Figura 6.9. Autoencoder undercomplete, con il codice (i neuroni colorati) più piccolo dell'input, e overcomplete, con il codice più grande dell'input.

Nel caso undercomplete il codice fa da **collo di bottiglia** (bottleneck): l'informazione deve passare da D a k numeri, quindi la rete deve scegliere cosa tenere. È il caso della figura 6.7. La matrice W impara dei pattern ricorrenti nei dati, la cui combinazione lineare serve a ricostruire l'input, e per ogni x il codice h dice quanto è presente ciascuna delle k feature.

Il collo di bottiglia, da solo, non basta. Se encoder e decoder sono molto complessi (ad esempio, con tanti layer), la rete può ottenere un errore di ricostruzione basso anche con k piccolo, senza catturare proprietà interessanti dei dati. Al limite, con $k = 1$ potrebbe imparare a dare un indice a

ogni esempio del training set e a ricostruirlo a partire dall'indice [7]. In questo caso l'autoencoder è utile per la compressione, ma molto meno per estrarre feature.

Nel caso overcomplete il problema è ancora più evidente. Se il codice ha almeno la dimensione dell'input, la rete può raggiungere un errore nullo semplicemente copiando l'input nell'output, ad esempio con f e g uguali alla funzione identità. Il codice, in questo caso, non cattura nessuna proprietà dei dati (nella figura 6.7 succede con $k = 64$).

In entrambi i casi, quindi, l'errore di ricostruzione non basta: bisogna vincolare quello che la rete può imparare. In altre parole, un autoencoder tende ad andare in overfitting (impara "troppo bene" a copiare i dati), e va regolarizzato.

6.4 Autoencoder regolarizzati

Per evitare che un autoencoder con troppa capacità impari la funzione identità (soprattutto nel caso overcomplete), si aggiunge alla loss un **termine di regolarizzazione** R. La loss diventa quindi:

$$L(x, \hat{x}) + R$$

dove R penalizza le rappresentazioni che non vogliamo, e costringe la rete a imparare caratteristiche utili della distribuzione dei dati invece di una semplice copia dell'input. Vediamo tre modi per farlo.

6.4.1 Sparse autoencoder

Uno **sparse autoencoder** impone un vincolo di sparsità sul codice h: per ogni input, solo pochi neuroni del codice possono essere attivi (diversi da zero). Anche con $k \geq D$, quindi, ogni input viene descritto da poche feature, e la rete non può limitarsi a copiarlo. Ci sono due modi principali per ottenere la sparsità:

- **Penalità** L_1: si aggiunge alla loss la somma dei valori assoluti delle attivazioni del codice:

  $$R(h) = \lambda \sum_{j=1}^{k} |h_j|$$

  dove λ regola il peso della regolarizzazione. Osserviamo che la penalità agisce sul codice h, non sui pesi (a differenza della regolarizzazione L_1 classica), e porta a zero il contributo dei neuroni meno rilevanti.

- **Penalità KL**: con attivazioni del codice in $[0, 1]$ (ad esempio, con una sigmoide), si fissa un'attivazione media target p molto bassa, ad esempio 0.05. Si penalizza poi la distanza tra p e l'attivazione media $\hat{p}_j$ di ogni neurone, misurata con la KL divergence:

  $$R(h) = \sum_{j=1}^{k} D_{KL}(p \,\|\, \hat{p}_j) = \sum_{j=1}^{k} \left(p \log \frac{p}{\hat{p}_j} + (1-p) \log \frac{1-p}{1-\hat{p}_j} \right)$$

  dove $\hat{p}_j = \frac{1}{m} \sum_{i=1}^{m} h_j(x^{(i)})$ è l'attivazione media del neurone j sul training set. In pratica, ogni neurone del codice deve restare spento per la maggior parte degli input.

6.4.2 Denoising autoencoder

Un **denoising autoencoder** (DAE) cambia il task: invece di ricostruire l'input, deve ripulirlo. Il training funziona così:

1. l'input x viene corrotto con un processo casuale (ad esempio, aggiungendo rumore gaussiano), ottenendo $\tilde{x}$;
2. l'encoder codifica l'input corrotto, $h = f(\tilde{x})$;
3. il decoder deve ricostruire l'input **originale** x, non quello corrotto.

La loss confronta quindi la ricostruzione con l'input pulito, in media su tutte le possibili corruzioni:

$$L_{DAE} = \mathbb{E}_{\tilde{x} \sim p(\tilde{x}|x)} \big[L(x, g(f(\tilde{x}))) \big]$$

dove $p(\tilde{x} \mid x)$ è il processo di corruzione. Copiare l'input, qui, non serve: la copia di $\tilde{x}$ contiene ancora il rumore. La rete deve quindi imparare la struttura dei dati, per capire quale parte dell'input è rumore e quale no. È lo stesso principio di applicazioni come l'inpainting viste prima.

6.4.3 Contractive autoencoder

Mentre il DAE rende robusta la ricostruzione rispetto a input corrotti, un **contractive autoencoder** (CAE) rende robusto il codice: se l'input cambia di poco, anche il codice $h = f(x)$ deve cambiare di poco. La sensibilità dell'encoder si misura con la sua matrice Jacobiana, ovvero la matrice delle derivate parziali $\partial h_j / \partial x_d$, e la loss la penalizza tramite la norma di Frobenius:

$$L_{CAE} = L(x, \hat{x}) + \lambda \left\| \frac{\partial f(x)}{\partial x} \right\|_F^2$$

dove $\|J\|_F^2 = \sum_{j,d} J_{jd}^2$ è la somma dei quadrati di tutti gli elementi della matrice J.

Il termine di regolarizzazione spinge il codice a essere localmente piatto, mentre l'errore di ricostruzione lo costringe a cambiare lungo le direzioni in cui variano davvero i dati. Il risultato è un codice che cattura solo le variazioni importanti dei dati.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione (5 Novembre 2026)
- Slide del corso *Generative Models: Autoregressive Models* (credit Justin Johnson), le slide sugli autoencoder e le note scritte a mano del corso.
- Appunti di Daniele Materia, capitolo 6 fino agli autoencoder regolarizzati: `https://danielemateria.github.io`.
- Capitolo *Generative classifiers: QDA, LDA and naive Bayes* di [6], per la regola di Bayes nei classificatori generativi.
- I paper originali di PixelRNN e PixelCNN: «Pixel Recurrent Neural Networks» [11] e «Conditional Image Generation with PixelCNN Decoders» [12].

- Capitolo 14, *Autoencoders*, del libro *Deep Learning* [7], per il rischio di capacità eccessiva negli autoencoder undercomplete.
- Il dataset *digits* di scikit-learn per il widget dell'autoencoder lineare: `https://scikit-learn.org/stable/datasets/toy_dataset.html`.

Bibliografia

[1] Weihua Chen et al. «Beyond Triplet Loss: A Deep Quadruplet Network for Person Re-identification». In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

[2] Sumit Chopra, Raia Hadsell e Yann LeCun. «Learning a Similarity Metric Discriminatively, with Application to Face Verification». In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2005.

[3] Jiankang Deng et al. «ArcFace: Additive Angular Margin Loss for Deep Face Recognition». In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.

[4] Jiankang Deng et al. «Sub-center ArcFace: Boosting Face Recognition by Large-Scale Noisy Web Faces». In: *European Conference on Computer Vision (ECCV)*. 2020.

[5] Vincent Dumoulin e Francesco Visin. *A guide to convolution arithmetic for deep learning*. 2016. arXiv: 1603.07285.

[6] Antonino Furnari. *Fundamentals of Data Analysis. From the question to the model, with Python*. 2026. URL: https://antoninofurnari.github.io/fad-2627/ (visitato il giorno 10/11/2026).

[7] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: http://www.deeplearningbook.org.

[8] Sergey Ioffe e Christian Szegedy. «Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift». In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research. 2015, pp. 448–456.

[9] Alex Krizhevsky, Ilya Sutskever e Geoffrey E. Hinton. «ImageNet Classification with Deep Convolutional Neural Networks». In: *Advances in Neural Information Processing Systems*. Vol. 25. 2012.

[10] Weiyang Liu et al. «SphereFace: Deep Hypersphere Embedding for Face Recognition». In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

[11] Aäron van den Oord, Nal Kalchbrenner e Koray Kavukcuoglu. «Pixel Recurrent Neural Networks». In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. 2016.

[12] Aäron van den Oord et al. «Conditional Image Generation with PixelCNN Decoders». In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016.

[13] Nils Reimers e Iryna Gurevych. «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks». In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 2019. URL: https://arxiv.org/abs/1908.10084.

[14] Raúl Rojas. *Neural Networks: A Systematic Introduction*. Berlin, Heidelberg: Springer, 1996. ISBN: 978-3540605058.

[15] Florian Schroff, Dmitry Kalenichenko e James Philbin. «FaceNet: A Unified Embedding for Face Recognition and Clustering». In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015.

[16] Nitish Srivastava et al. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». In: *Journal of Machine Learning Research* 15.56 (2014), pp. 1929–1958.

[17] Hao Wang et al. «CosFace: Large Margin Cosine Loss for Deep Face Recognition». In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018.

[18] Yandong Wen et al. «A Discriminative Feature Learning Approach for Deep Face Recognition». In: *European Conference on Computer Vision (ECCV)*. 2016.